

# 硕士学位论文

(专业学位论文)

基于 PCG 的关卡生成游戏系统设计与实现

**DESIGN AND IMPLEMENTATION OF A  
PCG-BASED LEVEL GENERATION GAME  
SYSTEM**

王鹏云

哈尔滨工业大学

2025 年 1 月

国内图书分类号：TN915.08  
国际图书分类号：004.8

学校代码：10213  
密级：公开

## 硕士学位论文

# 基于 PCG 的关卡生成游戏系统设计与实现

硕 士 研 究 生：王鹏云

导 师：柏军副教授

申 请 学 位：专业学位

学 科：计算机技术

所 在 单 位：计算机科学与技术学院

答 辩 日 期：2025 年 1 月

授予学位单位：哈尔滨工业大学

Classified Index: TN915.08

U.D.C: 004.8

Dissertation for the Master Degree

**DESIGN AND IMPLEMENTATION OF A  
PCG-BASED LEVEL GENERATION GAME  
SYSTEM**

|                                       |                                              |
|---------------------------------------|----------------------------------------------|
| <b>Candidate:</b>                     | Wang Pengyun                                 |
| <b>Supervisor:</b>                    | Associate Prof. Bai Jun                      |
| <b>Academic Degree Applied for:</b>   | Professional Degrees                         |
| <b>Speciality:</b>                    | Computer Technology                          |
| <b>Affiliation:</b>                   | School of Computer Science and<br>Technology |
| <b>Date of Defence:</b>               | January, 2025                                |
| <b>Degree-Conferring-Institution:</b> | Harbin Institute of Technology               |

## 摘 要

随着电子游戏技术的迅速发展,游戏逐渐成为人们生活中受欢迎的娱乐方式。然而,玩家对游戏内容的消耗速度远超开发者的生产速度,如何提高游戏开发效率成为游戏行业面临的重要挑战。在游戏内容中,关卡是连接各种游戏元素的桥梁,直接影响玩家的游戏体验。然而,传统的关卡设计方法往往依赖人工构建,耗时较长。程序化内容生成(Procedural Content Generation, PCG)作为一种基于算法自动生成游戏内容的技术,能够有效减少人工设计的工作量,提高开发效率。本文围绕关卡自动化生成展开研究,重点探讨关卡修复、关卡生成及长关卡构建三个方向,旨在通过 PCG 技术提高游戏关卡设计的效率。

针对生成的关卡中常见的错误图块问题,本文提出了基于错误识别网络 LevelErrorNet 和遗传算法的关卡段修复方法。通过 LevelErrorNet 对关卡中的错误图块进行识别,然后再结合遗传算法对错误的图块进行替换修复。通过 KL 散度对修复前后的效果进行评价,实验结果表明关卡段修复方法在保留原有关卡的设计风格方面要优于现有的方法。

针对传统关卡生成方法存在的模板化问题,本文提出基于生成对抗网络 DCNN-DCGAN 和 QD 算法的关卡段生成方法。通过生成器和判别器的对抗训练,生成器能够生成符合游戏逻辑的关卡段,接着通过 CMA-ME 算法优化潜在向量搜索过程来提高生成关卡的质量。通过覆盖场景百分比和 KL 散度两个指标分别对生成关卡的可玩性和多样性进行评价,实验结果表明该方法在生成可玩且多样化的关卡方面具有不错的表现,和现有方法生成的关卡相比本方法生成的关卡更具有多样性。

针对现有生成方法仅能保证生成出的短关卡的可玩性,在长关卡生成方面表现不佳的问题,本文提出了基于强化学习的长关卡构建策略。通过设计多个奖励函数来优化关卡段的生成与拼接,保证生成的长关卡在可玩性、机制量和重复度上达到理想平衡。实验结果表明,同时考虑了三种奖励函数的代理在生成可玩且机制量和重复度适中的长关卡方面有着较好的表现。

最后,本文结合上述研究成果设计并实现了基于 PCG 的关卡生成游戏系统。该系统能够满足游戏开发中的实际需求,通过对其进行详细测试,验证了各功能模块的稳定性和可靠性。

**关键词:** 关卡设计; 关卡生成; 关卡修复; 强化学习; 程序化内容生成

## Abstract

With the rapid advancement of video game technology, games have become a popular form of entertainment in daily life. However, players' consumption speed of game content far exceeds developers' production capacity, making efficiency improvement a critical challenge for the gaming industry. As bridges connecting various game elements, levels directly influence player experience. Traditional manual design methods remain labor-intensive and time-consuming. Procedural Content Generation (PCG) addresses this gap by algorithmically automating content creation, significantly reducing manual workloads. This study focuses on automated level generation through PCG technology, specifically addressing three aspects: level repair, level generation, and long-level construction.

To resolve erroneous tile issues in generated levels, we propose a repair method combining LevelErrorNet (an error detection network) with genetic algorithms. LevelErrorNet identifies faulty tiles, while genetic algorithms replace them. Evaluations using Kullback-Leibler (KL) divergence demonstrate that our method better preserves original design styles compared to existing approaches.

To overcome the template-like limitations of conventional level generation methods, this paper proposes a segment-level generation approach based on Deep Convolutional Generative Adversarial Networks (DCNN-DCGAN) and Quality-Diversity (QD) algorithms. Through adversarial training between the generator and discriminator, the generator is capable of producing level segments that conform to game logic. Subsequently, the CMA-ME algorithm is employed to optimize the latent vector search process, further enhancing the quality of generated levels. Experimental evaluations using scenario coverage (for playability) and KL divergence (for diversity) demonstrate that this method effectively generates levels that are both playable and diverse, achieving superior diversity compared to existing approaches.

To tackle the challenge that existing generation methods primarily ensure playability in short levels but struggle with constructing coherent long levels, this paper further proposes a reinforcement learning-based long-level construction strategy. By designing multiple reward functions, the method guides the generation and concatenation of level segments to maintain a desirable balance between playability, mechanism density, and repetition. Experimental results indicate that agents incorporating all three types of rewards perform well in generating long levels that are not only playable but also exhibit appropriate mechanical richness and reduced redundancy.

Finally, we implement a PCG-based level generation system incorporating these methods. Comprehensive testing validates the stability and reliability of all functional modules, meeting practical game development requirements.

**Keywords:** level design, level generation, level repair, reinforcement learning, pcg

# 目 录

|                                   |    |
|-----------------------------------|----|
| 第 1 章 绪论.....                     | 1  |
| 1.1 课题研究的背景和意义.....               | 1  |
| 1.2 国内外研究现状.....                  | 2  |
| 1.2.1 关卡生成国内外研究现状.....            | 2  |
| 1.2.2 关卡修复国内外研究现状.....            | 3  |
| 1.3 本文的研究内容及组织结构 .....            | 4  |
| 第 2 章 关卡段修复方法研究 .....             | 6  |
| 2.1 错误识别网络的研究.....                | 6  |
| 2.2 错误图块修复算法.....                 | 8  |
| 2.2.1 初始种群优化阶段.....               | 8  |
| 2.2.2 迭代进化阶段.....                 | 9  |
| 2.3 实验分析.....                     | 10 |
| 2.3.1 数据集简介 .....                 | 10 |
| 2.3.2 参数设置.....                   | 11 |
| 2.3.3 评价指标.....                   | 11 |
| 2.3.4 实验对比与分析.....                | 12 |
| 2.4 本章小结.....                     | 16 |
| 第 3 章 关卡生成方法研究 .....              | 17 |
| 3.1 基于生成对抗网络和 QD 算法的关卡段生成方法 ..... | 17 |
| 3.1.1 关卡段生成对抗网络的研究.....           | 18 |
| 3.1.2 潜在向量搜索.....                 | 19 |
| 3.2 长关卡构建策略.....                  | 20 |
| 3.2.1 奖励函数设计.....                 | 21 |
| 3.2.2 RL 训练策略的研究.....             | 22 |
| 3.3 实验分析.....                     | 24 |
| 3.3.1 数据集简介 .....                 | 24 |
| 3.3.2 参数设置.....                   | 25 |
| 3.3.3 评价指标.....                   | 26 |
| 3.3.4 实验对比与分析.....                | 27 |
| 3.4 本章小结.....                     | 30 |
| 第 4 章 关卡生成游戏系统设计与实现 .....         | 32 |
| 4.1 关卡生成游戏系统需求分析 .....            | 32 |
| 4.1.1 关卡生成游戏系统功能性需求分析 .....       | 32 |
| 4.1.2 关卡生成游戏系统非功能性需求分析 .....      | 35 |

|                             |    |
|-----------------------------|----|
| 4.1.3 关卡生成游戏系统业务流程分析.....   | 35 |
| 4.2 关卡生成游戏系统架构.....         | 36 |
| 4.3 关卡生成游戏系统数据库构建 .....     | 38 |
| 4.4 关卡生成游戏系统功能模块设计与实现 ..... | 41 |
| 4.4.1 关卡修复模块.....           | 41 |
| 4.4.2 关卡生成模块.....           | 43 |
| 4.4.3 模型管理模块.....           | 47 |
| 4.4.4 关卡游玩模块.....           | 48 |
| 4.4.5 系统管理模块.....           | 52 |
| 4.5 本章小结.....               | 53 |
| 第5章 关卡生成游戏系统测试 .....        | 55 |
| 5.1 关卡生成游戏系统测试环境 .....      | 55 |
| 5.2 系统功能模块测试.....           | 55 |
| 5.2.1 关卡修复模块测试.....         | 55 |
| 5.2.2 关卡生成模块测试.....         | 58 |
| 5.2.3 模型管理模块测试.....         | 62 |
| 5.2.4 关卡游玩模块测试.....         | 63 |
| 5.2.5 系统管理模块测试.....         | 68 |
| 5.3 本章小结.....               | 70 |
| 结论.....                     | 71 |
| 参考文献.....                   | 72 |



## 第1章 绪 论

### 1.1 课题研究的背景和意义

伴随着互联网与计算机技术的快速发展，数字化与智能化已逐渐成为社会进步的核心趋势。在这一背景下，电子游戏作为一种深受大众喜爱的娱乐方式，正经历着前所未有的增长<sup>[1]</sup>。全球数亿玩家沉浸于虚拟世界中，通过游戏体验乐趣、挑战和成就感，推动了游戏产业的蓬勃发展。游戏内容作为游戏体验的核心基础，不仅直接影响玩家黏性，还对用户留存率和游戏市场竞争力起着至关重要的作用。

然而，随着玩家对游戏品质要求的不断提升，高质量游戏的开发变得日益复杂和昂贵<sup>[2]</sup>。当前，开发一款优质游戏往往需要规模庞大的专业团队投入数年时间，这不仅大幅增加了开发成本和周期，还带来了两大突出问题：一方面，能够取得商业成功的游戏在市场中所占比例逐渐缩小；另一方面，开发门槛的提高使得中小型团队和独立开发者面临更大的生存压力<sup>[3]</sup>。在游戏开发流程中，游戏设计师与艺术家的工作量占据了较大比重，他们负责设计玩法、场景、角色等创意内容，并进一步将这些构想转化为可玩的游戏内容。这一创作过程高度依赖人工创意，既耗时又成本高昂<sup>[4]</sup>。因此，如何在确保创意质量的前提下，有效降低开发成本和时间，成为游戏行业亟待解决的关键问题。

程序化内容生成（Procedural Content Generation, PCG）技术的兴起<sup>[5]</sup>，为游戏开发注入了全新的活力<sup>[6]</sup>。PCG 技术通过预设算法与规则，自动生成游戏内容，从而显著提升生产效率，使设计师能够更专注于核心设计工作<sup>[7]</sup>。PCG 技术的目标并非完全取代设计师和艺术家，而是通过智能算法辅助创意过程，实现人机协同创作。这种协作模式能够有效减轻设计师在重复性任务上的负担，提高设计与生产效率，为资源有限的小型团队和个人开发者提供更多可能性。在实际应用中，PCG 技术已逐渐展现出显著优势。过去依赖人工完成的繁琐任务，如关卡设计，如今可以通过 PCG 技术实现自动化与批量化生产<sup>[8]</sup>。此外，PCG 技术还能够为游戏开发带来意料之外的创意灵感，算法生成的内容往往能突破人类设计师的固有思维模式，激发出新的设计思路。

因此，研究基于 PCG 的平台跳跃游戏关卡生成系统，不仅具有重要的学术价值，还能为游戏开发技术的创新注入新动能。本研究将探索如何高效应用 PCG 技术，优化关卡生成过程，丰富游戏内容的多样性与趣味性，为未来游戏开发实践

提供有力支持。

## 1.2 国内外研究现状

### 1.2.1 关卡生成国内外研究现状

近年来,随着 PCG 技术的发展<sup>[9]</sup>,关卡生成在游戏设计中逐渐成为一个重要的研究方向<sup>[10]</sup>。许多学者提出了多种基于不同算法的关卡生成方法<sup>[11]</sup>,旨在提高关卡生成的效率与多样性,并且增强玩家体验。在赛车游戏领域,Togelius 等人<sup>[12]</sup>提出了基于适应度排序和变异操作的 Cascading Elitism 算法,用于生成个性化的 2D 赛道。该算法通过筛选赛道种群个体,并结合多个适应度指标,如进度差异和速度等,生成符合玩家技术水平的赛道。随后,Togelius 等人<sup>[13]</sup>对该算法进行了进一步改进,使其能够模仿玩家的驾驶风格,并调整适应度排名,从而生成更加个性化的赛道。此外,Cardamone 等人<sup>[14]</sup>提出了交互式进化赛道生成方法,通过在线平台让用户参与赛道进化过程,实验结果表明这种方法能够生成更加符合玩家主观意愿的赛道,提升了用户的参与感和满意度。Loiacono 等人<sup>[15]</sup>则结合信息熵的方法来生成赛道,他们通过最大化赛道曲率与速度分布的熵值,能够生成多样化且富有挑战性的赛道。Georgiou 和 Demiris<sup>[16]</sup>则通过结合玩家行为数据和传感器信息,提出了一种基于机器学习的实时赛道生成方法,该方法能够根据玩家的表现动态调整赛道难度,增强游戏的适应性。

在平台跳跃游戏关卡生成方面,近年来也涌现出大量研究。Green 等人<sup>[17]</sup>通过进化算法生成可玩性关卡,并通过基因型编码的动作序列进行交叉与变异操作,以保证关卡的可玩性。与此同时,Snodgrass 等人<sup>[18]</sup>提出了基于多维马尔可夫链(MdMCs)的关卡生成方法,采用增量生成方式与测试方式相结合,既考虑了可玩性,又兼顾了美学与设计约束。Hauck 和 Aranha<sup>[19]</sup>则提出了一种基于图语法的关卡生成方法,通过图的结构识别、匹配与生成过程,能够根据输入地图数据生成新的关卡,并且考虑了关卡中各结构元素之间的约束关系。此外,基于机器学习的关卡生成方法也得到了广泛研究,例如 Summerville 和 Mateas<sup>[20]</sup>利用长短期记忆网络(LSTMs)生成 Mario 关卡,通过控制训练集来提升生成关卡的可玩性。

在开放世界游戏地图生成方面,Frade 等人<sup>[21]</sup>提出了一种基于遗传地形编程(GTP)的方法,通过对高度图的进化操作,生成具有特定特征的地形。该方法通过控制种群大小与进化停止条件,能够生成符合美学要求的地形。Van Aanholt 和 Bidarra<sup>[22]</sup>提出了基于建筑轮廓的建筑生成方法,通过语义规范限制建筑求解器的生成范围,实现建筑的自动化生成。Freiknecht 和 Effelsberg<sup>[23]</sup>则提出了一种多

层建筑的程序化生成方法，能够生成包含楼梯间且可遍历的建筑结构。Mark 等人<sup>[24]</sup>提出了模块化管道生成地下洞穴的方法，通过 L-System 生成洞穴结构，结合渲染器生成 3D 模型，提供了灵活的参数调整选项，用以生成不同风格的洞穴。

本研究选取了平台跳跃游戏关卡作为研究对象，尽管国内外在该类型游戏关卡生成领域取得了显著进展，但仍然存在着生成后的关卡多样性不足的局限。传统的生成方法，如进化算法、马尔可夫链和图语法等，通常依赖优化固定规则来生成关卡。虽然这些方法能确保关卡的基本可玩性，但往往缺乏灵活性，导致生成的关卡结构单一，容易陷入模板化，重复性较高。这种生成方式无法持续提供新鲜的挑战或多样的游戏体验。

### 1.2.2 关卡修复国内外研究现状

在 PCG 领域，关卡修复技术主要旨在解决生成关卡过程中出现的各种缺陷，尤其是可玩性和美观性的问题。尽管许多研究聚焦于关卡生成，但仍存在生成关卡无法满足玩家需求的问题，例如缺乏可玩性、局部不连贯或元素放置错误等。因此，如何针对生成后的关卡缺陷进行修复，逐渐成为一个重要的研究方向。一些国外的研究使用基于搜索的寻路智能体来评估关卡的可玩性。在这些研究中，Robin<sup>[25]</sup>提出的物理模拟智能体和基于瓦片的智能体被广泛应用于生成关卡的评估。这些智能体的作用是判断关卡是否具有可通过路径，并在优化目标函数时纳入可玩性指标。然而，尽管这些智能体有效地评估了生成关卡的可行性，它们并未涉及修复那些存在缺陷的关卡。

在关卡修复方面，近年来的研究采用了多种机器学习方法。Jain 等人<sup>[26]</sup>通过去噪自编码器来修复不可玩的关卡。他们的研究表明，去噪自编码器可以学习关卡的风格规则，并通过应用这些规则将不合适的关卡部分修复为符合预定风格的新设计。然而去噪自编码器也存在局限性，尤其是在处理关卡中的长间隙等复杂情况时效果不佳。Mott、Nandi 和 Zeller<sup>[27]</sup>尝试使用长短期记忆（LSTM）网络修复 Mario 关卡中的局部不连贯性，通过学习关卡元素的时序信息，来对生成过程中出现的空白或不连续部分进行填补。

总的来说，关卡修复研究正在逐步发展。尽管已有的修复技术可以解决一些常见问题，但仍面临着挑战，即修复后的关卡与原关卡风格有显著差异。在现有许多现有的关卡修复方法中，元素的替换和调整往往没有充分考虑到原关卡的设计风格 and 视觉效果。它们倾向于使用标准化或通用的元素来修补缺失部分，虽然这样可以短期内满足关卡的可达性要求，但却可能破坏原有的艺术氛围和视觉连贯性。这种修复方式往往让关卡呈现出生硬、不协调的感觉，降低了玩家的沉浸

感和游戏体验。

### 1.3 本文的研究内容及组织结构

本文的主要研究内容包括四个部分：第一部分针对生成后的关卡可能存在缺陷的问题，研究并设计了关卡段修复方法；第二部分是关卡生成领域的研究，研究并设计了关卡段生成方法，然后针对长关卡生成可玩性不佳的问题，研究并提出了一个基于强化学习的长关卡构建策略；第三部分整合了前面的研究内容，完成了基于 PCG 的关卡生成游戏系统；第四部分则是关于这个基于 PCG 的关卡生成游戏系统的测试。本文的研究整体框架如图 1-1 所示。

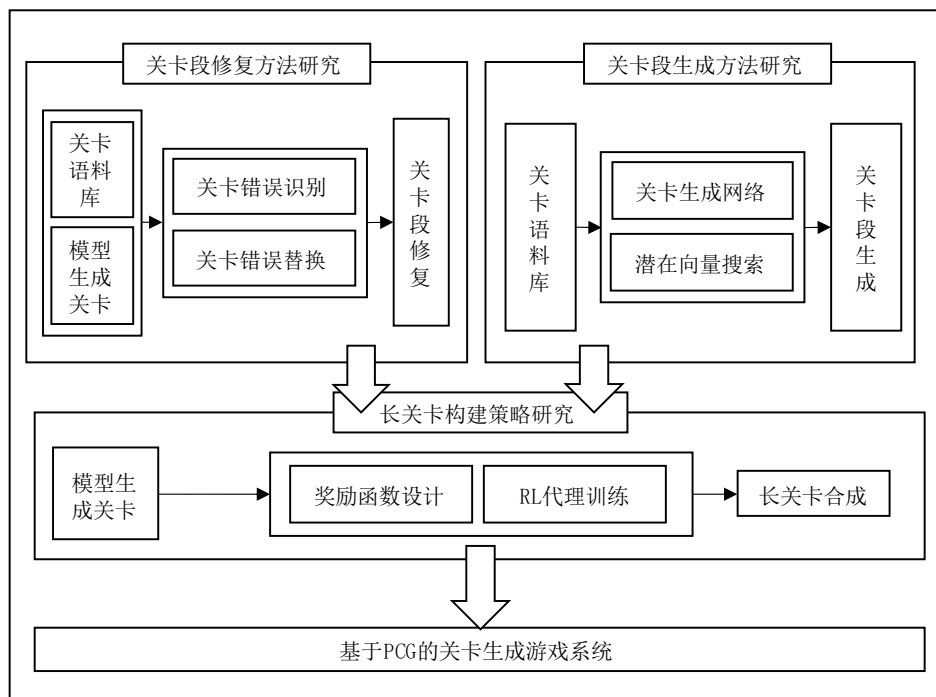


图 1-1 论文研究整体框架图

论文的章节安排如下：

第 1 章：绪论。本章主要介绍了本研究的背景，分别对关卡生成和关卡修复的国内外研究现状进行概述和分析。在此基础上，明确了本研究的切入点和研究目标。最后，对论文的组织结构进行了简要说明。

第 2 章：基于错误识别网络和遗传算法的关卡段修复方法。针对模型生成的关卡可能存在错误的问题，本文提出了一种基于错误识别网络和遗传算法（GA）的关卡段修复方法。该方法主要包括两个部分：首先，通过错误识别网络对关卡中的错误图块进行识别；接着，采用遗传算法对识别出的错误图块进行修复和替换。最后通过实验证明了本研究所提出的关卡段修复方法的有效性。

第 3 章：基于关卡段生成方法和长关卡构建策略的关卡生成方法。首先针对现有生成方法依赖优化固定规则来进行关卡生成，最后得到的关卡偏模板化的问题，提出了一种关卡段生成方法。方法由两部分构成：第一部分是通过训练获得能够生成具有初步可玩性关卡段的关卡段生成网络；第二部分是采用 QD 算法用于潜在向量的搜索，通过将搜索到的潜在向量作为关卡段生成网络的输入，生成符合要求的多样化关卡段。实验结果表明，该方法能够生成具有多样性的关卡段，并且所选用的 CMA-ME 算法在潜在向量的搜索上具有最好的表现。接着，针对长关卡生成可玩性不佳的问题，提出了一种长关卡构建策略。该策略首先设计合理的奖励函数，接着通过强化学习（RL）策略，将前述的关卡段修复方法与关卡段生成方法有机结合，实现长关卡的自动构建。最后通过实验证明了 RL 代理在指导长关卡生成方面的有效性。

第 4 章：基于 PCG 的关卡生成游戏系统设计与实现。本章将前面的研究内容结合，搭建了基于 PCG 的关卡生成游戏系统。首先根据实际应用要求，对基于 PCG 的关卡生成游戏系统功能性和非功能性需求进行分析。接着依据需求分析，完成系统架构、系统功能模块和系统数据库的设计与实现。

第 5 章：基于 PCG 的关卡生成游戏系统的系统测试。本章围绕基于 PCG 的关卡生成游戏系统的实际实现，进行了全面的系统功能模块测试，并展示系统测试结果。

## 第2章 关卡段修复方法研究

平台跳跃游戏关卡是由不同类型的图块组合而成的，每个图块代表地图上的一个固定位置，具有独立的属性和外观。以 Mario 游戏为例，图块有着砖块、管道、敌人等类型，它们共同组合成了游戏关卡。通过模型生成的关卡内容可能会出现不符合预期的图块分布，这会导致关卡的可玩性下降，因此需要对这些图块进行修复。虽然现有的修复方法能够有效解决不可达图块的问题，使得关卡变得可玩，但通常修复过程会涉及大量图块替换，这使得关卡原有的设计风格难以保持。为了解决这一问题，本研究以 Mario 的关卡为研究对象，探讨了关卡段修复方法的实现。本研究将修复过程分为两部分：第一部分为关卡错误识别网络，第二部分为错误图块修复算法。

### 2.1 错误识别网络的研究

在 Mario 游戏关卡的构成中，从物理角度出发，图块可分为不可穿越图块和可穿越图块两种，不可穿越图块指的是玩家无法穿越的地块，如地面、管道等，而可穿越图块指的是玩家能够顺利穿越的图块，如金币、道具等。这些图块的正确布局是保证关卡合理性的关键，而通过程序生成的关卡内容可能会出现图块布局不符合游戏规则的情况，即逻辑错误。图 2-1(a)和图 2-1(b)分别展示了逻辑错误的两种类型，前者是空中的不可穿越图块不完整，后者是组成管道的图块顺序错误导致破坏了视觉连续性的情况。

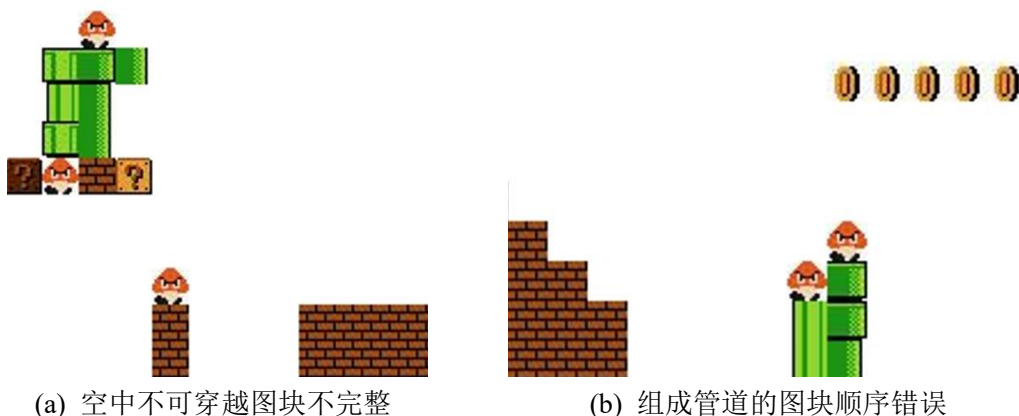


图 2-1 关卡修复示意图

在关卡修复任务中，图块错误识别是修复流程的第一步，主要目标是检测和

定位错误图块。由于错误图块的修复往往需要上下文信息（即周围图块的布局和属性），因此需要综合考虑其局部邻域。具体采取如下思路进行关卡图块的识别：

在关卡处理过程中，针对任意位置 $(i, j)$ 的图块，会以该位置为中心，构建一个 $3 \times 5$ 的图块网格进行分析。将中央图块与其周围 15 种图块类型共同构成的高度序列定义为组合，将中央图块的高度以及周围 14 个图块的类型组成的序列定义为周围信息。对于每个位置和图块类型，若其周围信息出现在训练数据中，则定义为真实周围信息；若未出现，则定义为假周围信息。例如，对于组合 $(12, 0, 6, 7, 0, 0, 0, 8, 9, 0, 0, 5, 8, 9, 0, 5)$ ，其中中央图块为 9，周围信息为 $(12, 0, 6, 7, 0, 0, 0, 8, 0, 0, 5, 8, 9, 0, 5)$ 。当遇到真实周围信息时，如果在训练数据中找到包含该周围信息且中心图块合适的组合，该图块则被视为合法图块；否则，它被视为非法图块。例如，对于周围信息 $(12, 0, 6, 7, 0, 0, 0, 8, 0, 0, 5, 8, 9, 0, 5)$ ，图块 9 是合法的，而图块 1 则属于非法图块。

围绕上述的方法，本研究构建了一个名为 LevelErrorNet 的多层感知机模型<sup>[28]</sup>，旨在学习 Mario 关卡中的管道相关约束，并对生成的关卡进行检查。该模型是一个前馈神经网络，通过学习关卡数据特征，生成多样化且符合游戏逻辑的关卡，该网络结构如图 2-2 所示。

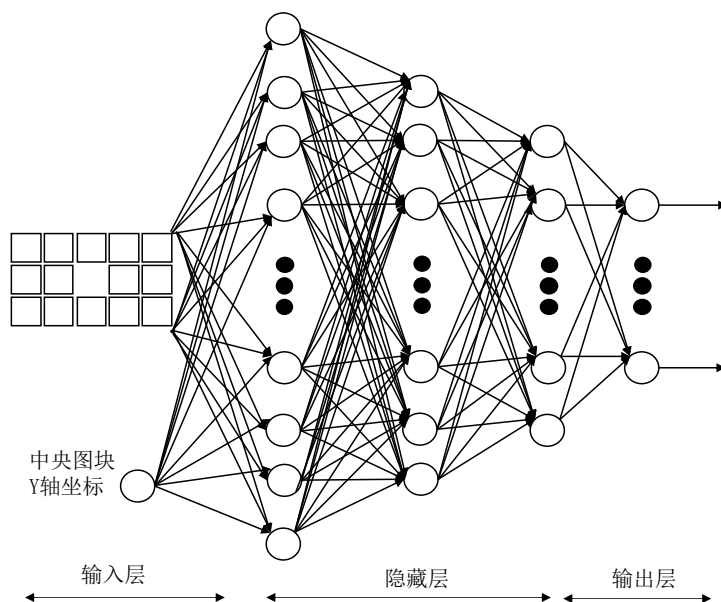


图 2-2 LevelErrorNet 网络结构图

输入层负责接收外部信息，包含与游戏关卡相关的关键特征。这些特征的维度由参数  $n\_feature$  确定，通常包括地形布局、障碍物分布、道具位置以及关卡难度等信息，这些数据通过特征向量输入到模型，为关卡生成过程提供基础。LevelErrorNet 模型由三层隐藏层组成。第一层接收来自输入层的特征向量，经过线性变换后映射到 200 维空间，并通过批归一化层稳定数据分布，避免梯度消失

或爆炸，最后通过 ReLU 激活函数进行非线性激活，提取更高层次的特征。第二层将数据映射到 100 维空间，再次通过批归一化和 ReLU 激活函数进一步挖掘更契合关卡逻辑的特征。第三层将数据映射到 50 维空间，经过批归一化和 ReLU 激活函数处理后，得到更为精细的关卡特征表示。

输出层接收第三层输出的 50 维数据，经过线性变换后映射到  $n\_output$  维度，该维度与关卡内容相关，如地形块类型、位置、障碍物及道具的具体摆放等。通过这种方式，模型将抽象的特征转化为具体的游戏关卡布局。在优化和损失计算方面，输出层使用 Adam 优化器和 CrossEntropy 损失函数。输出层给出每种图块类型的概率分布，并确保所有图块类型的概率和为 1。对于生成关卡中的每个位置  $(i, j)$ ，LevelErrorNet 根据周围图块类型输出不同图块的概率。如果某个图块类型的概率低于阈值  $\theta$ ，则该图块被判定为错误图块，为后续修复提供依据。

## 2.2 错误图块修复算法

在识别出关卡中的错误后，下一步便是对这些错误图块进行替换。在关卡修复过程中，鉴于关卡以离散的二维数组表示，使用遗传算法<sup>[29]</sup>来寻找合适的替换方案以修复损坏的区域，已成为一种行之有效的途径。本研究分为两个阶段：首先是初始种群优化阶段，为了使初始种群更适应修复任务需求，提出了一种优化策略；其次是迭代进化阶段，在每一代迭代中，依次执行交叉操作、适应度评估与更新、变异操作以及修复操作，持续迭代直到满足终止条件。

### 2.2.1 初始种群优化阶段

该阶段的目标是让初始种群更适应修复任务需求，从而提高算法找到全局最优解的概率，并加速收敛过程。

(1) 基于启发式采样生成部分较优个体 优化后的种群大小设为  $\Lambda$  的一半，针对关卡中的关键位置集合  $S$ （包含错误和可替换图块的位置）进行遍历。对于每个位置  $(i, j)$ ，根据关卡元素的空间关联性和统计特性，确定其可行替换元素集合  $pro\_tile$ ，并标记是否存在可替换元素（标志  $flag$ ）。如果  $flag$  为 True，依据历史数据或结构特征构建的映射关系，从  $pro\_tile$  中选择具有较高权重的元素为该位置赋值；若  $flag$  为 False，则保持原始元素值。这个过程通过分析当前位置与周围元素的组合模式、历史数据中的出现频率等，构建基于关卡结构的关卡元素关系模型。例如，若某个位置周围特定元素组合出现较多，则该元素会作为可替换的目标，若其概率超过设定阈值，则被纳入  $pro\_tile$  并设置  $flag$  为 True。

(2) 随机生成与修复操作生成剩余个体 生成剩余的个体。首先，将  $S$  中的位置初始化为原始关卡的值。接着遍历  $S$ ，查找可进行替换的原始元素位置，并



将这些位置存入 `pos` 列表。随机打乱该列表后，根据每个位置的邻域元素信息确定是否进行替换。如果满足条件，使用随机策略替换元素，并依据局部元素优化理论修复该位置，最终构成初始种群。修复操作基于局部一致性和连贯性要求进行，确保每个位置的元素在类型和功能上与周围元素一致。例如，当某位置的元素与其周围大多数元素的属性（如功能、类型等）差异较大时，将根据邻域属性分布选择合适的替换元素进行修复。

### 2.2.2 迭代进化阶段

迭代进化阶段包含了交叉操作、适应度评估与更新、变异操作和修复操作共四个部分，每个部分的具体介绍如下：

（1）交叉操作 此操作通过交换不同个体的优良基因片段，创造出更具潜力的新个体，增加种群的多样性，从而扩大解空间，获取更优化的关卡修复方案。具体方法是从当前种群中随机选择两个个体，设为个体 `A` 和 `B`，按一定概率交换和重组它们的替换方案信息（以字典形式编码的各位置及对应替代类型）。这一过程模拟自然界中的基因交叉，期望通过融合个体的优点，产生新个体，并探索更广泛的解空间，逐步接近全局最优修复结果。

（2）适应度评估与更新 此步骤旨在为自适应参数调整提供数据支持，增强算法的适应性。首先，计算适应度指标，遍历 `S` 中的每个位置  $(i, j)$ ，根据关卡元素关系模型，确定其可行替换元素集合 `pro_tile` 及标志 `flag`。若 `flag` 为 `True` 且元素在 `pro_tile` 中且元素数大于 1，则累加 `pro_tile` 的长度到 `pro_sum`；若不在，则错误计数 `error_sum` 加 1，同时累加 `pro_sum`。然后，通过 `get_step` 函数计算个体与原始关卡的差异步数，结合 `pro_sum` 和 `error_sum`，通过 `fitness_function` 计算适应度，并赋值给 `fit` 属性。自适应参数调整通过历史适应度列表 (`historical_fitness`) 来实现。当该列表长度超过某一值（如 10）时，计算适应度变化的均值。如果均值小于 0，变异概率 `P_M1` 将增加（乘以 1.1）；若均值大于 0，则降低（乘以 0.9），且 `P_M1` 的值限制在 0.001 至 0.5 之间。这一调整有助于优化搜索效率，当适应度持续下降时，表示搜索可能陷入局部最优，增加变异概率可以帮助跳出局部最优区域；反之，当适应度上升时，则适当减少变异概率，聚焦于当前优良解附近进行深入搜索。

（3）变异操作 此步骤旨在提高种群多样性，避免过早收敛。首先，遍历 `S` 中的位置  $(i, j)$ ，根据当前变异概率 `P_M1` 判断是否执行变异。若随机数小于 `P_M1`，根据个体元素与关卡结构关系确定可替换元素集合 `pro_tile` 及标志 `flag`。若 `flag` 为 `True`，则从 `pro_tile` 中随机选择一个元素替换该位置的元素，从而实现变异。变异操作需考虑其对关卡整体功能和结构的影响，确保变异后的元素与局部区域内其

他元素的交互逻辑正常，且不会破坏关卡的关键路径或目标条件。

(4) 修复操作 此操作对个体进行局部优化修复，以减少错误元素的存在。首先初始化一个空列表 `pos`，遍历 `S` 中的位置  $(i, j)$ ，根据关卡元素关系模型确定该位置的可行替换元素集合 `pro_tile` 及标志 `flag`。若 `flag` 为 `True` 且元素不在 `pro_tile` 中，则将位置存入 `pos` 列表，并对列表进行随机打乱。接下来，按修复概率 `RepairRatio` 判断是否进行修复。如果随机数小于 `RepairRatio`，则根据位置及其邻域元素的关系，确定可替换的元素并执行替换。该修复操作旨在确保局部区域的元素组合符合特定功能或美学标准，通过对局部规则模型的应用，提升关卡的整体一致性与正确性。

## 2.3 实验分析

为了评估 `LevelErrorNet` 在错误图块识别方面的性能，本文设计了两组实验。第一个实验测试了 `LevelErrorNet` 对合法图块的分类性能，旨在评估其在识别消除非法图块时的准确性与有效性，这为验证其是否能够正确识别关卡中的错误图块提供了基础。第二个实验测试了 `LevelErrorNet` 对非法图块的分类性能，旨在评估其是否可以有效检测出数据中的非法图块数据。针对错误图块修复算法的研究，本文设计了实验分别评估关卡段修复方法对图语法方法生成的有缺陷的关卡和随机替换图块产生缺陷的关卡这两者的修复效果，可以看到修复后的关卡可玩性显著高于修复前的。最后，本文对比了关卡段修复方法和 `LSTM` 方法进行关卡修复的表现，旨在体现经过修复前后的设计风格改变情况。

### 2.3.1 数据集简介

在针对关卡段错误识别网络的研究中，原始数据源来自于公开的 17 个 Mario 关卡，这些关卡以字符矩阵的形式存储在文本文件中（例如，字符 `X` 表示地面，字符 `<` 表示左上管道）。依据视频游戏关卡语料库（`VGLC`）中的编码方式<sup>[30]</sup>，定义了字符到数值的映射字典 `map_dic`，将关卡文件中的字符映射为数值编码（例如，`X` 映射为 0，`<` 映射为 6）。随后，采用滑动窗口遍历每个中央图块，提取其周围所有  $3 \times 5$  的图块网格。当滑动窗口超出关卡边界时，将边界外的图块标记为固定值 11，表示该位置为无效区域。每个图块对应的数据条目由包含 16 个元素的列表表示：前 15 个元素分别表示中央图块的垂直坐标（高度）及其周围 14 个图块的类型，最后一个元素表示中央图块的类型。对所有提取的数据进行去重后，共收集到 636 条合法组合，保存在列表 `legalList` 中。

为了验证 `LevelErrorNet` 在复杂错误场景下的鲁棒性，将合法组合列表 `legalList` 拷贝两份，针对每个数据条目分别修改第 2 到第 15 个元素中的 1 个和 2

个（即修改中央图块周围 14 个图块中的 1 个和 2 个位置）。修改后，确保新组合不与原始数据重叠，以避免与其他合法数据冲突。由此，生成了包含 1 个和 2 个虚假周围信息的数据列表，分别命名为 `legalListFake1` 和 `legalListFake2`。接下来，对合法数据列表 `legalList` 进行遍历。对于每个合法的 15 元素组合，收集所有未出现在该组合的第 16 个元素（中央图块类型），从而得到 5634 组非法数据列表，命名为 `illLegalList`。对非法数据列表同样复制两份，通过遍历修改，分别引入 1 个和 2 个真实周围信息，最终得到 `illLegalListTrue1` 和 `illLegalListTrue2` 两个列表。

在针对错误图块修复算法的研究中，数据集来源于两个部分。第一部分通过图语法方法生成 100 个测试关卡，并检测每个关卡中非法管道连接的数量。遍历关卡图块时，检查其  $3 \times 5$  邻域区域中是否包含管道图块（类型 6-9）。若邻域包含管道图块，则将其组合与合法数据列表 `legalList` 对比，验证该组合是否合法。若组合非法，则对破损管道计数加一；若邻域不包含管道或组合合法，则跳过计数。最终，选取破损计数最高的 25 个关卡用于关卡修复。第二部分数据集基于原有的 17 个关卡，通过随机破坏产生共 25 个有缺陷的关卡数据。具体操作为遍历关卡文件中的图块，当检测到管道类型图块时，将其随机替换为其他类型图块，生成随机破坏后的关卡数据。

### 2.3.2 参数设置

在关卡段错误识别网络的研究中，本课题将所有实验中的截断阈值  $\theta$  设置为 0.05。在对于错误图块修复算法的研究中，将进化修复器的适应度函数权重设定为  $w_1=5$ ,  $w_2=3$ ,  $w_3=1$ ，将用于判断图块类型是否符合要求的阈值设置为 0.05，即当图块类型的预测概率大于等于该阈值时，才会考虑将其纳入候选列表。在交叉操作后，变异操作的概率  $P\_M0$  设置为 0.8，当生成的随机数小于该概率时，子代个体将经历变异操作。将图块替换概率 `RepairRatio` 设为 0.3，迭代次数设置为 50 次，选择操作中个体间的比较次数 `RRT_M` 设为 4，种群大小 `Lamda` 设定为 20。

### 2.3.3 评价指标

在针对关卡修复的研究中，Shu 等人<sup>[31]</sup>提出了不稳定值等一系列指标，它们可以较好反映关卡的修复效果，本文选用它们作为评价指标来对关卡修复效果进行评估。第一个评价指标是剔除图块占总图块的比例，如公式（2-1）所示。 $R(x)$  代表要被剔除图块的数量， $Sum(x)$  代表关卡图块的总数量。这个比例越小，代表要剔除的图块越少，这有助于关卡保持原有的设计风格。

$$Q_{rate1} = \frac{R(x)}{Sum(x)} \quad (2-1)$$

第二个评价指标是评估检测到的非法图块的比例，如公式（2-2）所示。 $I(x)$ 代表识别出非法图块的数量， $Sum(x)$ 代表关卡错误图块的总数量。这个比例越大，代表对错误图块的识别能力越好。

$$Q_{rate2} = \frac{I(x)}{Sum(x)} \quad (2-2)$$

第三个评价指标不稳定值 $UV(l)$ 是其中所有不稳定图块的候选类型数量之和，如公式（2-3）所示。不稳定图块指的是替换时具有多种候选类型的图块，关卡 $l$ 中的不稳定图块集合表示为 $U(l)$ ， $C_{i,j}^l$ 表示第 $l$ 层位置 $(i, j)$ 处的候选类型集合。不稳定值反映了虚假周围信息的程度，较低的不稳定值表明关卡中虚假周围信息较少，从而意味着破损区域较少。

$$UV(l) = \sum_{(i,j) \in U(l)} |C_{i,j}^l| \quad (2-3)$$

在对方法修复效果的比较中，采用了 KL 散度<sup>[32]</sup>作为评价指标，KL 散度可以反映两个关卡间的相似性，修复前后关卡的 KL 散度接近代表设计风格改变较少。

#### 2.3.4 实验对比与分析

第一个实验是检测 LevelErrorNet 对于合法图块的识别能力，选取了 legalList、legalListFake1 和 legalListFake2 这三个数据列表进行实验，表 2-1 展示了实验结果。

表 2-1 合法图块实验中被消除图块的比例

| Type | Total | True  | Fake1(num) | Fake1(%) | Fake2(num) | Fake2(%) |
|------|-------|-------|------------|----------|------------|----------|
| 0    | 76    | 0.00% | 6          | 7.90%    | 18         | 23.70%   |
| 1    | 23    | 0.00% | 2          | 13.00%   | 6          | 26.10%   |
| 2    | 294   | 0.00% | 14         | 5.10%    | 25         | 8.80%    |
| 3    | 2     | 0.00% | 0          | 0.00%    | 0          | 0.00%    |
| 4    | 13    | 0.00% | 1          | 7.70%    | 3          | 23.10%   |
| 5    | 20    | 0.00% | 3          | 15.00%   | 6          | 30.00%   |
| 6    | 33    | 0.00% | 0          | 0.00%    | 3          | 12.10%   |
| 7    | 34    | 0.00% | 0          | 2.90%    | 5          | 17.60%   |
| 8    | 69    | 0.00% | 0          | 0.00%    | 4          | 7.20%    |
| 9    | 72    | 0.00% | 1          | 1.40%    | 7          | 11.10%   |

本研究通过 LevelErrorNet 对每个数据列表内的数据条目计算该位置下对应图块类型的概率，当概率小于设定的截断阈值  $\theta$  则代表该数据条目所对应的图块类型是被识别为要消除的。若 LevelErrorNet 计算后的概率大于设定的截断阈值  $\theta$ ，即代表该数据条目所对应的图块类型是不需要被消除的，该图块被记为候选图块。因为实际关卡中对于特定的周围信息组合，可能会存在多个合法的中央图块。实验中候选图块中始终包括编号为 6、7、8、9 的合法图块，它们代表管道的组成部

分, 这表明 LevelErrorNet 成功学习了不同类型的管道组合规则。然而, 在某些情况下, 管道周围的图块 (例如怪物图块) 有时会被错误地排除在候选图块之外, 这可能是由于训练数据量有限导致 LevelErrorNet 未能很好地学习此约束。

表 2-1 的 True 列为合法组合 legalList 的测试结果, 由于所有图块的组合均为合法, 不存在错误数据, 因此没有图块被消除, 这表明 LevelErrorNet 不会消除合法的图块。对于引入了虚假周围信息的 legalListFake1 和 legalListFake2 两个数据列表而言, 错误图块的引入使得分类变得更具挑战性, 在表 2-1 的 Fake1 和 Fake2 列中, 分别展示了它们的识别结果。通过和 True 列的对比可以发现, 随着虚假周围信息的增加, 合法图块被错误排除的概率也是在增加的, 但整体的消除比例还是在一个良好的范围内。

第二个实验则是检测 LevelErrorNet 对关卡中的非法图块进行识别的能力, 选取了 illLegalList、illLegalListTrue1 和 illLegalListTrue2 这三个数据列表进行实验, 表 2-2 展示了实验结果。当非法中央图块的预测概率低于设定的截断阈值  $\theta$ , 则代表该图块是被正确检测的。表 2-2 的 True 列为对非法数据列表 illLegalList 测试的结果, 可以看到 LevelErrorNet 识别出了大部分的非法图块。表 2-2 的 Fake1 和 Fake2 分别展示了 illLegalListTrue1 和 illLegalListTrue2 两个列表的测试结果, 可以看到在提供了真实周围信息的情况下, 被检测到的非法图块比例有所下降, 但仍处于一个较高的水准。总体而言, LevelErrorNet 在合法图块的识别和非法图块的检测方面有着良好的表现。

表 2-2 非法图块实验中被检测到的非法图块比例

| Type | Total | True    | Fake1(num) | Fake1(%) | Fake2(num) | Fake2(%) |
|------|-------|---------|------------|----------|------------|----------|
| 0    | 551   | 98.20%  | 499        | 90.60%   | 485        | 88.20%   |
| 1    | 604   | 97.20%  | 570        | 94.40%   | 568        | 94.20%   |
| 2    | 333   | 96.70%  | 288        | 86.50%   | 268        | 80.50%   |
| 3    | 625   | 99.00%  | 595        | 95.20%   | 581        | 93.10%   |
| 4    | 614   | 99.00%  | 585        | 95.30%   | 559        | 91.20%   |
| 5    | 607   | 98.20%  | 588        | 96.90%   | 582        | 95.90%   |
| 6    | 594   | 99.80%  | 582        | 98.00%   | 569        | 95.80%   |
| 7    | 593   | 100.00% | 585        | 98.70%   | 572        | 96.60%   |
| 8    | 558   | 100.00% | 549        | 98.40%   | 534        | 95.70%   |
| 9    | 555   | 100.00% | 540        | 97.30%   | 521        | 93.90%   |

接着对错误图块修复算法进行了实验, 图 2-3 展示了关卡段修复方法修复图语法方法生成的有缺陷关卡的进度。在修复开始前, 由于虚假周围信息和低截断阈值  $\theta$  的影响, 存在多个错误图块 (由红框标出) 和不稳定图块 (由蓝框标出), 效果如图 2-3(b)所示。随着优化过程的推进, 错误图块和不稳定图块的数量逐渐减少。在仅经过 9 代 (图 2-3(f)) 后, 仅剩 5 个不稳定图块, 经过 30 代的优化后,

关卡几乎完美修复。与原来的关卡相比，缺陷修复得非常完美，只改变了几块图块。与适应度函数的设计是一致的。有些破损的管道被清除，有些则被修复。这表明该算法可以得到合理的结果。

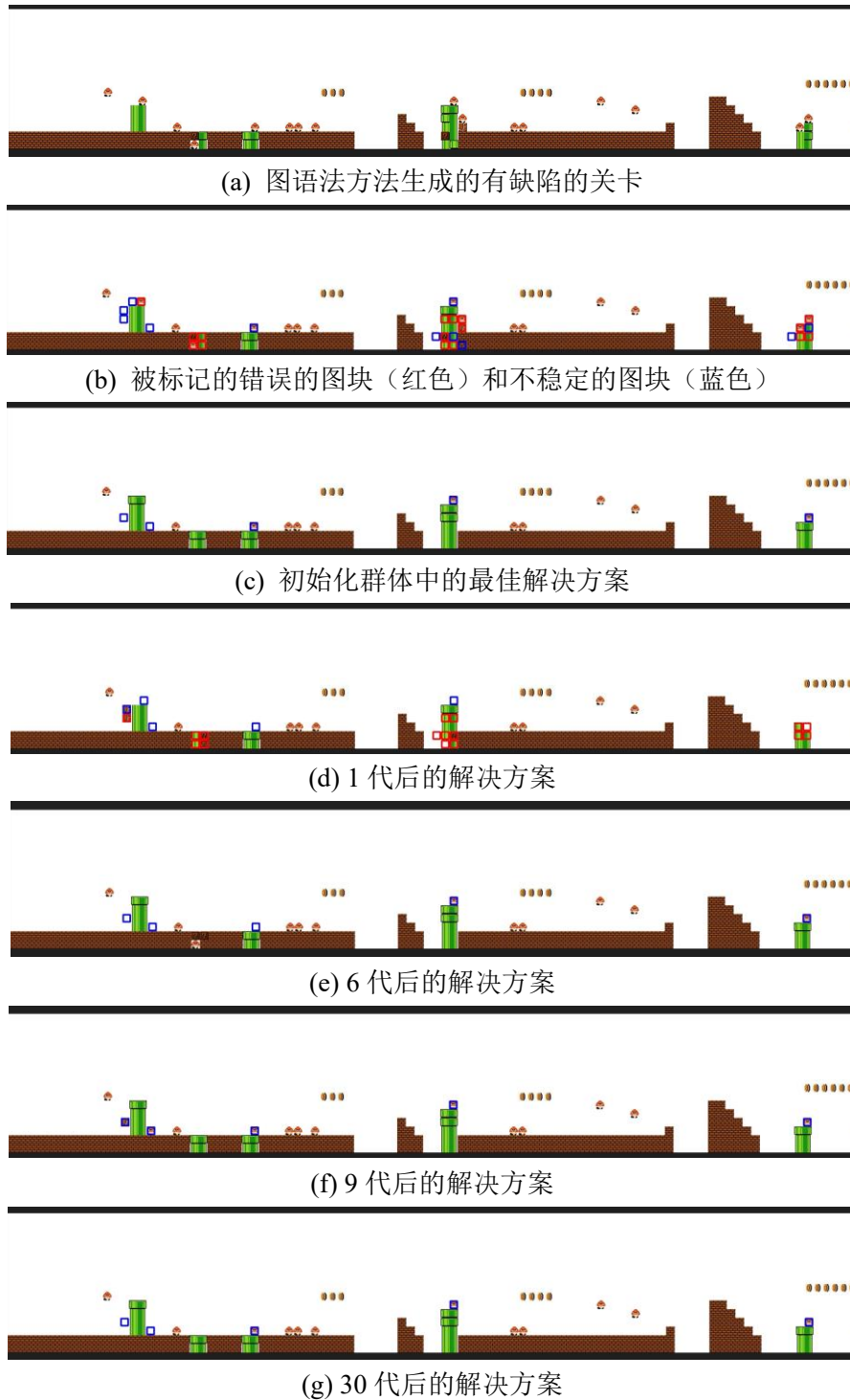


图 2-3 修复图语法方法生成的有缺陷的关卡

图 2-4 展示了修复随机破坏关卡的结果。在图 2-4(b)中，LevelErrorNet 成功

识别并标记了所有被破坏的图块。图 2-4(g)显示了经过 30 代后的修复状态。所有破损的管道得到了有效修复，尽管管道周围的某些图块发生了变化，可能是由于进化过程的不确定性或 LevelErrorNet 尚未完全训练，从而将一些合法图块错误地标记为非法。

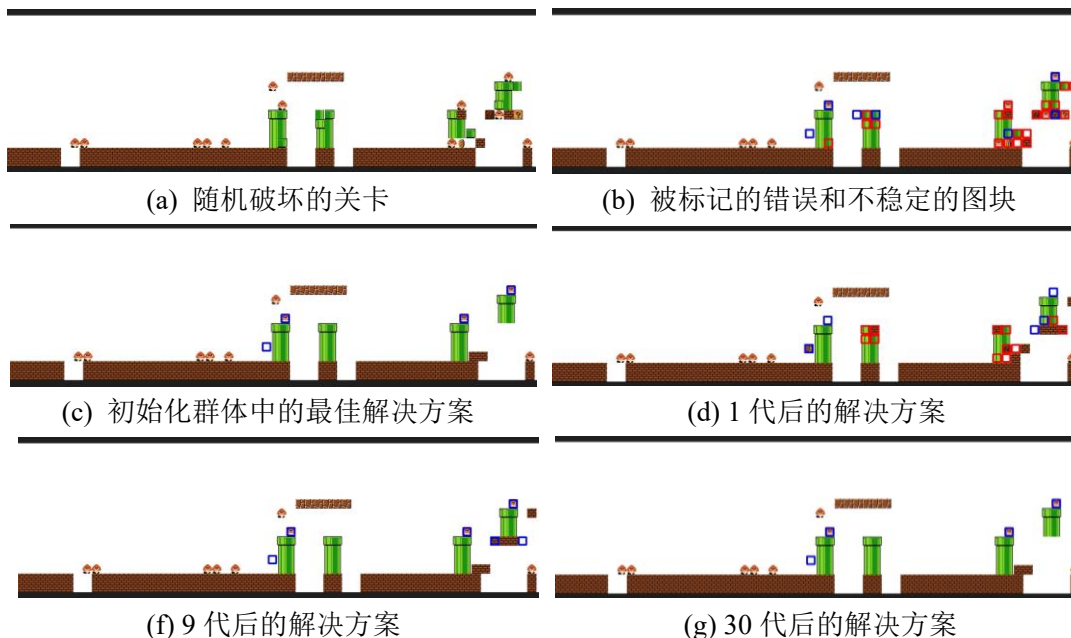


图 2-4 修复随机破坏的关卡

在本研究中，为最大程度减少关卡中的错误图块数量，使用了一个加权和作为适应度函数，如公式 (2-4) 所示。其中， $a_i$  是要设置的系数， $i \in \{1, 2, 3\}$ 。 $R(x)$  表示在修复过程中被替换的图块数量，较少的替换有助于保留原关卡的设计特征。 $UV(x)$  则是不稳定值，不稳定值越小，表示关卡中的假周围信息越少，破损区域也相应减少。通过最小化该适应度函数，使得修复过程中尽量减少错误图块和替换图块的数量，同时优化关卡的不稳定性。

$$F(x) = a_1 R(x) + a_2 W(x) + a_3 UV(x) \quad (2-4)$$

为了可视化进化过程，图 2-5 展示了 50 次实验的平均适应度值和适应度函数中三个指标的变化曲线。不稳定值（黄色曲线）和错误图块的数量（蓝色曲线）迅速下降，这是修复步骤的直接结果。而替换图块的数量则相对缓慢增加，且在约第 20 代时趋于平稳。这表明，尽管图块替换数量有所增加，但关卡修复质量却在持续提高，符合预期目标即在关卡修复过程中尽量减少替换图块数量。

在一次优化过程中，修复随机破坏关卡时所有个体的适应度值散点图如图 2-6 所示。从图中可以看到，个体适应度值逐渐趋于一致，适应度差异明显减小。综上所述，通过实验验证，本研究提出的关卡段修复方法在修复图语法方法生成的有缺陷的关卡（图 2-3(a)）和修复被随机破坏的关卡（图 2-4(a)）中表现出了良好

的修复效果。

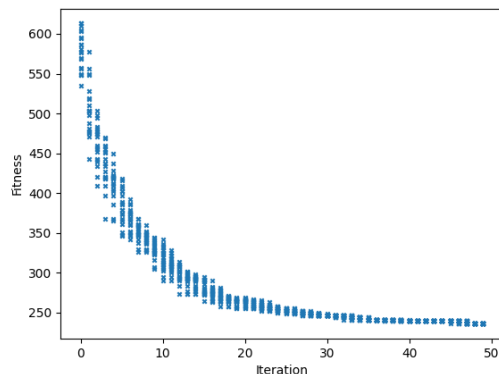
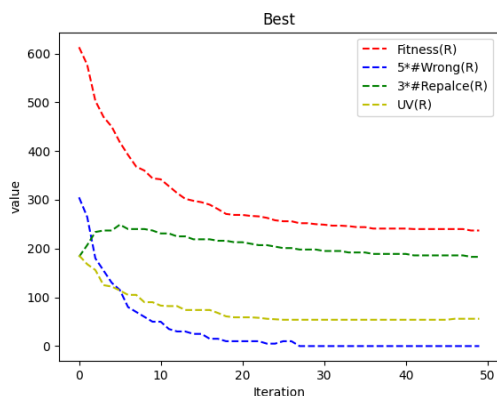


图 2-5 种群中适应度等指标的进化曲线 图 2-6 进化过程中所有个体的适应度散点图

最后本研究分别用 LSTM 的关卡修复方法和本文的关卡段修复方法对有错误图块的共 50 个关卡进行了修复，使用 KL 散度作为指标对修复结果进行了评估，结果如图 2-7 所示。可以看到使用关卡段修复方法修复后的关卡和原本关卡的 KL 散度差异较小，这表明修复后二者的设计风格类似，即代表修复的表现是更好的。

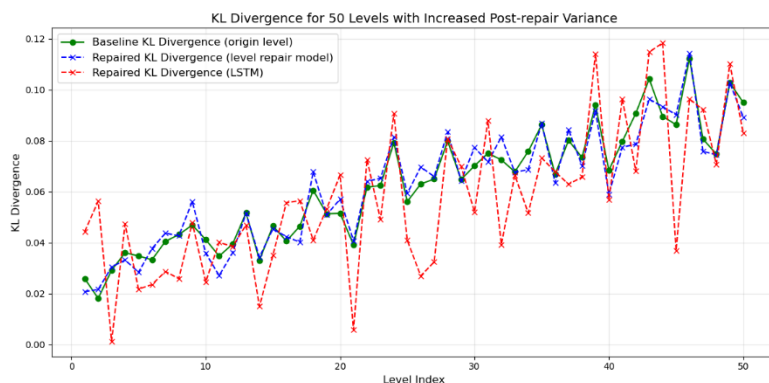


图 2-7 关卡修复结果 KL 散度对比图

## 2.4 本章小结

本章针对随机生成关卡中常见的错误图块问题，提出一种结合错误识别网络与遗传算法的关卡段修复方法。首先，通过关卡错误识别网络 LevelErrorNet，基于图块及其邻域布局特征，自动识别违反游戏逻辑的错误图块。随后进入修复阶段，采用遗传算法对关卡进行优化，过程分为初始化与迭代进化两个阶段，并结合空间关联性与关键位置分析生成有效的修复方案。实验结果表明，经遗传算法修复后，关卡中的错误图块被有效消除。同时，对比实验进一步验证了该方法在保持关卡原有设计风格方面具有更优表现。综上所述，所提出的关卡段修复方法不仅具备良好的错误检测与修复能力，还能够有效保持关卡的设计风格。



## 第3章 关卡生成方法研究

人工制作二维游戏关卡通常需要大量的精力和时间，通过引入 PCG 技术，开发者能够在较短时间内获得更多的关卡内容。现有的 PCG 生成方法，如基于规则的生成和图语法方法，依赖大量人工设定的规则和参数，操作复杂且最后生成的关卡偏模板化。为了解决这一问题，本研究聚焦于关卡段生成，探索了一种新的关卡生成方法。

这种随机生成的关卡通常只能在较短的范围内有良好的可玩性<sup>[33]</sup>，当关卡长度增加后容易出现不可玩的片段。针对这一问题，本研究以关卡段为单元，探索结合强化学习技术来驱动各个关卡段生成与拼接构建长关卡的策略。

### 3.1 基于生成对抗网络和 QD 算法的关卡段生成方法

在平台跳跃游戏中，关卡是游戏环境的基本单元，承载着玩家的游戏进程和互动体验，关卡的设计直接影响游戏的难度。从设计角度出发，关卡的构成主要包括基础地形（如地面、平台、管道）、互动元素（如金币、道具、敌人）、障碍物（如尖刺、陷阱），这些不同类型的图块组合成了游戏的关卡，如图 3-1(a)所示。关卡生成是指通过特定算法和规则，自动地构建符合游戏逻辑的关卡地图——即在给定输入参数和游戏规则的约束下，产生一种符合预期要求（如可玩性）的图块组合方式，如图 3-1(b)所示。

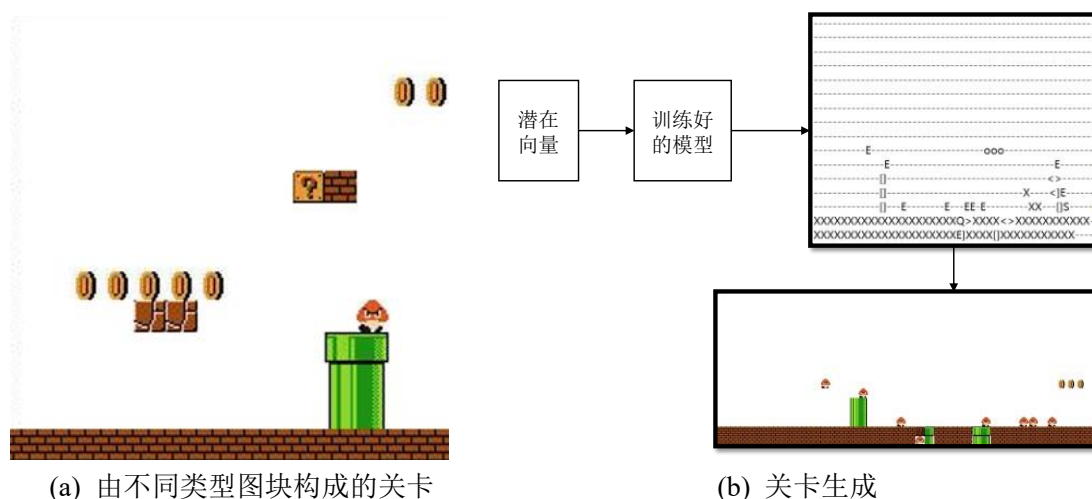


图 3-1 关卡生成示意图

本研究主要由两个关键阶段构成。第一个阶段，是在现有的 Mario 关卡上进

行关卡段生成对抗训练网络的训练。在这一阶段，关卡通过多维数组进行编码，生成器接收高斯噪声向量作为输入，经过训练后能够使用相同的表示方式输出新的关卡图块。此时，鉴别器的作用是评估并区分真实关卡和生成关卡，从而引导生成器优化输出结果。完成这一训练过程后，生成器就能够将潜在的实数向量作为输入，生成相应的关卡段。第二个阶段，通过搜索满足特定条件的潜在向量，然后将其作为关卡段生成对抗训练网络的输入，最后得到需要的关卡段。

### 3.1.1 关卡段生成对抗网络的研究

本课题采用生成对抗网络<sup>[34]</sup>构建关卡段生成方法的第一阶段，该网络主要由判别器和生成器两大部分组成，它们的具体设计介绍如下：

(1) 判别器设计 判别器采用经典卷积神经网络架构，通过从输入图像中提取特征来判断其真实性。其初始阶段使用特定尺寸的卷积核和步长进行特征提取和尺寸压缩，基于卷积神经网络的局部感知原则，卷积核在图像上滑动，捕捉局部特征，并逐步汇总为更高级的抽象特征表示。随后，多个卷积层进一步深化特征提取，通过调整卷积核的数量和尺寸，捕获不同尺度和层次的图像信息，从而为准确区分图像的真实性奠定基础。

判别器采用下采样金字塔结构，在网络层数增多时，图像尺寸逐步缩小，特征通道数逐步增多。此设计基于多尺度特征分析理论，不同尺度的特征对判别任务具有不同的重要性。较小尺度的特征提供全局结构信息，较大尺度则保留更为细节化的信息。通过逐步下采样，判别器能够有效整合不同尺度的特征，提升其对图像全局和局部特征的综合判断能力，从而提高判别精度。判别器所有层均使用 LeakyReLU 激活函数，相较传统 ReLU，其在输入为负值时仍保留较小斜率，从而避免神经元死亡现象，有助于捕捉图像中的微小差异和复杂特征。此外，为防止过拟合，在每个 LeakyReLU 层后添加 Dropout 层。Dropout 层通过随机失活部分神经元，减少神经元之间的共适应性，增强模型的泛化能力，提高判别器在不同类型图像上的稳定性。

(2) 生成器设计 生成器以随机噪声向量作为输入，经过多个转置卷积层逐步将其转换为图像特征图。转置卷积具有上采样特性，能够在增大特征图尺寸的同时调整通道数，逐步构建出细节丰富的图像结构。在这一过程中，每一步的转置卷积后接批量归一化层和 ReLU 激活函数。批量归一化通过对数据进行标准化处理，能够加速网络训练并提高模型稳定性，ReLU 则为生成器引入非线性因素，帮助其学习更复杂的图像特征映射关系。

生成器的中间层进一步通过卷积模块来细化图像特征。这些卷积操作旨在深入挖掘图像的细节与语义信息，通过调节卷积核参数和层数，增强生成器对图像

特征的表达能力。为了应对生成图像中的噪声问题<sup>[35]</sup>，生成器的末尾加入了 DnCNN 网络模块<sup>[36]</sup>，通过学习图像中的残差信息，有效去除噪声成分，提升生成图像的质量。DCNN-DCGAN 的架构如图 3-2 所示。

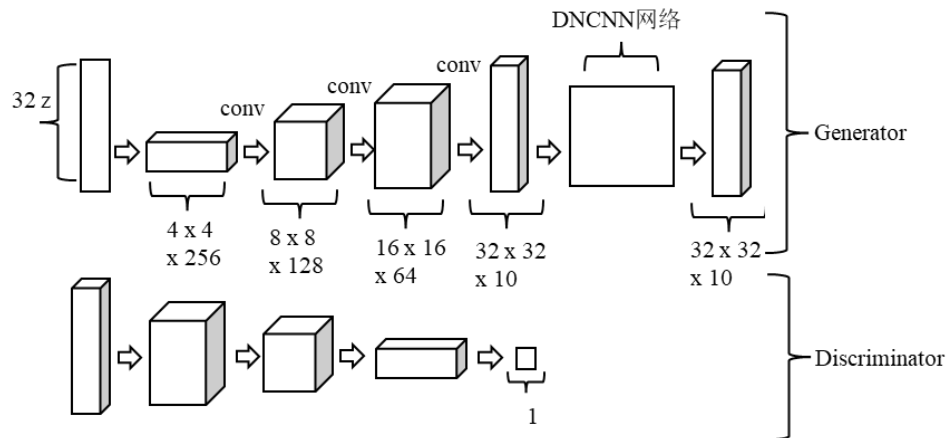


图 3-2 DCNN-DCGAN 架构

在模型训练过程中，每个整数图块都被转换为一个 one-hot 向量。具体来说，如果关卡中有 10 种不同的图块类型，则每个图块将对应一个长度为 10 的向量，其中与该图块类型对应的元素值为 1.0，其余元素为 0.0。经过这种处理，判别器的输入变为一个尺寸为  $32 \times 32$  像素、包含 10 个通道的张量，表示了 10 种图块类型的 one-hot 编码。为了符合模型的输入要求（输入尺寸必须是 16 的倍数），本研究对输入数据进行了适当的填充。在此张量中，第一个通道用于标记地面砖块的位置，仅在对应砖块的像素位置设置值为 1.0。生成器则以长度为 32 的随机潜在向量为基础，生成复杂的图像结构。在训练完成后，生成器会输出一个  $32 \times 32 \times 10$  的张量，表示潜在空间与多样化游戏关卡设计之间的映射关系。为了与下游的 Mario AI 框架兼容，输出张量将被裁剪为  $28 \times 14 \times 10$  的尺寸，随后对每个图块的输出向量应用  $\text{argmax}$  函数，将其转化为具体的整数类别标签。这些标签接着会被解码，用于生成实际可玩的 Mario 游戏关卡。

### 3.1.2 潜在向量搜索

经过前期研究，训练后的模型已能够通过潜在向量生成新的关卡段。尽管生成包含地砖和敌人的关卡是模型的理想目标，但通过随机潜在向量生成的关卡段存在两个固有的问题：

（1）地砖和敌人的数量并不总能有效地影响玩家或人工智能代理的游戏体验，例如敌人可能在玩家角色到达前就掉入陷阱，这可能导致生成的关卡未能达到预期的难度水平。

（2）随机生成的关卡可能存在较低的可玩性，当地面图块之间的间距过大时

将导致玩家角色无法跳跃通过平台。

为了解决这些问题，本课题采用了基于游戏数据的评估方法。通过代理测试，可以明确地评估关卡的可玩性并直接观察关卡特征。在评估过程中，首先将潜在向量映射到 $[-1, 1]^n$ 区间，使用 sigmoid 函数进行处理，之后将其传递给生成器以生成相应的关卡，接着将关卡通过视频游戏关卡语料库（VGLC）所述的编码方式导入 Mario AI 框架，并在其中进行代理模拟。本研究选择了 Robin 的 A\*寻路智能体作为关卡游玩代理，通过计算代理执行的跳跃次数来近似表示关卡的难度。跳跃次数与难度之间的关系是一个假设，因跳跃是 Mario 游戏中的核心机制，玩家通常需要通过跳跃来克服障碍。因此，在难度较大的关卡中，玩家需要进行更多的跳跃。使用的适应度函数如公式（3-1）所示。游戏进程中代理沿着 x 轴向前移动，其中  $s$  表示关卡在 x 轴上的完成进度， $j$  表示跳跃次数。

$$F_1 = \begin{cases} -s, 0 \leq s < 1 \\ -s - j, s = 1 \end{cases} \quad (3-1)$$

此外，为了衡量关卡的可玩性，结合代理的跳跃次数设计了一个旨在通过最小化跳跃次数来生成可玩关卡的适应度函数，如公式（2-2）所示。其中  $s$  代表关卡在 x 轴上的完成进度， $j$  为跳跃次数。

$$F_2 = \begin{cases} -s + 50, 0 \leq s < 1 \\ -s + j, s = 1 \end{cases} \quad (3-2)$$

为了避免代理在关卡中卡住，导致反复跳跃，针对未能成功完成的关卡设置了 50 的偏移量，这样成功完成（即具有可玩性）的关卡将始终比无法完成的关卡获得更好的适应度评分。由于跳跃次数存在一定的不确定性，并且如果智能体卡在某些地方，可能会产生异常值，因此在计算实际适应度值时，采用了 10 次模拟的平均值来确保评估的稳定性和准确性。本课题采用了 Quality-Diversity(QD)算法<sup>[37]</sup>进一步优化基于代理的适应度函数。QD 算法是一种新颖的进化类算法，旨在同时优化解决方案的质量和多样性。它通过明确的行为指标衡量目标，从而找到一组高质量且覆盖行为空间的多样化解决方案。

## 3.2 长关卡构建策略

随机生成的关卡通常只能在较短的范围内有良好的可玩性，当关卡长度增加后容易出现不可玩的片段，如图 3-3 所示。为了保障生成具有良好体验的长关卡，本研究以关卡段为单元，通过构建多维度的奖励函数将设计原则转化为可量化的优化目标，接着制定了一个 RL 训练策略来训练代理，使其能够对整个长关卡拼接过程进行指导。

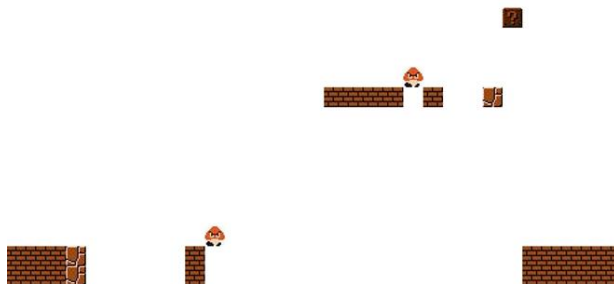


图 3-3 关卡长度增加后局部平台间距过大

### 3.2.1 奖励函数设计

为 RL 代理设计合适的奖励函数至关重要<sup>[38]</sup>。在本课题中，设计了 3 个指标来制定不同的奖励函数，旨在驱动 RL 代理来学习生成具有良好玩家体验的可玩关卡，奖励函数的具体设计如下：

#### (1) 机制量

对于整体关卡设计风格，它和关卡机制是密切相关的，机制是关卡体验的重要组成部分<sup>[39]</sup>，一个关卡内机制的数量和类型应该是相似的。机制的数量适中是游戏体验的重要保证，本课题假设关卡段之间的机制数量应该保持在中等水平，过多的机制数量会导致关卡风格混合，而过低则更容易产生同质化的片段。本课题采用基于图块的 KL 散度作为片段之间机制相似性的简单有效的度量，片段相似性  $F_s(S)$  的定义如公式 (3-3) 所示。

$$F_s(S) = \frac{1}{n} \sum_{i=1}^n KL(S, S_{w_i}) \quad (3-3)$$

其中  $S$  是生成的水平段，高度为  $h$ ，宽度为  $w$ ； $KL(S, S_{w_i})$  是基于图块的 KL 散度，它考虑两个给定关卡段  $S$  和  $S_{w_i}$  中图块模式出现的分布之间的标准 KL 散度。本课题定义一个与  $S$  大小相同的窗口  $W$ ， $S_w$  表示窗口  $W$  中包含的段。滑动窗口以步幅  $d$  从  $S(S_{w_0} = S)$  的位置移动到前一段  $n$  次。参数  $n$  限制窗口移动的次数， $n$  值越大，考虑过去的水平段越多。本课题将片段相似性的下限和上限分别定义为  $l$  和  $u$ ，最后得出机制量的奖励函数  $M(S)$  如公式 (3-4) 所示。

$$M(S) = \begin{cases} -(F_s(S) - u)^2, & F_s(S) > u \\ -(F_s(S) - l)^2, & F_s(S) < l \\ 0, & l \leq F_s(S) \leq u \end{cases} \quad (3-4)$$

#### (2) 重复度

对于关卡流程有差异化，则是不希望关卡内容同质化，降低玩家游玩的重复

感。本研究希望能够尽可能生成偏离早期创作的关卡段的内容<sup>[40]</sup>，所以将某一片段重复度定义为它与之前  $m(m > k)$  个先前片段中  $k$  个最相似片段的平均相似度，最后定义重复度的奖励函数  $S(S)$  如公式 (3-5) 所示。其中  $I_j$  是前  $m$  个片段中与  $S$  相比第  $j$  个最相似片段的索引； $m$  代表本课题的 RL 代理在其内存中保存的段数，该参数在概念上类似于新颖性搜索中的稀疏参数。

$$S(S) = -\frac{1}{k} \sum_{j=1}^k KL(S, S_{I_j}) \quad (3-5)$$

### (3) 可玩性

本研究认为关卡具有一定流程是非常重要的，应该鼓励代理生成尽可能长的关卡。为了保证整体关卡衔接的可玩性，把相接的 3 个关卡段串联作为一个整体进行测试。测试时，游戏代理从上一个关卡片段的结束位置开始玩。仅当游戏代理成功到达  $S$  的最右端时，段  $S$  才被确定为可玩。否则，如果生成的片段不可玩，则让游戏结束，并返回已完成片段数的加权。基于此思路得到一个奖励项  $P(S)$  如公式 (3-6) 所示。其中  $p_i$  代表片段  $i$  的可玩性， $p_i \in [0,1]$ 。

$$P(S) = \sum_{i=1}^n p_i, n \leq 100 \quad (3-6)$$

## 3.2.2 RL 训练策略的研究

本节研究了一个 RL 训练策略，如图 3-4 所示。

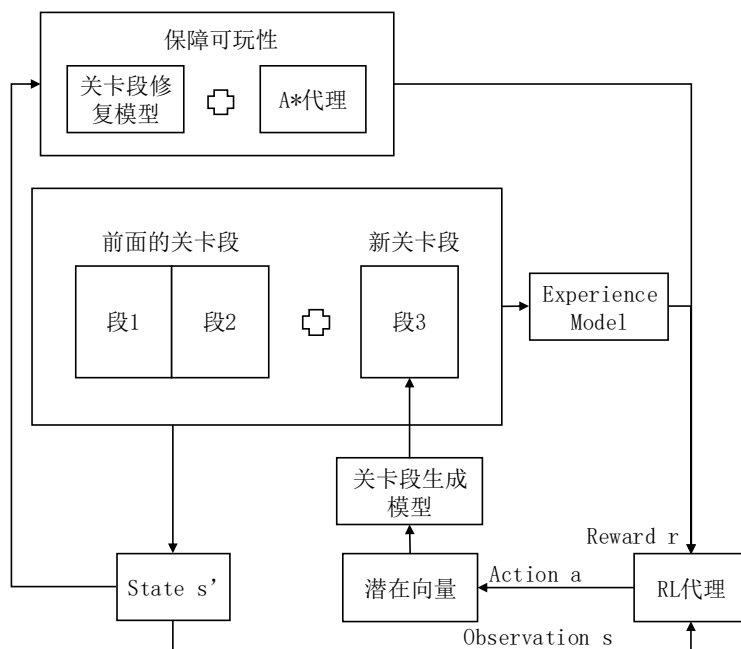


图 3-4 RL 训练策略

该策略包含三个关键组成部分，分别是关卡段生成模型和关卡段修复模型、测试可玩性的 A\*代理和 RL 代理，具体介绍如下：

(1) 关卡段生成模型和关卡段修复模型 本课题拟使用基于前面研究方法构建的关卡段生成模型和关卡段修复模型分别生成和修复生成的关卡片段。

(2) A\*代理 为了确保关卡的可玩性，本研究选用了 Robin 开发的 A\*代理进行测试，确保所生成的关卡具备可玩性。

(3) RL 代理 RL 代理通过与定义的 RL 环境交互，学习设计具备特定玩家体验的内容<sup>[41]</sup>。内容质量组件确保体验模型考虑到特定内容的质量（例如通过游戏模拟进行测试）。RL 代理通过采取与生成行为相对应的操作，调整关卡内容的状态，并根据体验模型获得奖励。代理在设计（状态和动作）空间和奖励空间中迭代，以找到优化体验模型的设计策略。

本研究采用 PPO 算法<sup>[42]</sup>来训练 RL 代理。在 PPO 训练过程中，动作和状态由长度为 32 的潜在向量表示，初始状态从训练关卡段生成模型的潜在空间中随机选取生成的可玩关卡片段。PPO 代理基于当前潜在向量选择一个动作，并将其作为关卡段生成模型的输入生成新的关卡片段。在生成的片段中，关卡段修复模型负责修复其中的错误部分，而 A\*代理会测试新片段的可玩性。随后，生成的关卡片段（与动作相同的潜在向量）作为新观察反馈给代理，代理根据所采取的动作获得奖励。具体设计如下：

(1) 初始化阶段 首先初始化时间步  $t$  为 0，并设定必要的输入条件：关卡段生成模型  $G$ （将  $[-1, 1]^{32}$  维向量映射到关卡段）、关卡段修复模型  $F$ （对关卡段进行修复）、A\*代理  $P$ （测试关卡段的可玩性）、强化学习智能体  $\pi$ （将  $[-1, 1]^{32}$  维输入映射为  $[-1, 1]^{32}$  维输出）、奖励函数  $Reward$ 、最大训练轮次  $T$  和每个关卡的最大片段数  $N$ 。

(2) 训练循环 开始训练循环，只要  $t < T$  则持续进行。在每个训练轮次中，初始化一个空列表  $PreS$  用于存储已生成的关卡片段，初始化片段计数  $n$  为 0，当前片段  $s_n$  为 NULL。

(3) 生成可玩初始片段 反复执行以下操作直到生成可玩的初始片段：从  $[-1, 1]^{32}$  中均匀采样向量  $s$ ；使用关卡段生成模型  $G$  生成关卡片段  $S$  ( $S=G(s)$ )；如果需要，使用  $F$  修复该片段 ( $S=F(S)$ )；利用  $P$  测试片段的可玩性 ( $isPlayable=P(S)$ )；若关卡片段可玩，则将其赋值给  $s_n$ ，并将其添加到  $PreS$  列表。

(4) 生成后续片段并更新智能体 当前生成的关卡片段可玩，且片段数  $n$  小于  $N$  时，执行以下步骤：基于  $s_n$ ，使用强化学习智能体  $\pi$  选择动作  $a_n$  ( $a_n=\pi(s_n)$ )；使用  $G$  根据动作  $a_n$  生成新片段  $S$  ( $S=G(a_n)$ )；修复该片段 ( $S=F(S)$ )；用  $P$  测试片段的可玩性 ( $isPlayable=P(S)$ )；根据  $PreS$  中存储的片段，使用奖励函数  $Reward$

计算得分  $s_n$  和  $a_n$ ，并用该得分更新智能体  $\pi$  的策略。更新 PreS 列表，将 S 添加其中，片段计数  $n$  增加 1。

(5) 训练轮次更新 每完成一个训练轮次，将  $t$  加 1。

(6) 最终返回 当  $t \geq T$  时，训练结束，返回训练好的 RL 智能体  $\pi$ 。

这个算法的核心目的是训练一个强化学习智能体  $\pi$ ，使其能够在平面跳跃游戏中生成一系列的可玩关卡片段。它首先通过关卡段生成模型 G 和关卡段修复模型 F 生成并修复一个可玩的初始关卡片段，然后不断使用智能体  $\pi$  生成后续关卡片段，同时使用 P 来测试其可玩性，使用 Reward 来评估生成的关卡片段，并根据得分更新智能体的策略，最终生成一系列满足一定要求的可玩关卡片段，并输出训练好的智能体。在生成后续关卡片段的过程中，不断更新存储的之前关卡片段列表 PreS 以便于后续的评估和更新操作。

### 3.3 实验分析

本课题针对关卡段生成方法的研究设计了三个实验，第一个是基于代理在关卡中的游戏表现来优化关卡，解决随机潜在向量生成的关卡段可能不可玩或难度不符合预期的问题。第二个是对比随机搜索、CMA-ES<sup>[43]</sup>、MAP-Elites<sup>[44]</sup>、MAP-Elites(line)<sup>[45]</sup>和 CMA-ME<sup>[46]</sup>这 5 种算法在潜在向量搜索的性能。第三个是对比图语法方法和本文的关卡段生成方法在生成关卡多样性方面的表现。

针对长关卡构建策略的研究，本课题设计了一个实验，旨在探究 RL 代理指导关卡生成的能力，重点考察不同奖励函数对关卡生成的影响，评估其生成的关卡在可玩性、机制量和重复度这三方面的表现。

#### 3.3.1 数据集简介

本研究使用的数据集来自视频游戏关卡语料库 (VGLC)，在该语料库中，原始关卡通过特定字符符号表示不同的图块类型。VGLC 的表示方法主要关注图块的功能属性，忽略了它们对应的艺术属性，因此无法区分在视觉上具有差异的图块类型。唯一的例外是管道，它由四种视觉上不同的图块类型表示，尽管这四种类型从功能上看都代表不可通行的地面块。此外，VGLC 仅用单一字符符号表示敌人，未考虑不同敌人类型的功能差异。因此，本研究将所有敌人类型统一映射为通用的板栗仔敌人类型。在编码训练关卡时，每个图块类型用一个不同的整数表示，并在输入到鉴别器之前转换为 one-hot 编码向量。生成器网络输出的关卡同样以 one-hot 编码格式表示，随后将其转换为整数值集合。转换后的整数格式关卡将被传送至 Mario AI 框架进行渲染。



表 3-1 生成的 Mario 关卡中使用的图块类型

| Tile type | Symbol | Identity | Visualization                                                                       |
|-----------|--------|----------|-------------------------------------------------------------------------------------|
| 实心/接地     | X      | 0        |  |
| 可破坏       | S      | 1        |  |
| 空（可通过）    | -      | 2        |                                                                                     |
| 完整的问题块    | ?      | 3        |  |
| 空问题块      | Q      | 4        |  |
| 敌人        | E      | 5        |  |
| 水管左上      | <      | 6        |  |
| 水管右上      | >      | 7        |  |
| 水管左边      | [      | 8        |  |
| 水管右边      | ]      | 9        |  |
| 金币        | o      | 10       |  |

表 3-1 详细展示了 VGLC 的图块类型与符号、模型训练数字编码及 Mario 游戏中的图块之间的映射关系。模型的输入文件是通过处理 VGLC 的原始关卡文件创建的。VGLC 中的每个关卡以纯文本文件保存，每行对应 Mario 关卡中的一行图块。在一个关卡中，所有行的长度相同，关卡的高度为 14 行。为了适应 DCNN-DCGAN 模型的输入需求，每个输入图像是通过在原始关卡上从左到右滑动窗口生成的，窗口大小为  $28 \times 14$ （宽 $\times$ 高），每次窗口仅移动一个图块。

在第三个实验中，采用基于图语法的方法和本文的关卡段生成方法分别生成了 500 个关卡作为实验的数据集。

### 3.3.2 参数设置

针对关卡段生成方法的研究，采用 RMSprop 优化器，训练 DCNN-DCGAN 模型进行了 5000 次迭代，学习率为  $5e^{-5}$ ，批量大小为 32，判别器每迭代 5 次生成器才会迭代 1 次。对于第一个实验，标准差初始化为 2.0，针对两个适应度函数（ $F_1$  和  $F_2$ ）各运行 40 次。对于第二个实验，本课题对于基于表示的行为特征，本课题将天空图块的范围设置为  $[0,150]$ ，将敌人的数量设置为  $[0,25]$ 。地图大小为  $151 \times 26$ ，其中每个单元格对应于行为特征的一个整数值。本课题根据每个算法覆盖基于表示的行为空间的程度来对应调整它，对于 CMA-ME 算法，本课题设置群体大小  $\lambda=37$ ，突变能力  $\sigma=0.2$ 。对于 CMA-ES 算法，本课题设置其群体大小  $\lambda=17$ ，突变能力  $\sigma=0.5$ 。对于 MAP-Elites 算法，本课题设置突变能力  $\sigma=0.2$ 。对于 ME(line)算法，本课题设置各向同性突变  $\sigma_1=0.02$  和方向分布突变  $\sigma_2=0.2$ 。

MAP-Elites 和 ME(line)的初始种群为 100。在随机搜索方法中, 解决方案是通过从训练生成器网络所使用的相同分布中直接采样 GAN 的潜在空间来生成的。潜在空间遵循均值为零、方差为 1 的正态分布。MAP-Elites 和 ME(line)的初始群体也通过相同的方式生成。对于第三个实验, 本课题将 KL 散度范围设为[0, 4.5], 地图分辨率为  $60 \times 60$ 。

针对长关卡构建策略的研究, 在计算多样性 F 时, 滑动窗口和片段大小均为  $14 \times 14$ , 计算基于瓦片的 KL 散度时, 控制参数 epsilon 设为 0.001, 窗口移动次数限制参数 n 设为 3, 窗口步长 d 设为 7; 在计算历史偏差 H 时: 参数 m 设定为 20, k 设定为 10; 训练 RL 智能体时: 采用 PPO 算法进行 106 轮训练, 动作和状态通过 32 维潜在向量表示; 测试训练好的智能体时: 使用 30 个不同关卡片段作为初始状态, 每个初始状态下智能体独立测试 10 次, 每次最多生成 100 个片段(若奖励函数包含可玩性 P, 则在生成不可玩片段时停止); 对于随机智能体  $\pi_R$ , 则从潜空间随机采样至多 100 个片段, 或者直到生成不可玩片段为止; 奖励函数组合时: 当奖励函数由多个指标组成时, 每个指标都根据其最近的 1000 个值进行归一化, 范围为[0,1]。

### 3.3.3 评价指标

针对关卡片段生成方法的研究中, 采用了 Fontaine 等人<sup>[47]</sup>提出的覆盖场景百分比的指标来衡量生成关卡的可玩性, 公式如(3-7)所示。其中  $S_{Valid}$  代表代理完成的关卡进度,  $S_{All}$  代表了该关卡的总体进度。比值越高代表代理完成的进度越高, 即代表关卡可玩性越好。

$$P = \frac{S_{Valid}}{S_{All}} \quad (3-7)$$

采用了由 Pugh 等人提出的 QD-Score<sup>[48]</sup>指标, 对几种算法在潜在向量搜索的能力方面进行评估。QD-Score 是一种定量评估指标, 旨在衡量算法在特定领域中对解决方案数量与质量的探索能力。该指标能够动态反映算法在运行过程中的性能变化: 当算法发现新的时解决方案或提升现有解决方案的质量时, 它将相应增加。为了评估不同方法在生成游戏关卡任务中的表现, 本文还引入了 KL 散度作为衡量关卡片段相似性的评价指标。在游戏关卡生成领域中, KL 散度广泛用于度量生成内容与训练样本之间的分布差异。KL 散度值越大, 表明生成的关卡片段与训练样本的相似性越低, 从而代表更高的多样性。

针对长关卡构建策略的研究中, 本文使用 M、S 和 P 三项独立指标, 分别表示关卡的机制量、重复度和可玩性, 以更全面地刻画所生成关卡的质量特征。

### 3.3.4 实验对比与分析

在第一个实验中，图 3-5 显示了两个适应度函数获得的一些结果。在针对适应度函数  $F_1$  进行优化时，可以得到需要大量必要跳跃的可玩关卡，如图 3-5(a)和(b)所示。前者有较多的管道，后者则是有着高处的砖块需要拾取高层的金币。

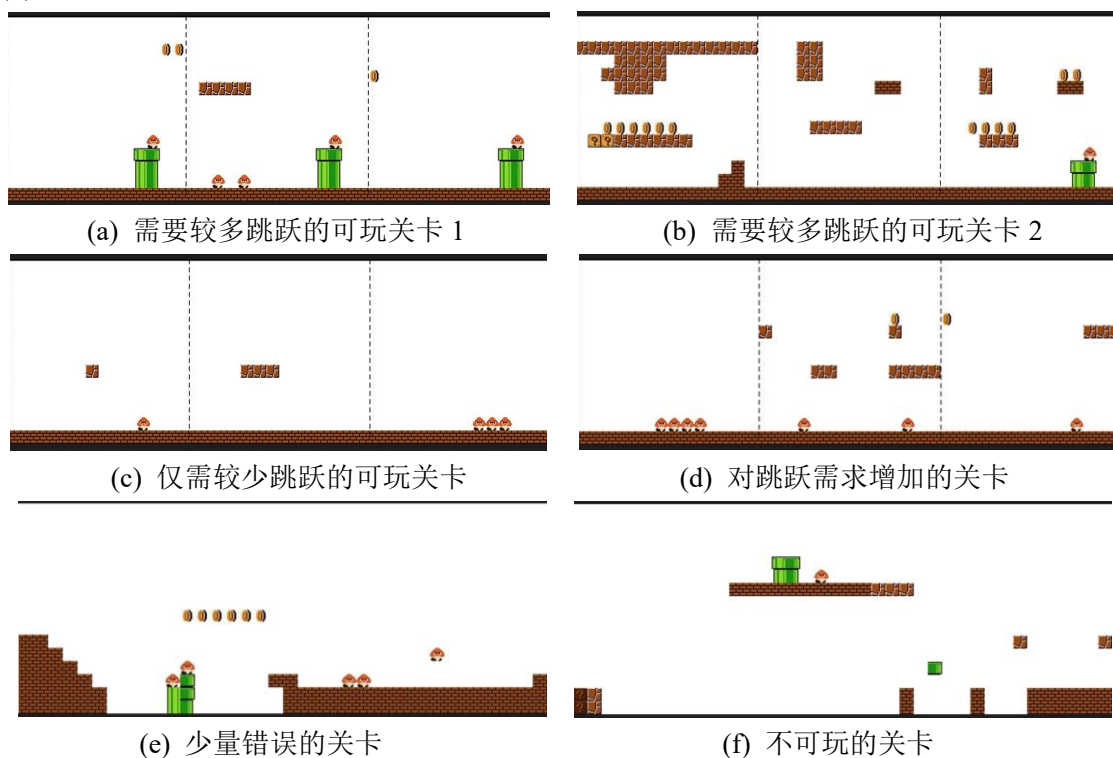


图 3-5 基于代理的优化示例

在针对适应度  $F_2$  实施优化操作时，得到了图 3-5(c)和图 3-5(d)的关卡，前者仅需要少量的跳跃即可完成解决难度较低，后者则有更多的敌人，需要更多一点的跳跃次数。在 CMA-ME 的研究过程中，也检测到了有少量图块错误的关卡，如图 3-5(e)所示，存在着错误管道图块。此外还检测到了不可玩的关卡，如图 3-5(f)所示，生成的关卡图块过高且彼此间隔较大。

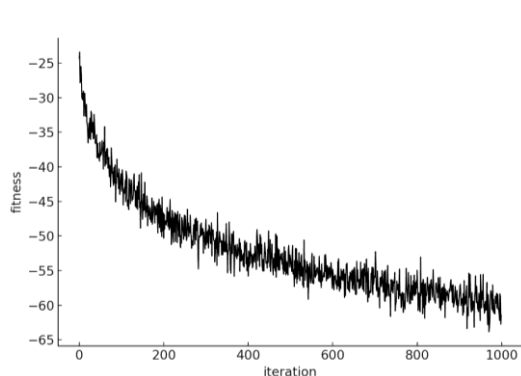


图 3-6 实验中个体平均适应度

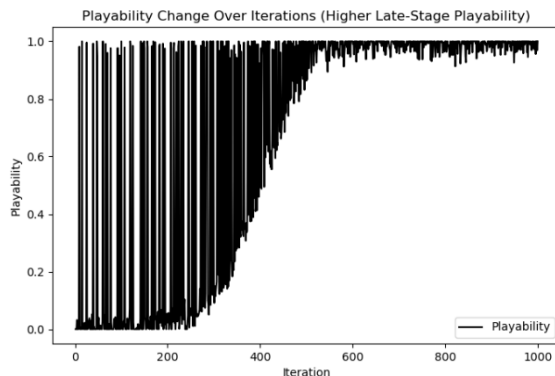


图 3-7 生成关卡的可玩性变化

在实验中，生成个体的平均适应度随迭代次数的增加而下降，如图 3-6 所示。适应度值的整体下降表明，关卡生成过程具有一定的可控性。在后期迭代中，适应度较高的个体可能是由于搜索空间中包含需要大量跳跃次数的关卡，或包含少量不可玩关卡的区域，导致较差表现的个体与这些关卡过于接近。实验过程中生成关卡的可玩性变化如图 3-7 所示，随着迭代次数增加，关卡的可玩性整体逐渐提高。总的来说，通过 CMA-ME 算法的优化，有效提高了生成关卡的可玩性。

第二个实验中，本研究定义了一组行为特征，根据图块的分布来捕获 Mario 场景表示的风格，这些行为特征不依赖于智能体的游戏过程：

- (1) 天空块 这些游戏对象位于特定高度以上，例如方块、问号块和硬币。该数字越大，意味着地面上有更多的游戏元素，玩家需要跳到更高的方块上；
- (2) 敌人数量 敌人数量的增加通常意味着更高的难度，并且需要玩家执行更多的跳跃操作。

本课题使用了 QD-Score 指标来对五种算法进行了评估，评估结果如图 3-8 所示，置信区间为 95%。首先，本课题观察到 MAP-Elites、ME(LINE)和 CMA-ME 这三种算法，在基于表示的行为特征上均表现优于 CMA-ES 和随机搜索。原因在于，CMA-ES 仅针对一个目标（场景的可玩性）进行优化，而未探索多样化的关卡行为。而随机搜索算法的表现较差，原因在于它是从与训练 GAN 的相同分布中进行采样，因此生成的解决方案遵循训练数据的平铺分布，这种分布仅覆盖了行为空间的很小一部分。通过对比可知，CMA-ME 算法在所有算法中的表现是最佳的，因此选择它用于潜在向量的搜索。

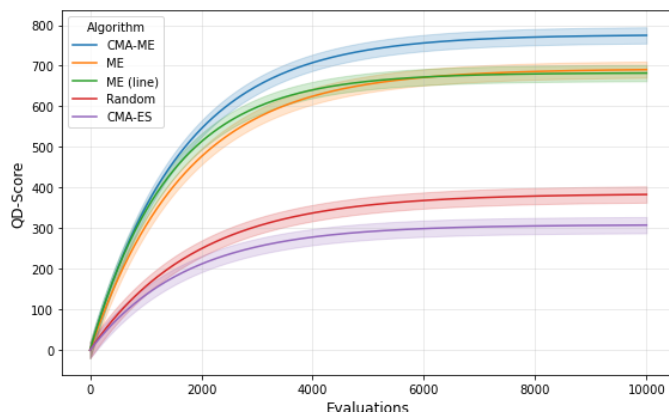
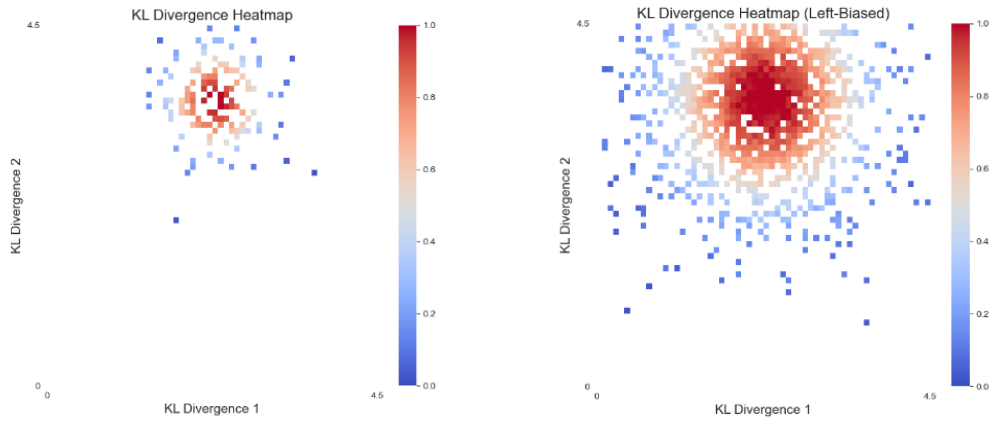


图 3-8 不同 QD 算法的 QD-Score

使用 KL 散度作为指标分别对基于图语法方法生成的关卡和本方法生成的关卡进行了评估，让两个方法分别生成 500 个关卡，然后采用 KL 散度对每个关卡场景评估并分配了独特的颜色，相似的场景会具有相同的颜色。其中图 3-9(a)是

图语法方法生成关卡的 KL 散度，图 3-9(b)是本文的关卡段生成方法生成的关卡，可以看到关卡段生成方法的结果覆盖了更大的区域面积，这代表其生成的关卡之间差异化更大一些，即其更具有多样性，表现更好。



(a) 图语法方法生成关卡的 KL 散度 (b) 关卡段生成方法生成关卡的 KL 散度

图 3-9 生成关卡的 KL 散度

$\pi_M$  代表仅使用机制量  $M$  作为奖励函数训练的 PPO 智能体； $\pi_{MS}$  则表示使用机制量  $M$  和重复度  $S$  之和作为奖励训练的智能体； $\pi_{MSP}$  则使用机制量  $M$ 、重复度  $S$  和可玩性  $P$  三者的综合奖励进行训练。为了验证训练效果，实验使用了 30 个不同的关卡片段作为初始状态。每个初始状态下，代理将独立测试最多 100 个关卡片段，或在奖励函数包含可玩性  $P$  时，直到生成不可玩片段为止。与此对比，随机代理  $\pi_R$  从潜在向量空间随机采样最多 100 个片段，直到生成不可玩片段。生成的关卡在机制量、重复度和可玩性方面的评估结果总结在表 3-2 中。

表 3-2 关卡的评估

| Agent       | Evaluation metrics |              |           |
|-------------|--------------------|--------------|-----------|
|             | M                  | S            | P         |
| $\pi_M$     | 88.1±13.1          | -0.96±0.27   | 30.6±27.3 |
| $\pi_S$     | 58.0±17.8          | -1.53.0±0.33 | 25.2±20.8 |
| $\pi_{MS}$  | 64.4±20.8          | -1.48±0.37   | 17.4±15.2 |
| $\pi_P$     | 72.7±4.1           | -0.73±0.08   | 98.3±10.8 |
| $\pi_{MP}$  | 76.2±4.2           | -0.90±0.09   | 97.6±13.4 |
| $\pi_{SP}$  | 74.7±4.7           | -1.07±0.11   | 98.0±12.9 |
| $\pi_{MSP}$ | 75.0±5.2           | -1.02±0.11   | 98.4±12.7 |
| $\pi_R$     | 65.3±20.8          | -1.42±0.27   | 17.0±13.3 |

结果表明，使用相应奖励函数时，生成关卡的机制量  $M$  和重复度  $S$  评估均达到了最高值。特别地， $\pi_M$  策略平均在 100 个片段中生成 88.1 个具有适当多样性

的关卡片段。类似地， $\pi_S$  策略在  $S$  上达到了最低平均值（-1.53）。当  $P$  作为奖励函数时，关卡的可玩性得到了提升，而与  $M$  和  $S$  结合使用时，可玩性达到了最高水平，即 100 个片段中平均有 98.4 个片段具有可玩性。

对于  $\pi_M$  策略，尽管主要关注敌人和问号方块的布置，但生成的关卡可玩性较差。 $\pi_S$  则能产生关卡重复度较低但不可玩的关卡。 $\pi_{MS}$  的组合增加了关卡中管道的数量，虽然这些关卡有趣，但它们大多数都无法玩。当奖励函数考虑可玩性  $P$  时，生成的关卡质量显著提升，每个关卡 100 个片段中， $P$  值超过 97 的片段占大多数。然而，最佳的策略是同时考虑  $M$ 、 $S$  和  $P$  三者的奖励组合，即  $\pi_{MSP}$  策略，该策略生成的关卡不仅保障了可玩性，而且在机制量和重复度上均有良好表现，平均在 100 个片段中，75 个片段达到了适当的机制量和较低的重度度（平均为-1.02）。

综合来看， $\pi_{MSP}$  策略在生成可玩、机制量适中且低重复度的长关卡内容方面表现最好，因此选择该策略作为构建理想长关卡的代理。图 3-10 展示了基于此策略生成的长关卡示例。

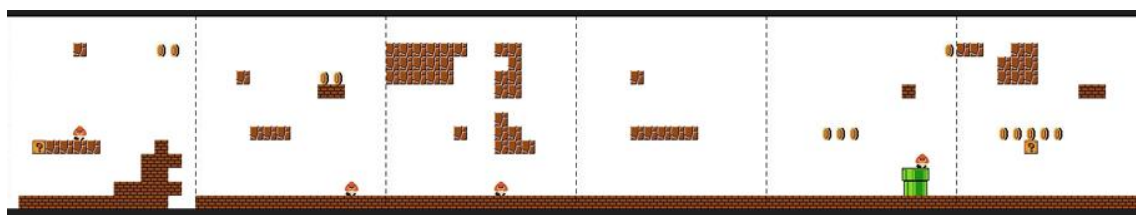


图 3-10 构建的长关卡

### 3.4 本章小结

本章以平台跳跃游戏的关卡生成任务为背景，针对传统方法因依赖固定规则而导致生成关卡模板化的问题，提出一种结合 DCNN-DCGAN 与潜在向量搜索的关卡片段生成方法。该方法利用训练后的生成对抗网络自动生成结构合理的关卡片段，并引入 CMA-ME 算法对潜在向量进行优化搜索，显著提升了生成关卡的可玩性与多样性。实验结果验证了所提出方法在生成具有高可玩性与多样性的关卡方面的有效性。

针对关卡长度增加导致的可玩性下降问题，进一步提出一种基于强化学习的长关卡构建策略。该策略以关卡片段为基本构建单元，结合强化学习技术优化各段的生成与拼接过程。通过设计三类奖励函数——机制量、重复度与可玩性，全面评估与引导关卡生成过程。其中，可玩性通过  $A^*$  代理评估关卡片段是否可通关；机制量基于 KL 散度量相邻关卡片段之间的机制差异，避免内容同质化；重复度则用于衡量新生成片段与历史片段的差异程度，防止内容重复。在训练阶段采用

PPO 算法，并通过与关卡段修复方法和 A\* 代理的结合，进一步优化生成性能。实验结果表明，该基于多重奖励机制的强化学习策略可有效生成可玩性强、机制丰富且重复度适中的长关卡内容。



## 第4章 关卡生成游戏系统设计与实现

本章主要将第2章、第3章所研究的关卡段修复方法、关卡段生成方法和长关卡构建策略应用于实际应用中，设计并实现基于PCG的关卡生成游戏系统。

### 4.1 关卡生成游戏系统需求分析

为了满足快速消费的游戏市场需求，提高游戏开发效率并降低成本是不可避免的。传统的游戏关卡设计通常依赖于人工策划，导致生产效率较低。利用PCG技术进行关卡生成，是提升游戏开发效率的有效手段。基于PCG的关卡生成游戏系统旨在为设计师提供关卡生成和关卡修复的服务，使设计师可以不需要从头设计关卡。本文以平台跳跃游戏关卡为研究载体，结合PCG技术开发一个能够自动生成关卡并替代人工设计的游戏系统。该系统的核心功能是通过训练好的RL代理引导关卡段生成模型生成随机的关卡段，之后利用关卡段修复模型检测并修复其中的错误图块，最终将各个关卡段组合成一个完整的可玩关卡。

#### 4.1.1 关卡生成游戏系统功能性需求分析

基于PCG的关卡生成游戏系统主要功能是为了设计师生成一些关卡或者修复已有的关卡，使得设计师从人工设计关卡转变为可以通过挑选自动生成的内容使用或者借助其进行二次创作，提高了游戏开发的效率。

##### （1）系统用户角色

该系统设定了三种职能不同的角色。关卡研究员负责关卡生成模型和关卡修复模型的训练，利用训练好的模型进行关卡生成和关卡修复；玩家主要负责选择关卡进行游玩，调整游戏中的音乐设置以及对角色进行强化；系统管理员具有最高权限，能够访问所有系统功能并管理相关权限与角色。

##### （2）系统用例图

在角色用例中，关卡研究员主要用例包括模型训练管理用例、关卡生成用例、关卡修复用例、关卡游玩用例和系统登录退出用例。系统管理员的主要用例包括系统登录退出用例、模型训练管理用例、用户配置用例、角色配置用例、权限配置用例、日志查看用例、关卡生成用例、关卡修复用例、关卡游玩用例和角色配置用例。玩家的角色主要用例包含关卡游玩用例和系统登录退出用例。

基于PCG的关卡生成游戏系统用例图如图4-1所示。





系统登录功能用于验证用户身份，并依据它给予用户对应的权限功能。

用户配置功能允许创建用户并分配角色，进行用户信息的维护。

角色配置功能是有关于角色数据管理的功能，目前系统存在的三种角色分别是关卡研究员、系统管理员和玩家。

权限配置功能是给角色配置细化至按钮的相应操作权限。

日志查看功能有关于查看和用户有关的日志，帮助错误排查并支持后期维护。

关卡修复功能可以选择对指定的关卡内容进行修复，并且可以查询关卡修复数据。此外还可以对关卡进行编辑，实现个性化修改。

关卡生成功能可以进行关卡内容生成、对生成的关卡内容进行查询、对关卡的主题和关卡内的敌人以及道具配置进行修改、对关卡任务进行管理（在生成关卡时会随机从关卡任务列表抽取任务）、对关卡事件进行管理（事件根据玩家的行为或关卡状态进行触发，内容包含了如额外的敌人刷新、道具增加和炸弹掉落等）、对关卡库进行管理（即将生成或修复好的关卡内容选择加入游戏的关卡库，以便能够在后续的关卡游玩模块进行选择。

商品信息管理功能可以对商家游戏商店的商品进行管理。

关卡游玩功能包含了游戏的运行逻辑实现，此外可以选择要体验的关卡，可以对游戏的音频大小进行设置和对音乐音效进行替换，可以在游戏商店购买所需的道具或皮肤，还可以对游戏内的角色进行管理（如对角色的外观进行更换以及对角色能力进行强化）。

#### （4）系统功能模块设计

根据上述分析，为本关卡生成游戏系统设计的系统功能模块图如图 4-2 所示。

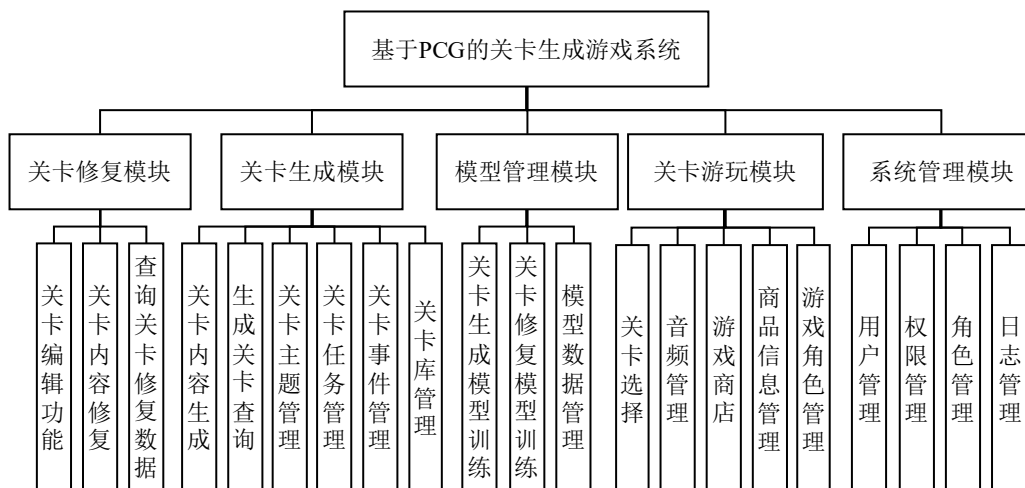


图 4-2 基于 PCG 的关卡生成游戏系统功能模块图

其中关卡研究员角色可以使用关卡修复模块、关卡生成模块、模型管理模块、关卡游玩模块；系统管理员角色有着最高的权限，可使用所有功能模块；玩家角

色可以使用关卡游玩模块除了商品信息管理外的功能。

#### 4.1.2 关卡生成游戏系统非功能性需求分析

基于 PCG 的关卡生成游戏系统的还应该满足以下几点非功能性需求：

##### （1）系统性能需求

系统应具备快速响应、界面元素加载速度快、操作反馈及时等特点，以保证流畅的用户体验<sup>[49]</sup>。特别是在关卡生成或修复命令发出后，系统应迅速反馈结果，避免长时间等待影响用户体验。

##### （2）系统准确性需求

系统应确保关卡修复模型能够准确识别并修复关卡中的错误，否则将增加游戏开发的成本与时间，影响后续的修复和生成过程。

##### （3）系统实时性需求

系统需具备实时监控关卡状态的能力。由于长关卡生成是动态进行的，因此系统必须具备足够快的处理速度，以确保能够在生成过程中及时发现并解决问题。

##### （4）系统鲁棒性需求

考虑到游戏关卡可能包含复杂的元素组合，系统需具备鲁棒性，能够适应各种复杂的关卡设计和游戏逻辑，确保在这些多样化条件下依然能够精确检测和修复，避免系统崩溃或给出错误的修复方案。

##### （5）系统稳定性需求

系统应能够高效生成并修复可玩的游戏关卡，提高游戏开发效率，并为后续的游戏测试与上线提供保障。关卡生成和修复模型的稳定性是高效开发的前提。

##### （6）系统可扩展性需求

游戏关卡的设计理念和要求会随着游戏业的发展不断更新。因此，系统需要具备可扩展性，便于未来添加新的关卡生成模板、修复算法或功能模块，满足新的开发和修复需求。

##### （7）系统安全性需求

系统需要存储关卡数据、模型信息和用户账户信息，因此必须通过权限管理、数据验证和操作提示等方式，确保数据访问受控、输入数据合规，并避免数据丢失或被恶意篡改，保障使用者的数据安全。

#### 4.1.3 关卡生成游戏系统业务流程分析

图 4-3 展示了基于 PCG 的关卡生成游戏系统核心业务处理流程。系统核心功能是通过关卡段生成和关卡段修复得到可玩的关卡段，实现自动化的关卡生成，减少关卡设计的人工成本和时间成本。

首先选择关卡主题、关卡任务和关卡事件。接着，选择适合的关卡段生成模型和关卡段修复模型，加载 RL 代理，并设置好需要生成的关卡数量，系统会根据设定的数量循环生成关卡。当生成数量小于设定数量时，系统先利用关卡段生成模型生成新关卡段，并通过修复模型进行修复，接着通过 RL 代理指导后续的关卡段生成，并将符合的关卡段加入关卡段列表。然后系统会加入关卡任务和关卡事件，并将整个关卡状态进行保存。当生成数量等于设定数量时，系统会打印生成日志，结束关卡生成。

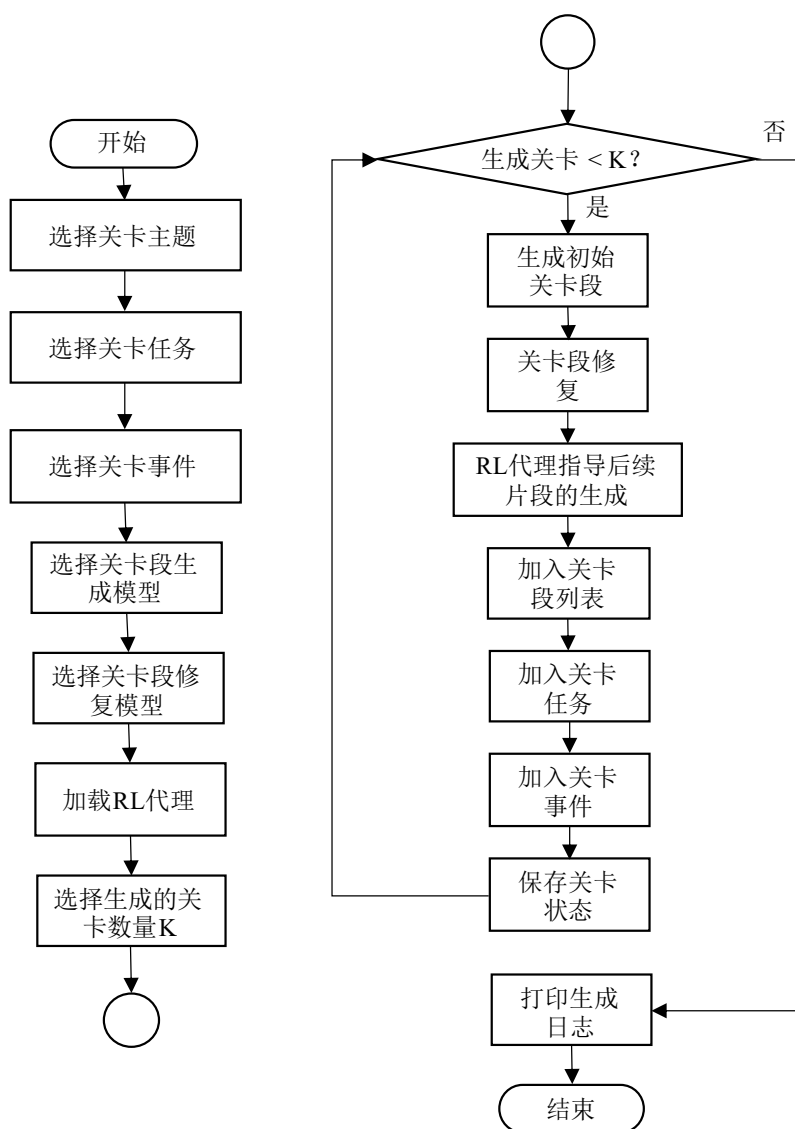


图 4-3 基于 PCG 的关卡生成游戏系统核心业务流程图

## 4.2 关卡生成游戏系统架构

本课题开发的基于 PCG 的关卡生成游戏系统，采用前后端分离的设计模式，总体架构如图 4-4 所示。

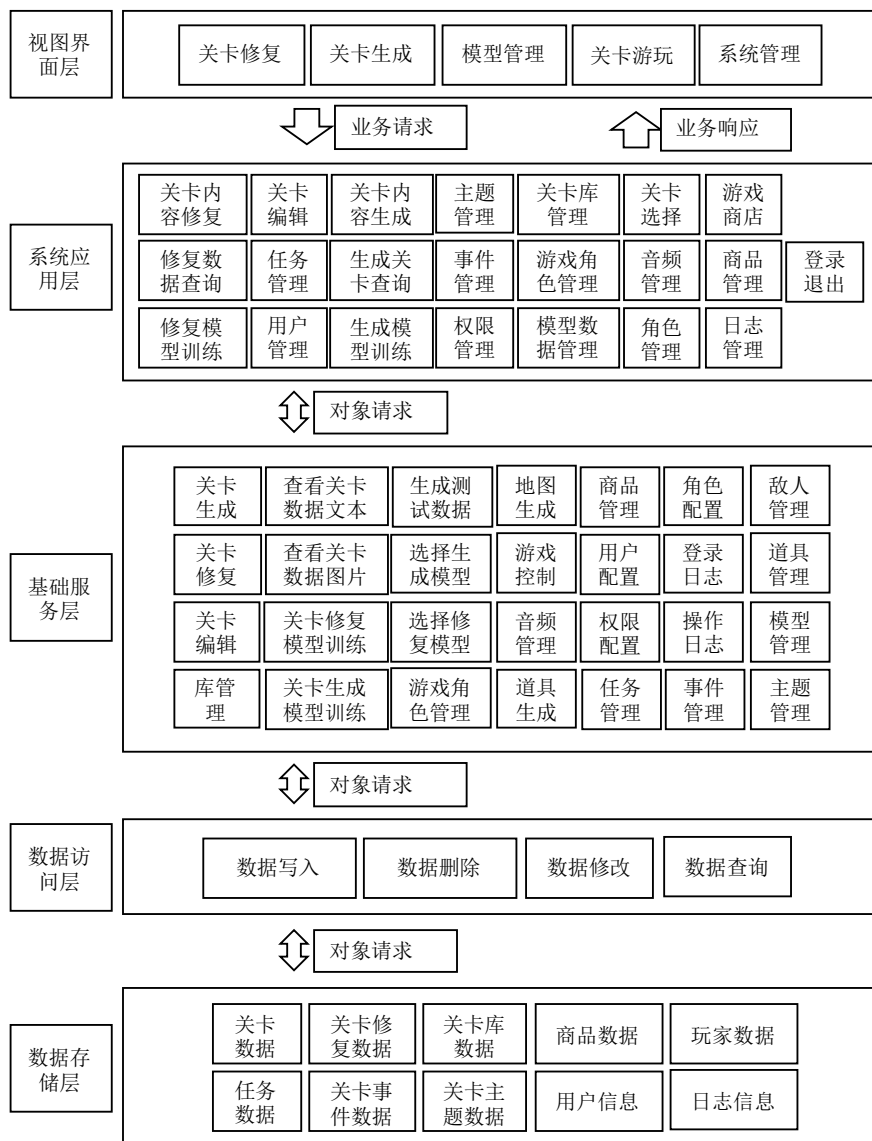


图 4-4 基于 PCG 的关卡生成游戏系统体系结构图

从图 4-4 可以看出，系统自下而上分为五个层次，分别是数据存储层、数据访问层、基础服务层、系统应用层和视图界面层。该架构同一层内的模块各自承担独立职责，相互之间不干扰；而不同层之间则通过协作完成任务，形成纵向的服务链条。当用户在界面上触发操作时，系统会将请求发送至应用层，然后应用层会调用服务层的业务逻辑处理，再通过数据访问层与数据存储层交互，最后将响应结果自下而上传递至视图层展示给用户。

基于 PCG 的关卡生成游戏系统体系结构各层次功能如下：

(1) 视图界面层 作为体系结构的最上层，视图界面层负责与用户直接交互。其主要功能包括关卡修复、关卡生成、模型管理、关卡游玩和系统管理。视图层接收来自应用层的业务响应，并将结果呈现给用户。

(2) 系统应用层 该层作为用户界面和业务处理的中介,主要负责执行关卡内容修复、关卡编辑、关卡内容生成、主题管理、关卡库管理、关卡选择、游戏商店、修复数据查询、任务管理、生成关卡查询、事件管理、游戏角色管理、音频管理、商品管理、登录退出、修复模型训练、用户管理、生成模型训练、权限管理、模型数据管理、角色管理、日志管理等功能。系统应用层通过提供接口,依据用户需求向基础服务层发送请求,完成具体的业务处理。

(3) 基础服务层 该层负责系统核心业务逻辑的处理,包括关卡生成、查看关卡数据文本、生成测试数据、地图生成、商品管理、角色配置、敌人管理、关卡修复、查看关卡数据图片、选择生成模型、游戏控制、用户配置、登录日志、道具管理、关卡编辑、关卡修复模型训练、选择修复模型、音频管理、权限配置、操作日志、模型管理、库管理、关卡生成模型训练、游戏角色管理、道具生成、任务管理、事件管理、主题管理。基础服务层根据系统应用层的请求,执行相应的业务逻辑,并通过与数据访问层的交互来获取和处理所需数据。

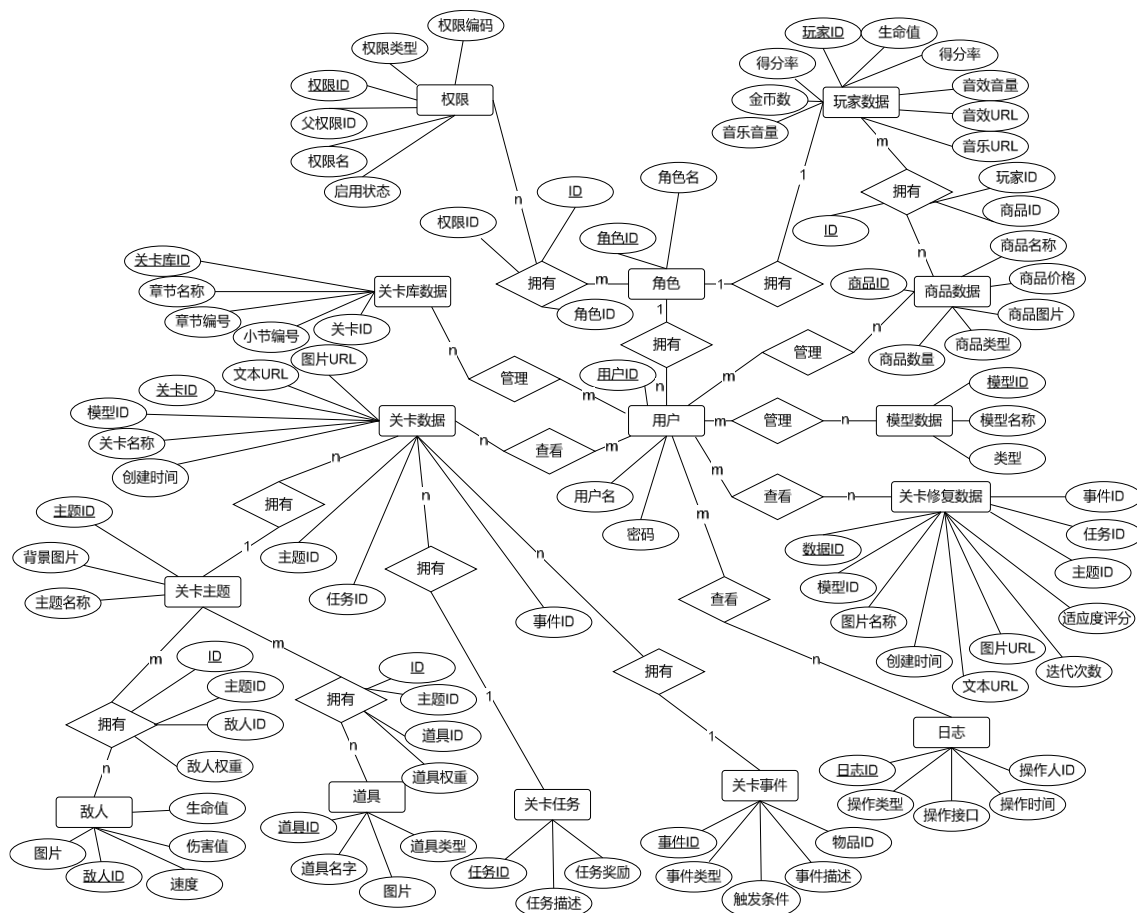
(4) 数据访问层 该层的主要职责是根据基础服务层的请求,对数据存储层进行数据的增删改查操作,并将结果反馈给基础服务层。

(5) 数据存储层 位于体系结构的最底层,负责存储关卡数据、关卡修复数据、关卡库数据、任务数据、关卡事件数据、关卡主题数据、商品数据、玩家数据、用户信息和日志信息,为数据访问层提供数据支持。

### 4.3 关卡生成游戏系统数据库构建

本系统使用 MySQL 数据库来持久化存储关卡数据、关卡修复数据、模型信息、角色信息及操作日志等各类数据。基于 PCG 的关卡生成游戏系统的 E-R 图如图 4-5 所示,图中展示了 15 种实体类型,包括用户、权限、日志、角色、玩家数据、关卡数据、关卡修复数据、模型信息、关卡主题、关卡库数据、敌人、道具、关卡任务、关卡事件、商品数据。

各个实体间的关系描述如下:角色实体与用户是一对多的关系,即多个用户可以被分配到相同的角色,每个用户可以被分配一个角色;权限与角色之间是多对多关系,一个权限可以被多个角色分配,且同一个角色可以具有多个权限;玩家数据和角色之间是一对一的关系,一个角色只能有一份玩家数据;用户与模型之间也是多对多关系,即一个模型可以由不同用户管理,而一个用户可以管理多个模型;用户与日志之间是多对多关系,即一个用户可以查看多个日志,而同一个日志可以支持被不同用户访问;关卡数据实体包括关卡 ID、名称和文本 URL 等属性。它与用户之间是多对多关系,即一个用户可以访问多个关卡数据,而一



关卡主题实体与关卡数据是一对多的关系，即一个关卡数据具有一个关卡主题，一个关卡主题会被分配到多个不同的关卡数据中；关卡任务实体与关卡数据是一对多的关系，即一个关卡数据具有一个关卡任务，一个关卡任务会被分配到多个不同的关卡数据中；关卡事件实体与关卡数据是一对多的关系，即一个关卡数据具有一个关卡事件，一个关卡事件会被分配到多个不同的关卡数据中；道具实体与关卡主题之间是多对多关系，一个关卡主题可以存在多个道具，且同一个

道具可以出现在多个关卡主题中，道具与关卡主题的多对多关系需要额外建表存储；敌人实体和关卡主题之间是多对多关系，一个关卡主题可以存在多种敌人，而同个敌人可以出现在多个关卡主题中，它们的多对多关系需要额外建表存储。

在数据库中有关卡生成表和关卡修复数据表两个关键数据表，详细描述如下：

#### (1) 关卡数据表

关卡数据表结构说明如表 4-1 所示。id 字段是关卡数据的唯一标识，model\_id 是调用关卡段生成模型的 ID，creation\_time 是关卡被创建的时间，txt\_url 为关卡文本对应 URL，pic\_url 为关卡图片对应 URL，theme\_id 为关卡主题 ID，task\_id 为关卡任务 ID，event\_id 为关卡事件 ID。

表 4-1 关卡数据表结构说明

| 字段名           | 类型        | 描述     | 主键  |
|---------------|-----------|--------|-----|
| id            | varchar   | 关卡 ID  | YES |
| model_id      | varchar   | 模型 ID  | NO  |
| level_name    | varchar   | 关卡名称   | NO  |
| creation_time | timestamp | 创建时间   | NO  |
| txt_url       | varchar   | 文本 URL | NO  |
| pic_url       | varchar   | 图片 URL | NO  |
| theme_id      | varchar   | 主题 ID  | NO  |
| task_id       | varchar   | 任务 ID  | NO  |
| event_id      | varchar   | 事件 ID  | NO  |

#### (2) 关卡修复数据表

表 4-2 关卡修复数据表结构说明

| 字段名           | 类型        | 描述     | 主键  |
|---------------|-----------|--------|-----|
| id            | varchar   | 数据 ID  | YES |
| model_id      | varchar   | 模型 ID  | NO  |
| pic_name      | varchar   | 数据名称   | NO  |
| creation_time | timestamp | 创建时间   | NO  |
| txt_url       | varchar   | 文本 URL | NO  |
| pic_url       | varchar   | 图片 URL | NO  |
| iteration_num | varchar   | 迭代次数   | NO  |
| fitness_score | varchar   | 适应度评分  | NO  |
| theme_id      | varchar   | 主题 ID  | NO  |
| task_id       | varchar   | 任务 ID  | NO  |
| event_id      | varchar   | 事件 ID  | NO  |

存储系统关卡修复数据的关卡修复数据表，用于存储系统用户执行的每一次



关卡修复操作的相关信息。其数据表结构说明如表 4-2 所示。`id` 字段是关卡修复数据的唯一标识, `model_id` 是调用关卡段修复模型的 ID, `creation_time` 是关卡修复数据被创建的时间, `txt_url` 为关卡修复数据的文本对应 URL, `pic_url` 为关卡修复数据的图片对应 URL, `iteration_num` 是关卡修复数据的迭代次数, `fitness_score` 是关卡修复数据的适应度评分, `theme_id` 为关卡主题 ID, `task_id` 为关卡任务 ID, `event_id` 为关卡事件 ID。

除此上述的两个关键数据表, 系统数据库中还有着商品信息表、操作日志表 and 用户信息表等数据表, 在此就不作赘述。

## 4.4 关卡生成游戏系统功能模块设计与实现

### 4.4.1 关卡修复模块

关卡修复模块旨在提高随机生成关卡的可玩性与逻辑合理性, 主要包括关卡内容修复、查询关卡修复数据和关卡编辑三个子模块。关卡内容修复模块基于第 2 章中训练得到的关卡段修复模型, 利用 `LevelErrorNet` 检测并标记错误图块, 并采用 `GA` 算法进行修复, 确保修复后的关卡符合游戏逻辑。查询关卡修复数据模块支持用户检索与查看已修复关卡的文本与图片数据, 实现修复数据的管理。关卡编辑模块提供关卡编辑功能, 用户可选择并修改关卡图块, 系统将对编辑后的关卡转换并存储。

#### (1) 关卡内容修复模块

关卡内容修复模块是通过利用训练好的关卡段修复模型, 实现对随机生成关卡中错误图块的检测与修复, 该模块的实现时序图如图 4-6 所示。首先, 用户选择需要修复的关卡段和修复模型, 系统调用模型接口, 先利用 `LevelErrorNet` 检测并标记存在逻辑错误的图块, 接着返回标记后的数据并采用 `GA` 算法来对存在逻辑错误的图块数据进行替换, 将其调整为满足游戏逻辑的图块。修复完成后, 系统将可视化结果和关卡文件保存, 返回关卡段修复信息, 提示用户修复完成。

#### (2) 查询关卡修复数据模块

查询关卡修复数据模块提供了对关卡修复数据的查询功能, 实现流程如图 4-7 所示。用户在进入查询关卡修复数据界面后, 系统将查询已完成修复的关卡修复数据记录, 按照逻辑进行列表查询后得到关卡修复数据信息列表。接着用户选择任意关卡修复数据的文本或图片进行查看, 系统再根据用户的选择查询并打开相应的文件内容。

#### (3) 关卡编辑模块

本模块提供了对关卡文件编辑的功能，实现流程如图 4-8 所示。用户在进入关卡编辑界面后，选择要编辑的关卡文件，系统在查询并返回关卡数据结果后，将它进行解码成二维的地图格式。用户选择并对地图的图块进行替换，在用户完成修改后，系统将其修改保存，接着将修改后的地图重新编码成文本格式，然后将其进行存储并返回存储结果。

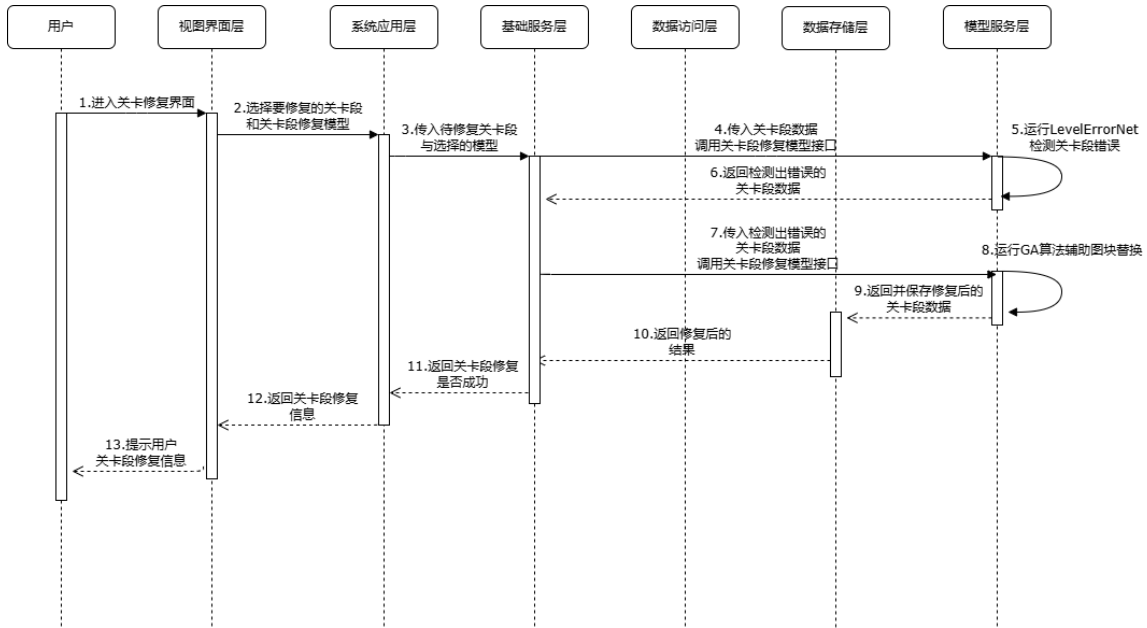


图 4-6 关卡内容修复模块业务时序图

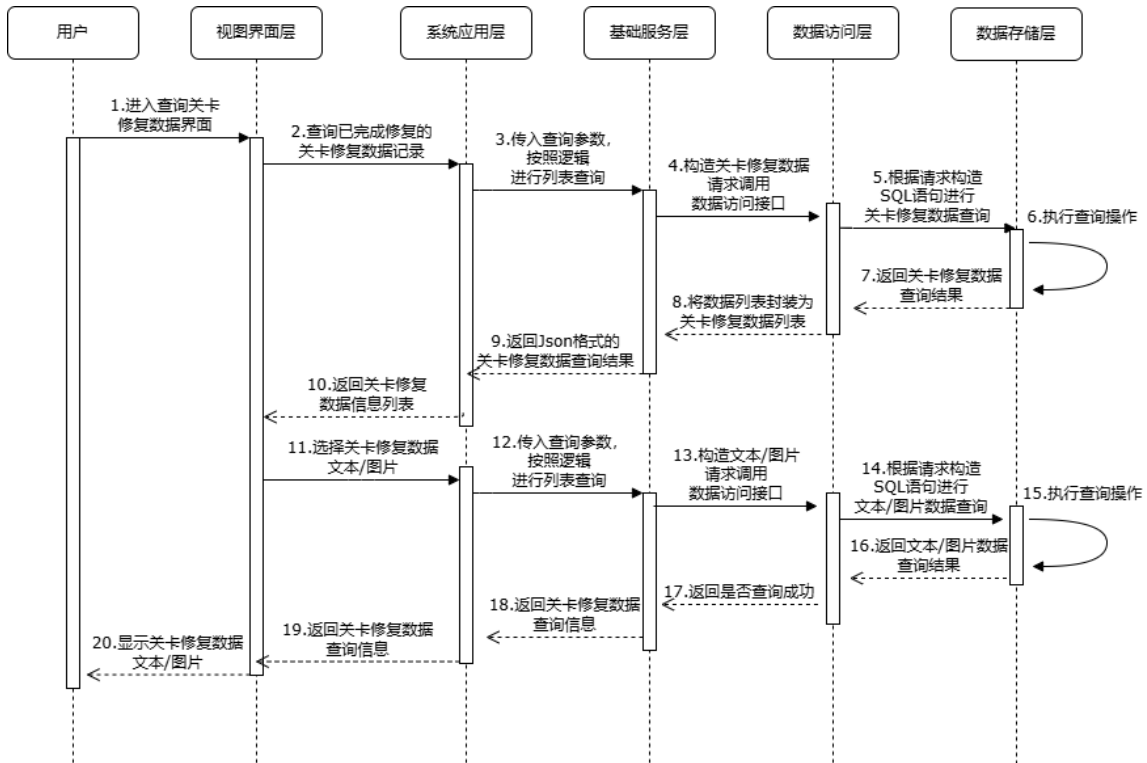


图 4-7 查询关卡修复数据模块实现时序图

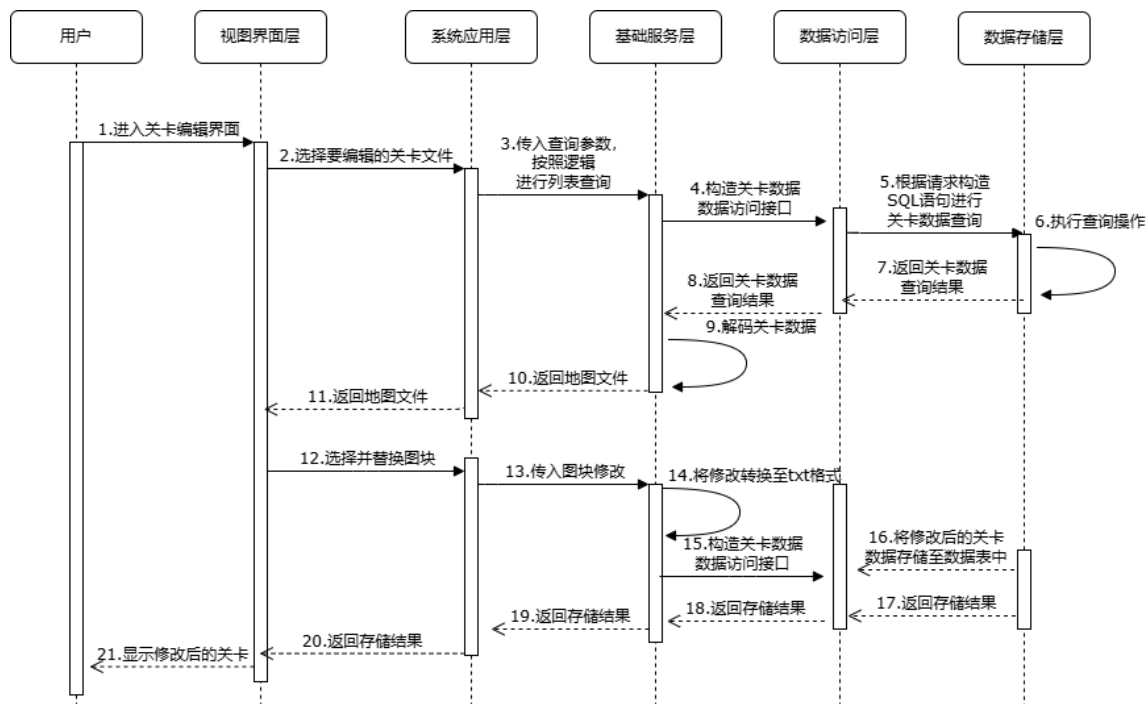


图 4-8 关卡编辑模块实现时序图

#### 4.4.2 关卡生成模块

关卡生成模块负责生成与管理游戏关卡，包含关卡内容生成、查询生成关卡数据、关卡主题管理、关卡任务管理、关卡事件管理和关卡库管理六个子模块。关卡内容生成模块结合训练好的关卡段生成模型、关卡段修复模型和 RL 代理来进行关卡内容生成。查询生成关卡数据模块提供对已生成关卡的查询功能。关卡主题管理模块允许用户管理关卡主题，包括敌人和道具的权重设定。关卡任务管理模块支持对关卡任务的创建与维护。关卡事件管理模块提供对关卡事件的创建与维护管理。关卡库管理模块用于新增与维护游戏关卡库。

##### (1) 关卡内容生成模块

关卡内容生成模块利用第 2 章训练好的关卡段修复模型和第 3 章训练好的关卡段生成模型，以及第 3 章训练好的 RL 代理来进行关卡内容生成，该模块的实现流程如图 4-9 所示。首先，用户选择好关卡的主题、事件和任务，然后选择所需关卡段生成模型和修复模型，系统将 RL 代理加入并调用模型接口生成初始关卡段。在此过程中，系统利用 CMA-ME 算法搜索潜在向量并将其输入生成模型生成关卡段，同时系统会调用关卡段修复模型对生成的关卡进行修复优化。将生成结果返回后，系统通过奖励函数评估优化关卡段并将其加入关卡段列表，接着重复生成新关卡段的流程直到判定结束长关卡生成。最后系统将选择的关卡主题、任务和事件与关卡文件信息一起保存进数据库表中，逐级返回生成结果并提示用

户关卡内容生成结束。

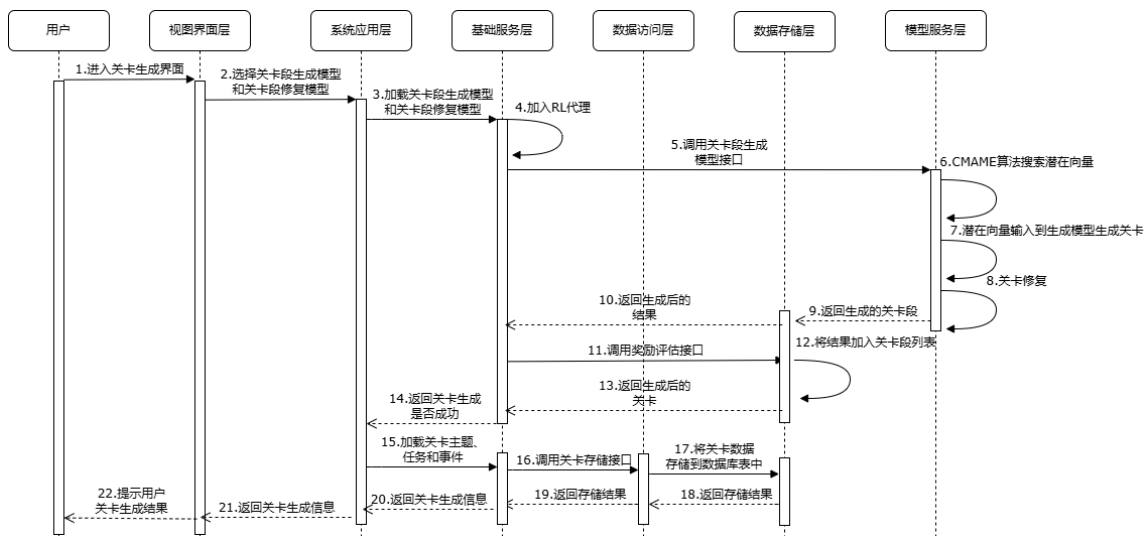


图 4-9 关卡内容生成模块实现时序图

## (2) 查询生成关卡数据模块

查询生成关卡数据模块的主要功能是为用户提供生成关卡数据的访问和展示，该模块的实现流程如图 4-10 所示。用户在进入查询生成关卡数据界面后，系统将查询已完成生成的生成关卡数据，按照逻辑进行列表查询后得到生成关卡数据信息列表，然后用户选择要查看的关卡数据的文本或图片，系统再根据用户的选择查询并打开对应的文件内容。

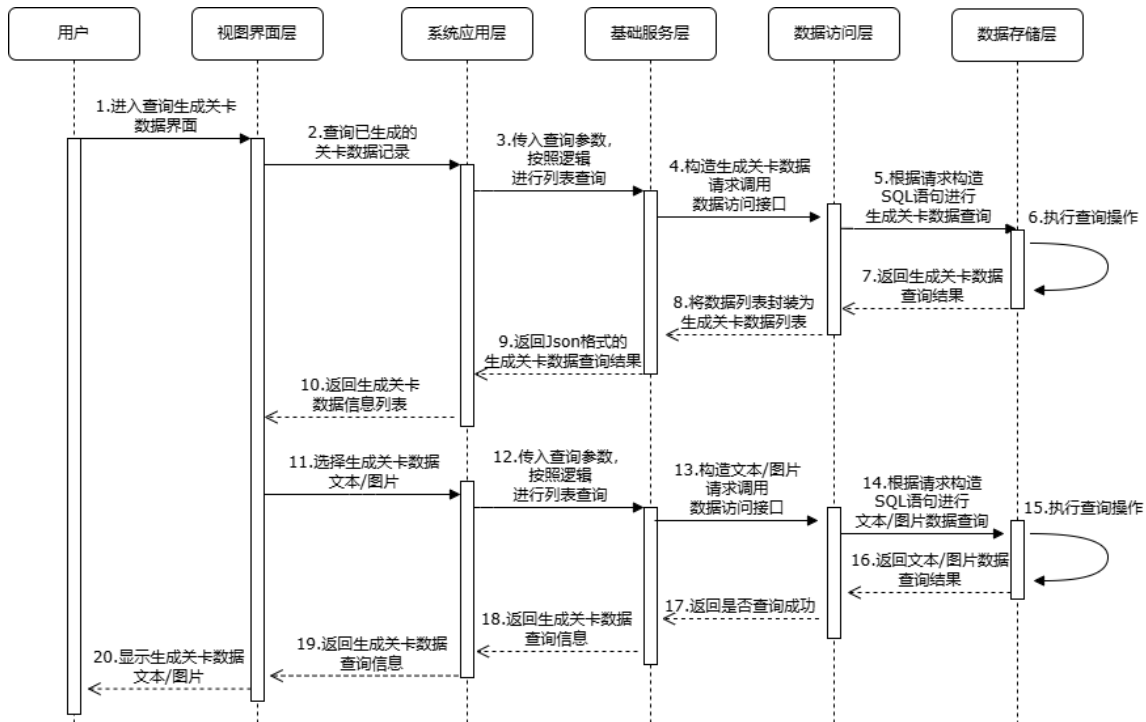


图 4-10 查询生成关卡数据模块实现时序图

### (3) 关卡主题管理模块

关卡主题管理模块的主要功能是为用户提供便捷的对游戏关卡主题进行管理的功能，新增关卡主题的实现流程如图 4-11 所示。用户在新建主题后，在系统给出的表单输入主题名称并上传主题图片，系统查询是否已有相同名称，并返回查询结果。接着用户选择敌人类型并设定其权重，然后选择道具类型并设定其权重。完成后点击完成创建按钮，系统将关卡主题数据写入数据表中，最后返回主题添加成功。

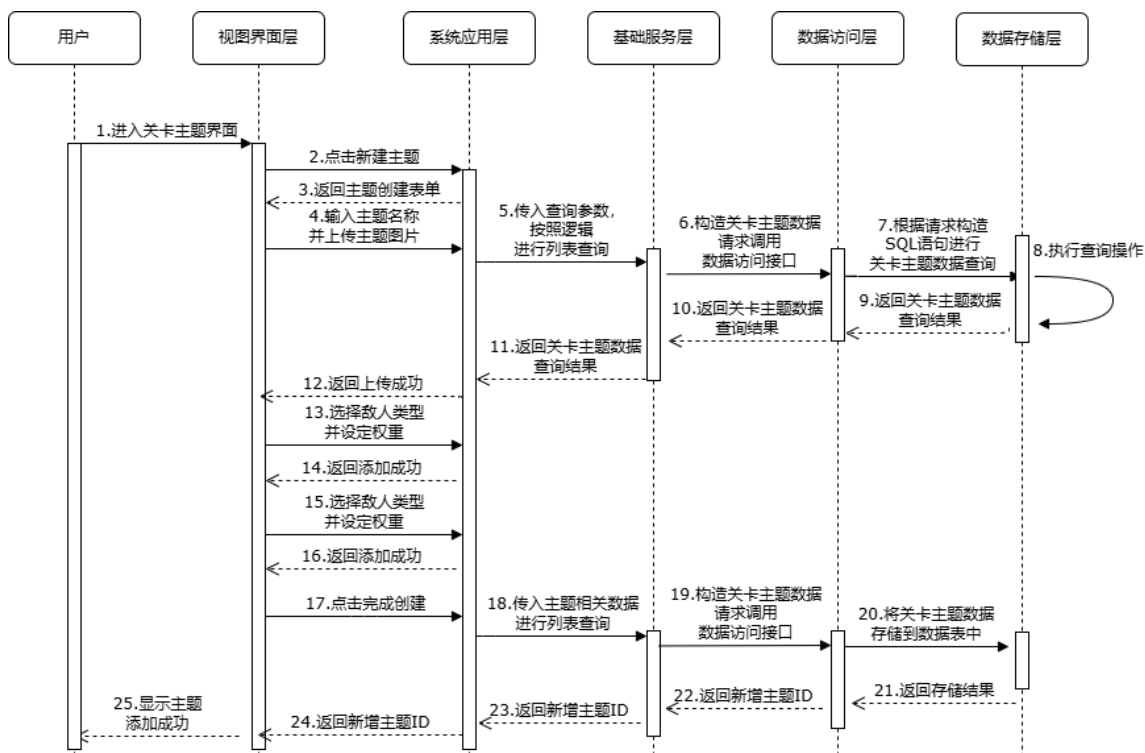


图 4-11 新增关卡主题实现时序图

### (4) 关卡任务管理模块

关卡任务管理模块的主要功能是为用户提供对游戏关卡任务数据的管理功能，该模块新增关卡任务的实现流程如图 4-12 所示。用户在进入关卡任务管理界面后，点击新建关卡任务，系统返回关卡任务表单供用户填写任务描述和奖励信息。接着用户输入任务数据并点击保存后，系统将任务数据传入，由基础服务层调用数据接口执行存储操作。数据访问层将任务数据写入数据库，返回存储结果及生成的任务 ID。系统返回新增任务 ID 及操作状态至视图界面层，最终向用户提示新增关卡任务成功。

### (5) 关卡事件管理模块

关卡事件管理模块提供了对关卡事件的创建与管理功能，该模块新增关卡事

件的实现流程如图 4-13 所示。用户进入关卡事件管理界面后，点击新建关卡事件，系统返回事件表单。用户选择事件触发条件类型，系统通过基础服务层构造触发物品参数并调用数据访问接口查询关联物品数据。数据访问层执行 SQL 查询，返回符合条件的物品列表。用户输入事件描述并提交完整事件数据，系统将事件数据通过标准化接口保存，存储成功后返回新增事件的唯一 ID，最终界面提示新增关卡事件成功。

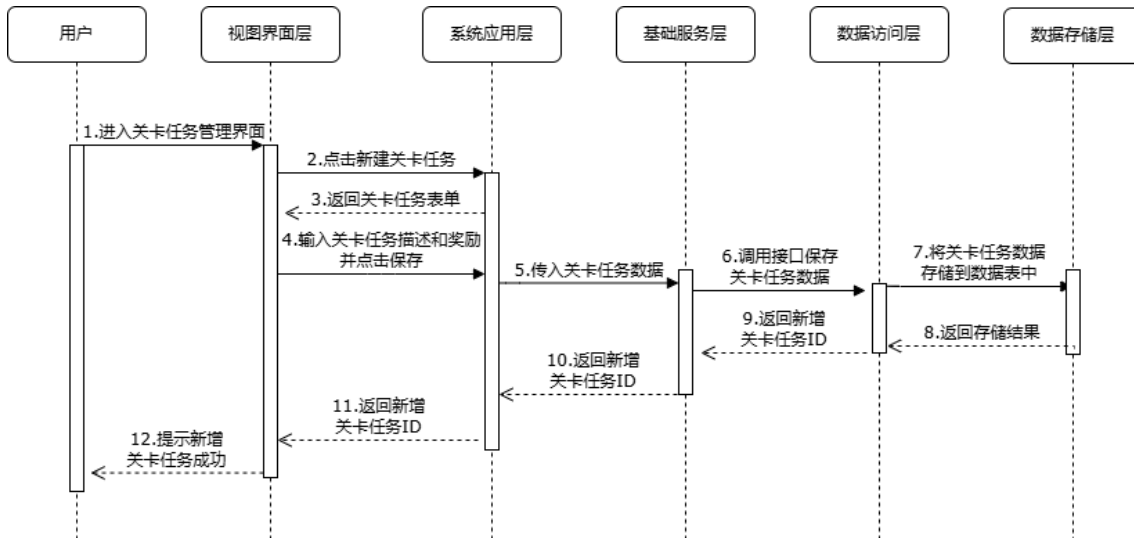


图 4-12 新增关卡任务实现时序图

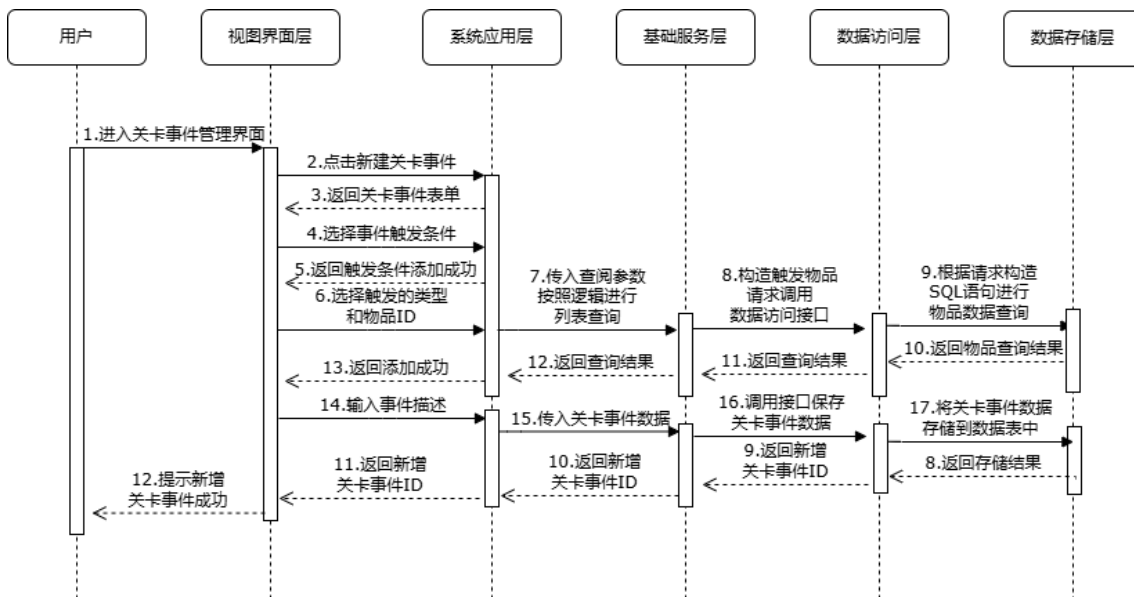


图 4-13 新增关卡事件实现时序图

#### (6) 关卡库管理模块

关卡库管理模块提供对游戏关卡库数据的新增与管理功能，该模块的新增关卡库关卡的实现流程如图 4-14 所示。用户进入关卡库管理界面后，点击新增关卡

按钮，系统返回新增关卡的表单。用户选择所属章节并填写关卡库数据，系统在校验数据合法性后，调用接口保存该新增关卡库数据，将它写入数据库的指定表中，然后逐级返回存储结果及生成的自增关卡 ID，最终在界面向用户提示新增关卡库数据成功。

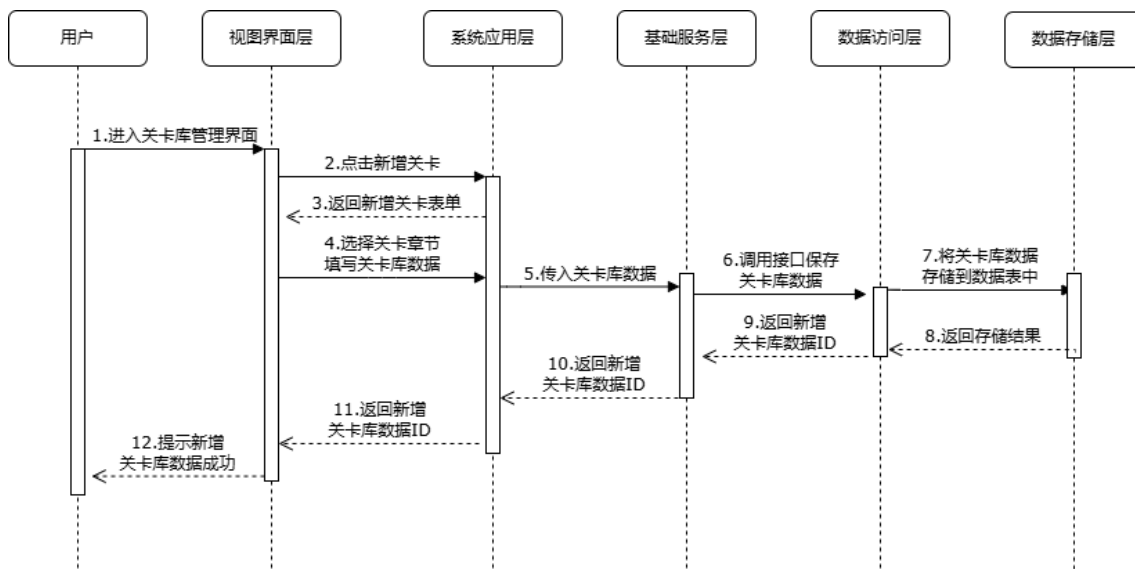


图 4-14 新增关卡库关卡实现时序图

#### 4.4.3 模型管理模块

模型管理模块负责关卡生成与修复模型的训练及管理，包含关卡生成模型训练、关卡修复模型训练和模型数据管理三个子模块。

##### (1) 关卡生成模型训练模块

该模块主要是负责提供训练关卡生成模型的功能，实现流程如图 4-15 所示。该模块的流程是首先选择加载数据集和游玩代理，接着加载它们并调用关卡段生成模型训练接口训练模型，训练完毕后将训练好的模型和训练日志一并保存，最后提示用户训练完成。

##### (2) 关卡修复模型训练模块

该模块主要是负责提供训练关卡修复模型的功能，实现流程如图 4-16 所示。该模块的流程是首先点击生成测试数据集，然后调用接口进行测试数据集生成。待数据集生成成功，在页面向用户返回提示，然后用户可以点击按钮进行下一步的模型训练。训练完毕后将训练好的模型和训练日志一并保存，最后提示用户训练完成。

##### (3) 模型数据管理模块

该模块主要是用于管理关卡生成模型和关卡修复模型的文件，用户可以在本

界面对模型数据进行修改，在此不多做赘述。

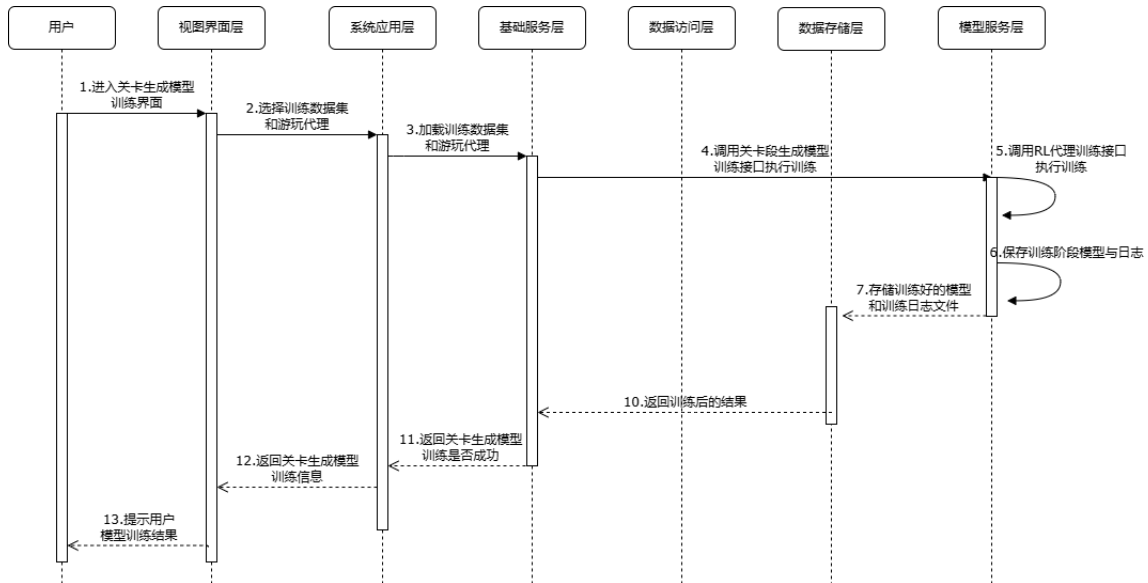


图 4-15 关卡生成模型训练模块实现时序图

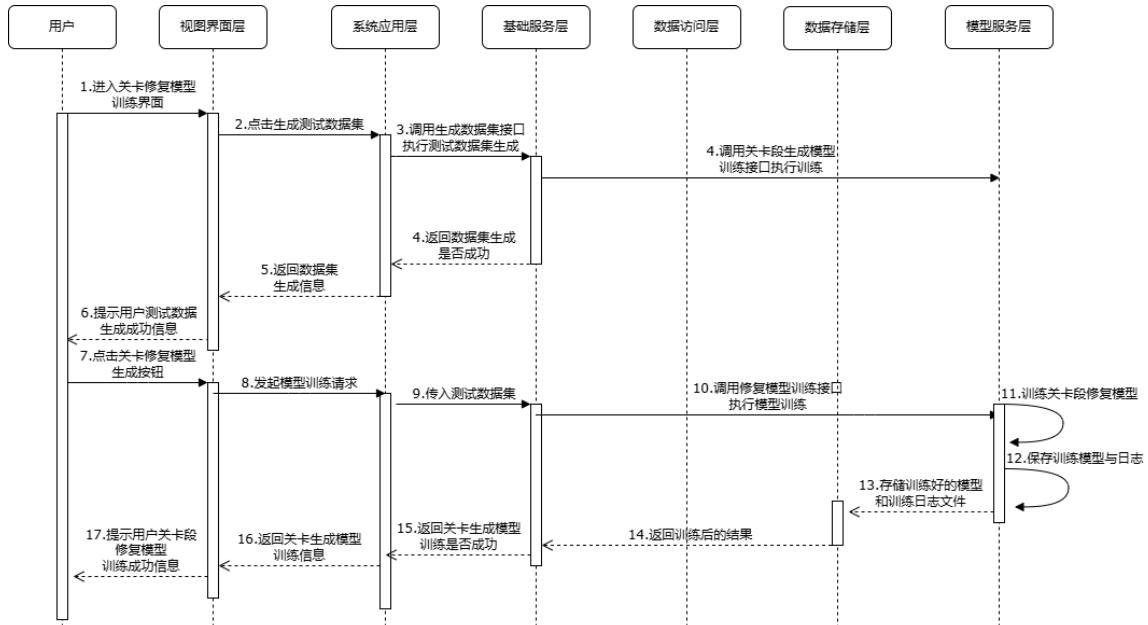


图 4-16 关卡修复模型训练模块实现时序图

#### 4.4.4 关卡游玩模块

关卡游玩模块提供了游戏关卡的游玩、音频调整、商店购买、商品管理及角色管理等功能，包含关卡选择、音频管理、游戏商店、商品信息管理和游戏角色管理五个子模块。本模块还包含了游戏的基本功能逻辑构建，在玩家选择好了指定关卡后，系统将通过调用对应的接口完成可以运行的游戏关卡的创建。关卡选择模块允许用户或代理游玩选定的关卡，并负责加载游戏资源、初始化角色与控



制游戏逻辑，同时记录游戏运行结果。音频管理模块支持背景音乐和音效的个性化调整，并实时保存用户的音频设置。游戏商店模块提供道具和皮肤的购买功能，确保玩家数据符合购买条件后完成交易。商品信息管理模块支持对商店中的商品数据进行修改与维护。游戏角色管理模块允许用户调整角色的生命值、得分率及外观，并支持强化或重置角色属性，以提升游戏体验。

### (1) 关卡选择模块

关卡选择模块提供了关卡游玩相关的功能，模块的实现流程如图 4-17 所示。该模块流程首先是用户选择要游玩的关卡，系统调用接口对关卡有关的角色、音频、定时器和游戏逻辑进行初始化与加载；在开启了代理游玩的情况下，系统会加载代理，它会负责游玩后续关卡内容；在未开启代理的情况下，用户通过输入指令游玩创建好的关卡内容，系统实时调用游戏控制逻辑接口对游戏运行行为进行计算与控制；当游戏结束后，系统保存并返回游戏运行结果与日志，流程结束。

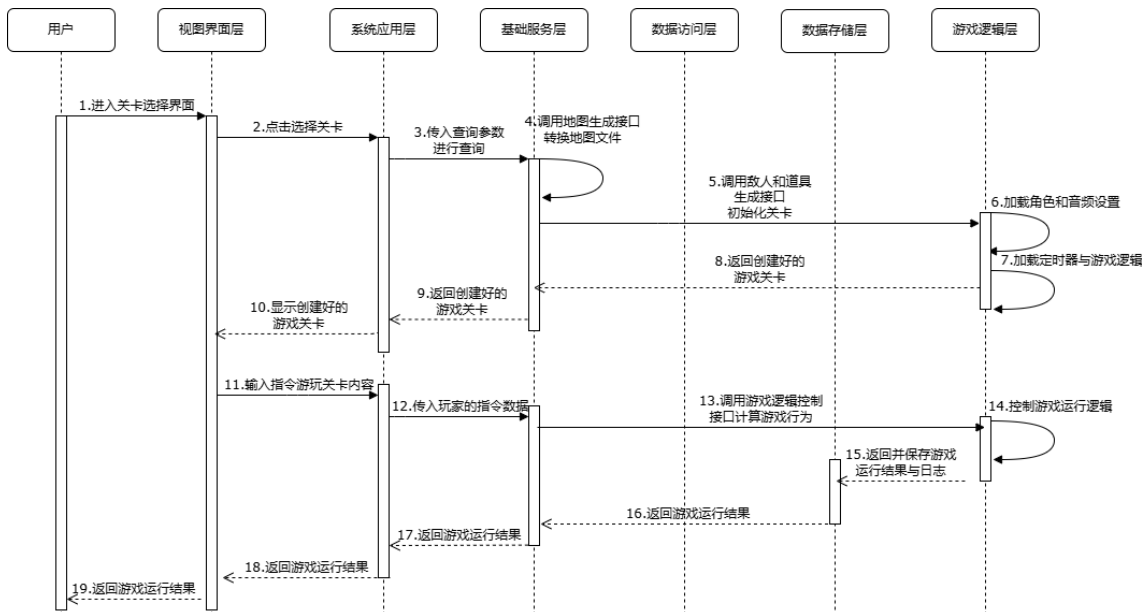


图 4-17 关卡选择模块实现时序图

### (2) 音频管理模块

音频管理模块提供了对游戏音频的个性化调整功能，实现流程如图 4-18 所示。音频管理模块提供了对游戏音频的个性化调整功能。用户可以选择是否开启背景音乐或音效，并根据需要分别调整音乐和音效的音量大小。在页面对音乐和音效数据进行修改后，系统会保存用户的修改，以便在游戏中即时生效。

### (3) 游戏商店模块

游戏商店模块主要用于购买游戏中的皮肤和道具，购买游戏商店商品的流程图如图 4-19 所示。在用户选择了需要购买的商品后，系统查询用户的玩家数据是否符合购买要求，符合则对该玩家数据进行修改更新，然后对商品数据也做一并

修改，最后提示商品购买成功。

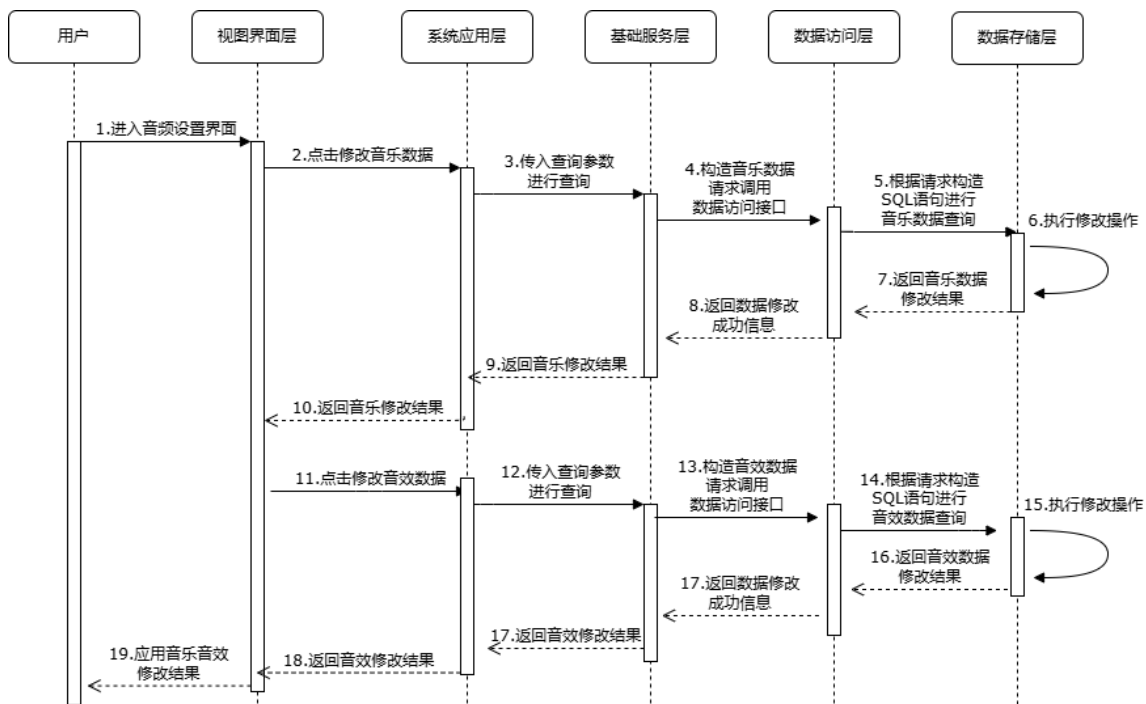


图 4-18 音频管理模块实现时序图

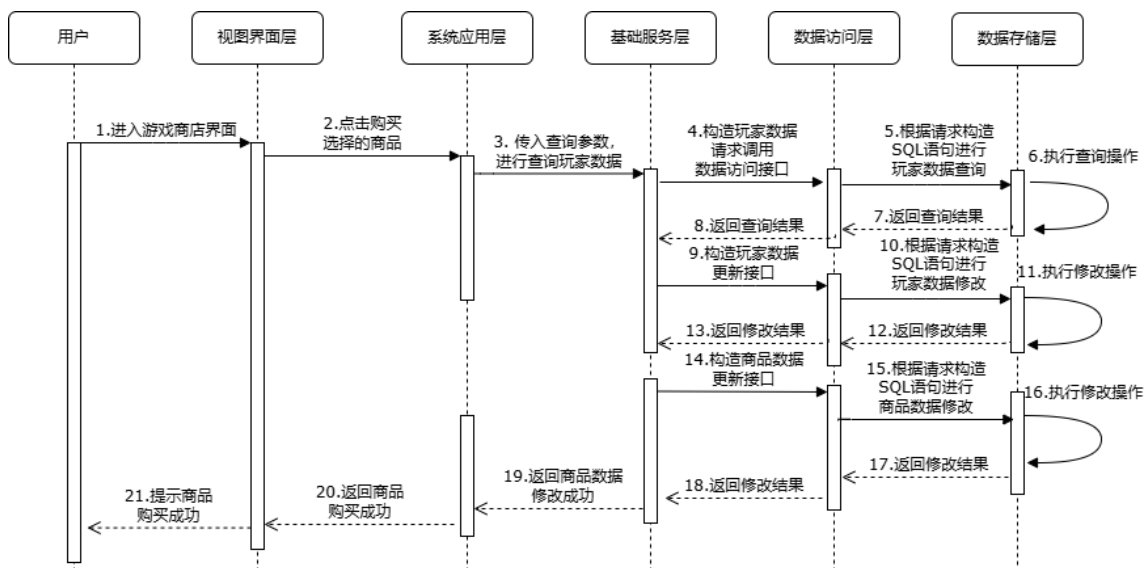


图 4-19 购买游戏商店商品实现时序图

#### (4) 商品信息管理模块

商品信息管理模块主要用于对游戏商店中贩售的商品进行管理，修改商品信息的流程图如图 4-20 所示。用户进入商品信息管理界面后，点击选择需要修改的商品，系统构造数据访问接口对数据库中的商品数据进行查询，接着返回查询到的商品数据。用户输入对商品数据的修改并点击保存，系统将修改后的商品数据存储至数据库表中，然后逐层反馈存储状态，最终向用户提示商品数据修改成功

并刷新界面展示最新数据。

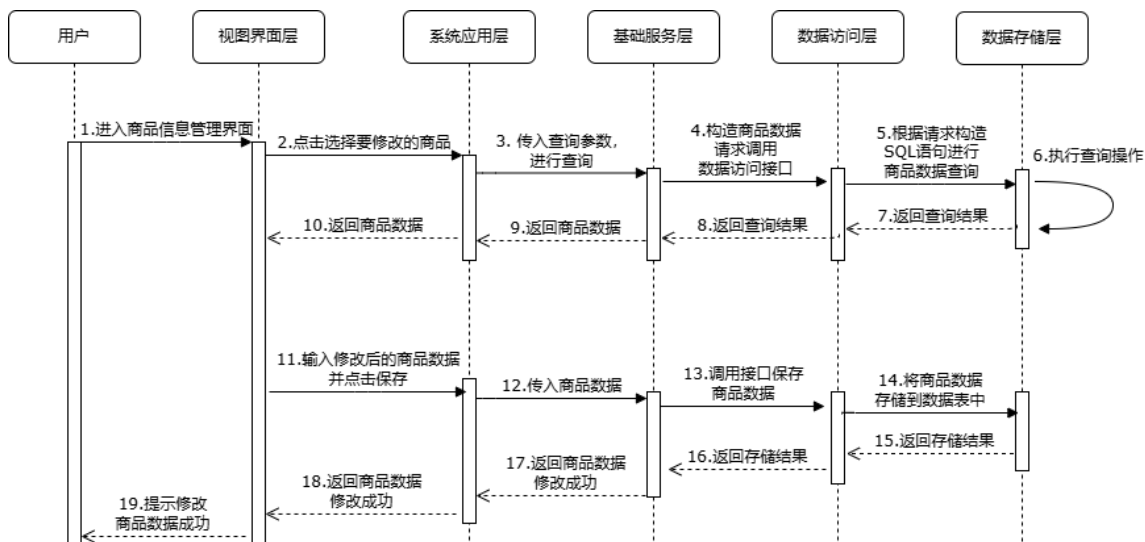


图 4-20 修改商品信息实现时序图

#### (5) 游戏角色管理模块

游戏角色管理模块主要用于调整游戏角色的属性值，包括角色外观、生命值和得分率的强化，流程图如图 4-21 所示。在本页面，用户可以对生命值和得分率进行强化来增加对应属性的数值，并根据自身需求决定是否要对数据进行重置。在完成相关属性的调整后，系统会保存修改。

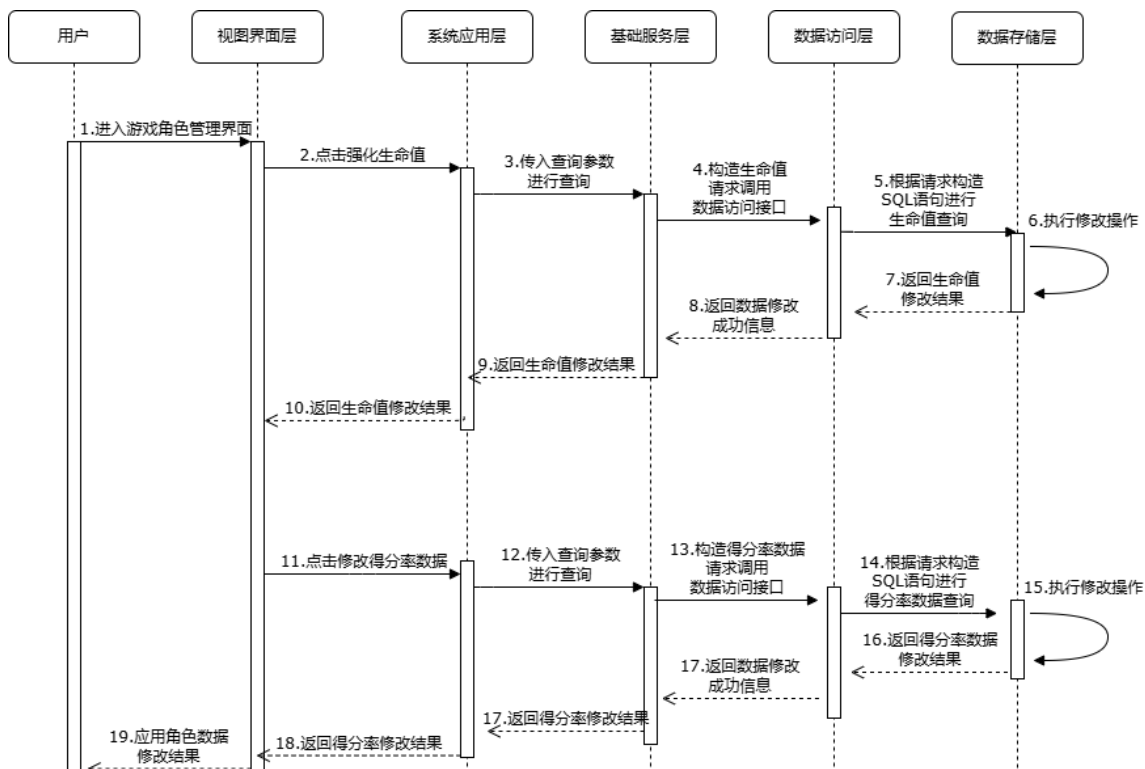


图 4-21 游戏角色管理模块实现时序图

#### (6) 游戏基本功能

首先是游戏的角色设计，游戏角色是用户控制的主体，它一共包含了四个状态，分别是站立、移动、跳跃和受击，在游戏内容初始化后游戏角色默认是站立态。游戏角色接受用户输入在站立、移动和跳跃间进行状态切换。当该角色受到场景敌人撞击则会切换至受击态，在无敌帧结束后会切换回站立态，状态转换如图 4-22 所示。

接着是敌人的设计，敌人一共包含了三个状态，分别是死亡、行走和转向，状态转换如图 4-23 所示。在完成创建后敌人默认是行走态，它会一直朝着当前方向移动直至撞击障碍，此时会切换至转向态，完成转向后则切换回行走态。当敌人遭遇玩家角色踩踏，它会播放受击动画，然后切换至受击态。若受击后生命值大于 0，则敌人会切换回行走态。

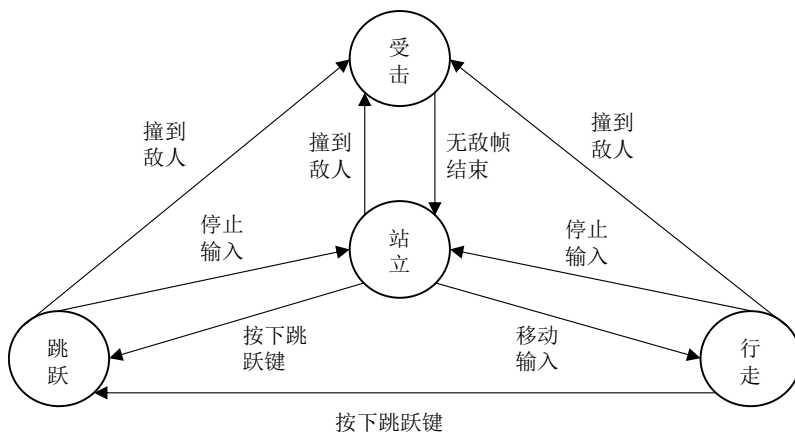


图 4-22 游戏角色状态转换图

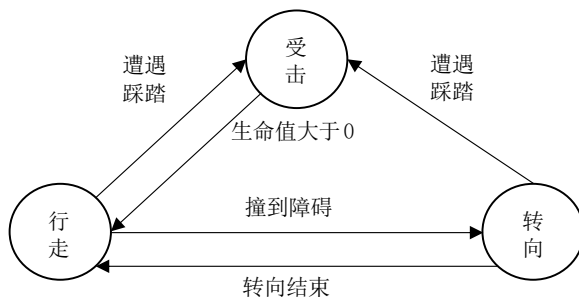


图 4-23 敌人状态转换图

#### 4.4.5 系统管理模块

系统管理模块主要用于用户身份验证与权限管理，通过登录验证保障系统安全，依据用户角色分配权限以规范使用，记录登录与操作日志以便追溯，支持有操作权限用户执行任务，确保系统有序运行。该模块的实现流程如图 4-24 所示。

该模块首先进行用户登录操作，接着验证用户名和密码，判断是否成功登录。

若登录不成功，则写入登录日志后流程结束；若登录成功，则写入登录日志，然后获取用户角色，用户发起操作请求，获取角色对应权限。再判断是否有操作权限，若没有操作权限，则输出提示“无操作权限”后流程结束；若有操作权限，则执行用户操作，最后写入操作日志并结束流程。

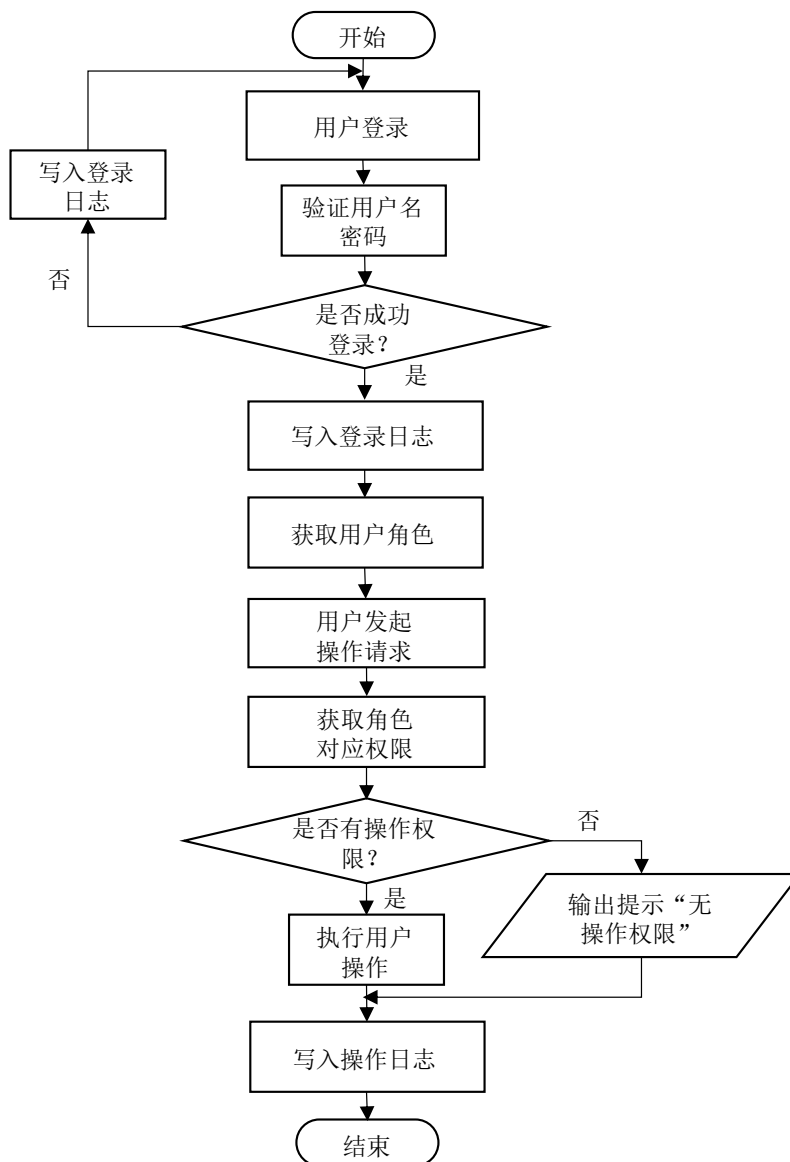


图 4-24 系统管理模块实现流程图

## 4.5 本章小结

本章将第 2 至第 3 章中的关卡段修复方法、关卡段生成方法及长关卡构建策略应用于实际设计了基于 PCG 的关卡生成游戏系统。首先，通过对系统需求的深入分析，明确了系统的核心功能需求，包括关卡生成、关卡修复及关卡游玩等关

键功能，同时对不同角色用户的权限进行了有效的规范与管理。在此基础上，系统采用分层架构进行设计，充分实现各模块之间的解耦，并能够满足高性能、准确性、实时性、鲁棒性、稳定性与可扩展性等非功能性要求，从而为系统在实际游戏开发和后续维护中提供了技术支撑。随后，详细阐述了系统的业务流程、整体架构及数据库设计。最后，围绕系统的关键功能模块，明确了各模块的功能流程与交互逻辑，主要包括关卡修复模块、关卡生成模块、模型管理模块、关卡游玩模块及系统管理模块。这些模块构成了系统的核心功能体系，也为未来功能拓展与系统迭代提供了良好的基础框架。

## 第5章 关卡生成游戏系统测试

本章在实现基于 PCG 的关卡生成游戏系统的基础上，完成了对系统各功能模块的详细测试。测试主要围绕第五章中设计的五个功能模块进行，旨在评估各子模块的完整性和可用性。通过系统测试，能够及时识别潜在问题，有助于修复漏洞，从而确保系统的稳定运行。

### 5.1 关卡生成游戏系统测试环境

测试工作在 64 位 Ubuntu 16.04 操作系统环境下进行，具体的测试环境参数如表 5-1 所示。

表 5-1 系统测试环境表

| 测试环境   | 说明                                             |
|--------|------------------------------------------------|
| 操作系统   | Windows 10-64 位                                |
| CPU    | 11th Gen Intel(R) Core(TM) i7-11800H @ 2.30GHz |
| 显卡设备   | NVIDIA GeForce RTX 3090                        |
| 开发语言   | Python 3.8.19、Java1.8.0                        |
| 深度学习框架 | Pytorch1.7.1, Gym0.18.0                        |
| 开发工具   | IDEA、Unity3D、Pycharm                           |
| 数据库工具  | Mysql、Navicat Premium                          |

### 5.2 系统功能模块测试

系统功能模块的测试侧重于验证基于 PCG 的关卡生成游戏系统的各个功能模块（关卡修复模块、关卡生成模块、模型管理模块、关卡游玩模块和系统管理模块）与游戏本体的功能是否按照需求分析的设计要求进行实现。在本系统涉及到的角色中，由于系统管理员角色拥有最高级的权限，所以采用系统管理员账号来进行本系统所有模块的功能测试。

#### 5.2.1 关卡修复模块测试

关卡修复模块是负责对生成关卡内容修复、查询和编辑的模块，包含了关卡内容修复功能、查询关卡修复数据功能以及对关卡编辑的功能，模块的测试用例和结果如表 5-2 所示。

表 5-2 关卡修复模块功能测试用例与结果表

| 测试用例      | 测试内容                                    | 预期结果                            | 测试结果 |
|-----------|-----------------------------------------|---------------------------------|------|
| 选择关卡段修复模型 | 点击页面关卡修复模型按钮选择关卡段修复模型                   | 正常弹出关卡修复模型选择对话框, 模型列表显示正常       | 正确   |
| 选择要修复的关卡段 | 点击页面选择关卡段按钮选择要修复的关卡段                    | 正常弹出关卡段选择对话框, 关卡段列表显示正常         | 正确   |
| 修复可视化查询   | 点击修复可视化查询按钮, 得到关卡修复数据列表                 | 关卡修复数据列表展示正常, 数据完善              | 正确   |
| 修复数据文本查询  | 点击关卡修复数据文本名称, 打开文本文件                    | 文本文件展示正常, 数据完善                  | 正确   |
| 修复数据图片查询  | 点击关卡修复数据的查看按钮, 打开图片文件                   | 图片文件展示正常, 数据完善                  | 正确   |
| 关卡编辑      | 点击关卡编辑按钮                                | 跳转至关卡编辑界面                       | 正确   |
| 选择关卡      | 点击选择关卡按钮, 从关卡列表选择关卡                     | 关卡列表展示正常, 数据完善, 选择关卡后正常加载地图文件   | 正确   |
| 编辑图块      | 在左侧地图文件上选择要修改的图块, 点击右侧选择图块按钮, 从图块列表选择图块 | 图块列表展示正常, 数据完善, 选择图块后地图文件的图块被替换 | 正确   |
| 保存编辑关卡    | 点击保存按钮                                  | 编辑好的关卡成功保存到本地                   | 正确   |

关卡内容修复界面如图 5-1 所示, 用户可以在这里选择训练好的关卡段修复模型来对选择的关卡文件进行修复。修复可视化查询界面如图 5-2 所示, 用户可以在此界面查询关卡修复数据文件。在修复可视化查询界面下, 点击文本名称可以打开对应关卡的查看文本文件, 效果如图 5-3 所示。点击查看按钮可以查看图片文件, 效果如图 5-4 所示。在关卡编辑界面可以对关卡文件进行编辑, 效果如



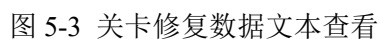
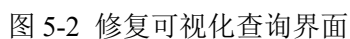
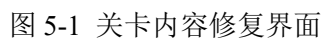




图 5-4 关卡修复数据图片查看



图 5-5 关卡编辑界面

5.2.2 关卡生成模块测试

关卡生成模块包含了关卡内容生成功能、查询生成关卡功能、关卡主题管理功能、关卡任务管理功能、关卡事件管理功能和关卡库管理功能，本模块的测试用例和结果如表 5-3 所示。

表 5-3 关卡生成模块功能测试用例与结果表

| 测试用例      | 测试内容                 | 预期结果                           | 测试结果 |
|-----------|----------------------|--------------------------------|------|
| 选择关卡段生成模型 | 点击页面关卡段生成模型按钮，选择生成模型 | 弹出关卡段生成模型选择对话框，生成模型列表展示正常，数据完善 | 正确   |

表 5-3（续表）

| 测试用例      | 测试内容                                           | 预期结果                           | 测试结果 |
|-----------|------------------------------------------------|--------------------------------|------|
| 选择关卡段修复模型 | 点击页面关卡段修复模型按钮，选择关卡段修复模型                        | 弹出关卡段修复模型选择对话框，修复模型列表展示正常，数据完善 | 正确   |
| 进行关卡生成    | 点击页面的进行关卡生成按钮                                  | 界面的文本框展示关卡生成过程的日志，功能正常         | 正确   |
| 生成关卡查询    | 点击生成关卡查询按钮，得到生成关卡列表                            | 生成关卡列表展示正常，数据完善                | 正确   |
| 生成关卡文本查询  | 点击关卡文本名称，打开文本文件                                | 文本文件展示正常，数据完善                  | 正确   |
| 生成关卡图片查询  | 点击关卡图片名称，打开图片文件                                | 图片文件展示正常，数据完善                  | 正确   |
| 关卡主题新增与维护 | 点击新增关卡主题按钮，输入新增的关卡主题数据并提交；点击编辑/删除按钮编辑/删除选中关卡主题 | 成功上传新增关卡主题数据；对关卡主题数据编辑、删除操作成功  | 正确   |
| 关卡任务新增与维护 | 点击新增关卡任务按钮，输入新增的关卡任务数据并提交；点击编辑/删除按钮编辑/删除选中关卡任务 | 成功上传新增关卡任务数据；对关卡任务数据编辑、删除操作成功  | 正确   |
| 关卡事件新增与维护 | 点击新增关卡事件按钮，输入新增的关卡事件数据并提交；点击编辑/删除按钮编辑/删除选中关卡事件 | 成功上传新增关卡事件数据；对关卡事件数据编辑、删除操作成功  | 正确   |
| 关卡库新增与维护  | 点击新增按钮，输入新增的关卡库数据并提交；点击编辑/删除按钮编辑/删除选中数据        | 成功上传新增关卡库数据；对关卡库数据编辑、删除操作成功    | 正确   |

关卡生成界面如图 5-6 所示，用户可以在这里选择关卡段生成模型和关卡段修复模型，选择完毕后点击页面的关卡生成按钮即可进行关卡的生成。生成关卡查询界面如图 5-7 所示，用户可以在此界面查询生成的关卡文件。



图 5-6 关卡生成界面



图 5-7 生成关卡查询界面



图 5-8 新增关卡主题

在关卡主题管理界面，用户可以对关卡主题数据进行新增与维护，新增关卡主题如图 5-8 所示。在关卡任务管理界面可以对游戏内的关卡任务进行新增与维护，界面如图 5-9 所示。



图 5-9 关卡任务管理界面



图 5-10 关卡事件管理界面



图 5-11 关卡库管理界面

在关卡事件管理界面可以对游戏内的关卡事件数据进行新增与维护，界面如图 5-10 所示。关卡库管理界面用于新增与维护游戏内的关卡数据，可以在本页面将生成好的关卡添加进游戏内，界面如图 5-11 所示。

### 5.2.3 模型管理模块测试

模型管理模块测试用例和结果如表 5-4 所示。

表 5-4 模型管理模块功能测试用例与结果表

| 测试用例      | 测试内容                          | 预期结果                    | 测试结果 |
|-----------|-------------------------------|-------------------------|------|
| 选择关卡段修复模型 | 点击关卡段修复模型按钮，选择关卡段修复模型         | 弹出模型选择对话框，模型列表显示正常，数据完善 | 正确   |
| 生成测试数据    | 点击生成测试数据按钮，生成数据               | 生成数据功能执行正常，生成成功         | 正确   |
| 模型训练      | 点击模型训练按钮，训练模型                 | 模型正常训练，将训练日志打印至对话框内     | 正确   |
| 模型库数据维护   | 点击编辑按钮单行编辑模型库信息；点击删除按钮删除模型库数据 | 对模型库数据编辑、删除操作成功         | 正确   |

关卡生成模型训练界面如图 5-12 所示，用户可以在这里选择关卡段生成模型的训练数据集和关卡段修复模型。关卡修复模型训练界面如图 5-13 所示，用户可以在这里生成模型训练所需的测试数据并点按钮进行模型训练。



图 5-12 关卡生成模型训练界面



图 5-13 关卡修复模型训练界面

## 5.2.4 关卡游玩模块测试

关卡游玩模块包含了和游戏游玩相关的 5 个子模块，在关卡选择界面点击开始游玩后会加载游戏本体。首先对游戏本体进行测试，旨在确保角色控制以及与场景交互等功能正常，测试用例和结果如表 5-5 所示。

表 5-5 游戏功能测试用例与结果表

| 测试用例   | 测试内容                | 预期结果                | 测试结果 |
|--------|---------------------|---------------------|------|
| 开始游戏   | 点击开始游戏按钮            | 显示加载界面，之后游戏内容正常加载   | 正确   |
| 退出游戏   | 点击退出按钮              | 退出游戏界面，返回关卡选择界面     | 正确   |
| 角色移动   | 键盘控制角色前后移动，空格控制角色跳跃 | 角色可以按照玩家输入指令正确移动与跳跃 | 正确   |
| 角色碰撞砖块 | 在可破坏砖块下用角色跳跃        | 角色可以成功顶碎可破坏砖块       | 正确   |
| 敌人行为测试 | 角色移动到敌人生成地点，观察敌人行为  | 敌人会在障碍之间往返移动        | 正确   |
| 角色碰撞敌人 | 角色移动到敌人面前让敌人撞击      | 角色扣除生命值并进入 2 秒无敌状态  | 正确   |
| 角色踩踏敌人 | 控制角色跳跃到敌人上方踩踏       | 敌人播放被踩动画，然后从场景移除    | 正确   |



表 5-5（续表）

| 测试用例   | 测试内容                        | 预期结果                          | 测试结果 |
|--------|-----------------------------|-------------------------------|------|
| 角色拾取金币 | 角色移动至金币<br>位置撞击拾取           | 金币消失,角色金币<br>计数加 1            | 正确   |
| 角色拾取道具 | 角色移动至道具<br>位置撞击拾取           | 道具消失,角色应用<br>道具效果             | 正确   |
| 游戏暂停   | 在游戏运行状态<br>下,键盘按下 ESC<br>按键 | 游戏内容暂停,弹出<br>游戏暂停菜单           | 正确   |
| 角色复活   | 控制角色从平台<br>掉下               | 角色生命值减 1, 同<br>时角色从关卡起点<br>复活 | 正确   |
| 游戏结束   | 控制角色使其生<br>命值归 0            | 游戏结束,弹出游戏<br>结束菜单             | 正确   |

游戏本体的主界面如图 5-14 所示，用户可以在这里选择开始游戏或者选择退出游戏。当用户选择了开始游戏，则对游戏内容进行加载并弹出加载菜单，当加载完毕后即显示游戏内容。在游戏过程中，用户控制角色撞击头顶方块，方块破碎并掉落道具蘑菇，效果如图 5-15 所示。当遇到敌人后，控制角色跳跃至它上方并对其进行踩踏，敌人会播放受击动画并从场景中清除消失，效果如图 5-16 所示。在游戏中，用户可以随时对游戏进度进行暂停，在按下 ESC 按键后游戏内容会暂停，接着会弹出暂停菜单，用户可以选择是继续游戏还是直接退出游玩。



图 5-14 游戏主界面





图 5-15 角色撞击方块掉落蘑菇



图 5-16 角色跳跃踩踏敌人

完成对游戏本体功能测试后，接着对本模块的其他子功能进行测试，测试用例和结果如表 5-6 所示。

表 5-6 关卡游玩模块功能测试用例与结果表

| 测试用例   | 测试内容             | 预期结果                | 测试结果 |
|--------|------------------|---------------------|------|
| 选择关卡   | 点击页面关卡列表选择要游玩的关卡 | 关卡列表展示正常，数据完善       | 正确   |
| 开始游玩   | 点击开始游玩按钮         | 正常加载游戏本体主界面         | 正确   |
| 音频设置面板 | 点击音频设置按钮         | 音频设置面板显示正常，数据完善     | 正确   |
| 调节音乐音量 | 滑动音乐进度条，调整音量     | 音乐大小根据进度条移动而变化，功能正常 | 正确   |

表 5-6 (续表)

| 测试用例      | 测试内容                                                    | 预期结果                       | 测试结果 |
|-----------|---------------------------------------------------------|----------------------------|------|
| 调节音效音量    | 滑动音效进度条, 调整音频                                           | 音效大小根据进度条移动而变化, 功能正常       | 正确   |
| 上传音乐      | 点击上传音乐按钮, 在对话框选择音乐上传                                    | 成功上传背景音乐, 功能正常             | 正确   |
| 上传音效      | 点击上传音效按钮, 在对话框选择音效上传                                    | 成功上传游戏音效, 功能正常             | 正确   |
| 查看商品详情    | 点击详情按钮, 打开对话框显示商品数据信息                                   | 商品数据信息显示正常, 数据完善           | 正确   |
| 购买商品      | 点击选择好的商品购买按钮                                            | 金币扣除成功, 商品数量减 1, 购买成功      | 正确   |
| 商品数据新增与维护 | 点击新增商品按钮弹出对话框, 输入新增的商品数据并提交; 点击编辑按钮编辑商品数据; 点击删除按钮删除商品数据 | 成功上传新增商品数据; 对商品数据编辑、删除操作成功 | 正确   |
| 皮肤替换      | 点击角色替换皮肤按钮打开皮肤列表, 选择一款皮肤装扮                              | 皮肤列表显示正常, 角色更换皮肤成功         | 正确   |
| 生命值强化     | 点击生命值强化按钮                                               | 生命值强化数字增加, 功能正常            | 正确   |
| 生命值重置     | 点击生命值重置按钮                                               | 生命值重置成功                    | 正确   |
| 得分率强化     | 点击得分率强化按钮                                               | 得分率强化数字增加, 功能正常            | 正确   |
| 得分率重置     | 点击得分率重置按钮                                               | 得分率重置成功                    | 正确   |

关卡选择界面如图 5-17 所示, 用户可以在这里选择要游玩的关卡内容。点击开始游玩按钮后, 加载游戏本体, 运行界面如图 5-14 所示。点击音频设置按钮跳转至音频设置界面, 效果如图 5-18 所示, 可以在本界面可以管理音乐和音效。游戏商店页面展示了可以购买的商品, 效果如图 5-19 所示。商品信息管理界面可以对商品信息进行添加和维护, 效果如图 5-20 所示。



图 5-17 关卡选择界面



图 5-18 音频设置界面



图 5-19 游戏商店页面

点击游戏角色管理按钮跳转至游戏角色管理界面，如图 5-21 所示，可以在本界面替换角色皮肤、强化角色生命值和得分率。



图 5-20 商品信息管理界面



图 5-21 游戏角色管理界面

5.2.5 系统管理模块测试

系统管理模块测试用例和测试的结果如表 5-7 所示。

表 5-7 系统管理模块功能测试用例与结果表

| 测试用例     | 测试内容                              | 预期结果                 | 测试结果 |
|----------|-----------------------------------|----------------------|------|
| 权限管理页面展示 | 点击权限管理按钮<br>显示权限界面内容              | 权限界面的内容展<br>示正常，数据完整 | 正确   |
| 新增子权限    | 点击新增子权限按<br>钮，填写子权限信<br>息，完成子权限创建 | 新增子权限成功              | 正确   |

表 5-7（续表）

| 测试用例     | 测试内容                              | 预期结果                  | 测试结果 |
|----------|-----------------------------------|-----------------------|------|
| 角色管理页面展示 | 点击角色管理按钮<br>显示权限界面内容<br>点击新增按钮,填写 | 角色管理界面内容<br>展示正常,数据完整 | 正确   |
| 新增角色     | 新角色的信息,完成<br>新角色创建                | 新增角色成功                | 正确   |
| 用户管理界面展示 | 点击用户管理按钮<br>显示用户界面内容<br>点击新增按钮,填写 | 用户管理界面内容<br>展示正常,数据完整 | 正确   |
| 新增用户     | 新用户的信息,完成<br>新用户创建                | 新增用户成功                | 正确   |
| 日志管理界面展示 | 点击日志管理按钮<br>显示日志界面内容              | 日志管理界面内容<br>展示正常,数据完整 | 正确   |



图 5-22 权限管理界面



图 5-23 角色管理界面

系统的权限管理界面如图 5-22 所示，角色管理界面如图 5-23 所示，用户管理界面如图 5-24 所示，日志管理界面如图 5-25 所示。



图 5-24 用户管理界面



图 5-25 日志管理界面

### 5.3 本章小结

本章基于已实现的基于 PCG 的关卡生成游戏系统，对游戏本体功能和各个功能模块进行了全面的测试与验证。测试过程中，针对系统设计阶段所定义的功能需求，构建了测试用例并逐项对各功能模块进行详细评估，测试结果以表格形式进行汇总与展示。每个测试用例均对应系统中的具体功能，旨在验证各模块是否能够在实际运行中准确执行预定任务，并满足相应的设计标准。从测试结果来看，系统的所有功能模块均能够稳定运行，实现了设计所期望的目标，为系统的实际部署与后续优化提供了有力保障。

## 结 论

本研究聚焦平台跳跃游戏关卡的生成,针对现有研究存在的不足,分别从关卡段修复、关卡段生成和长关卡构建等方面展开了深入的研究,最后结合研究内容构建了关卡生成游戏系统,取得主要成果如下:

针对生成的关卡中常见的错误图块问题,本文提出了一种结合 LevelErrorNet 与遗传算法的关卡段修复方法。通过 LevelErrorNet 可以高效识别出错误的图块,在此基础上利用遗传算法进行图块的合理替换。实验通过 KL 散度对修复前后的效果进行评价,结果表明关卡段修复方法在保留原有关卡的设计风格方面要优于现有的方法。

针对传统关卡生成方法中普遍存在的模板化问题,本研究提出了一种结合 DCNN-DCGAN 和 QD 算法的关卡段生成方法,在完成对 DCNN-DCGAN 的训练后,通过 CMA-ME 算法优化潜在向量搜索过程来提高生成关卡的质量。采用了覆盖场景百分比和 KL 散度两个指标对生成关卡进行评价,实验结果表明,所提出的方法能够生成具有可玩性和多样性的关卡段,和现有方法生成的关卡相比本文方法生成的关卡更具有多样性。

针对现有关卡生成方法在长关卡生成方面表现不佳的问题,本文提出了基于强化学习的长关卡构建策略。通过设计可玩性、机制量与重复度三种奖励函数,结合 A\*代理进行可玩性测试,优化了长关卡的生成与拼接过程。实验结果表明,同时考虑了三种奖励函数的代理在生成可玩且机制量和重复度适中的长关卡方面有着较好的表现,为长关卡生成提供了新方法。

在系统实现方面,本文基于前述研究成果,设计并实现了一个完整的关卡生成游戏系统,并针对系统功能模块和游戏本体内容进行了测试。测试结果表明,系统的各项功能均能按照设计要求稳定运行。

本研究的工作期望如下:

(1) 优化与扩展关卡生成算法的多样性与创意性:本研究提出的关卡段生成方法已在一定程度上提升了关卡的可玩性与多样性,未来的工作期望进一步优化现有方法,期望能通过引入更多的深度学习算法,实现更加个性化的关卡生成。

(2) 提升系统的实时性与可扩展性:尽管本研究设计的关卡生成游戏系统已具备较高的稳定性与功能性,未来的工作期望在此基础上进一步提升系统的实时性,尤其是在生成长关卡时,如何在保证高质量生成的同时减少计算时间,将是一个重要的研究方向。

## 参考文献

- [1] Milani S, Juliani A, Momennejad I, et al. Navigates like me: Understanding how people evaluate human-like ai in video games[C]// Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems. New York: ACM, 2023: 1-18.
- [2] Kaur A, Behki P. The Evolution and Impact of Game Development in the Digital Era[C]// 2024 International Conference on Emerging Technologies in Computer Science for Interdisciplinary Applications (ICETCS). Piscataway: IEEE, 2024: 1-6.
- [3] Lozano-Osorio I, Sánchez-Oro J, Duarte A. An efficient and effective grasp algorithm for the budget influence maximization problem[J]. Journal of Ambient Intelligence and Humanized Computing, 2024, 15(3): 2023-2034.
- [4] Sestini A, Bergdahl J, Tollmar K, et al. Towards informed design and validation assistance in computer games using imitation learning[C]// 2023 IEEE Conference on Games (CoG). Piscataway: IEEE, 2023: 1-8.
- [5] Ashby T, Webb B K, Knapp G, et al. Personalized quest and dialogue generation in role-playing games: A knowledge graph-and language model-based approach[C]// Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems. New York: ACM, 2023: 1-20.
- [6] Fernandes P M, Lopes M, Prada R. Generating game levels by defining player experiences[C]// Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment. Palo Alto: AAAI, 2024, 20(1): 179-188.
- [7] Obergh A. Discussion of Game Design and Construction of a Videogame Utilizing PCG, CA, and ABM[D]. Bard College, 2023: 14-16.
- [8] Ling P. AI-assisted Game Level Generation based on Difficulty[D]. Aalto University, 2023: 14-20.
- [9] Withington O, Cook M, Tokarchuk L. On the Evaluation of Procedural Level Generation Systems[C]// Proceedings of the 19th International Conference on the Foundations of Digital Games. New York: ACM, 2024: 1-10.
- [10] Dang H, Brudy F, Fitzmaurice G, et al. Worldsmith: Iterative and expressive prompting for world building with a generative ai[C]// Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology. New York: ACM, 2023: 1-17.
- [11] Nasir M U, Togelius J. Practical PCG through large language models[C]// 2



- 023 IEEE Conference on Games (CoG). Piscataway: IEEE, 2023: 1-4.
- [12]Shaker N, Yannakakis G N, Togelius J. Towards player-driven procedural content generation[C]// Proceedings of the 9th conference on Computing Frontiers. New York: ACM, 2012: 237-240.
- [13]Risi S, Togelius J. Increasing generality in machine learning through procedural content generation[J]. Nature Machine Intelligence, 2020, 2(8): 428-436.
- [14]Cardamone L, Loiacono D, Lanzi P L. Interactive evolution for the procedural generation of tracks in a high-end racing game[C]// Proceedings of the 13th annual conference on Genetic and evolutionary computation, New York: ACM, 2011: 395-402.
- [15]Loiacono D, Cardamone L, Lanzi P L. Automatic Track Generation for High-End Racing Games Using Evolutionary Computation[J]. IEEE Transactions on computational intelligence and AI in games, 2011, 3(3): 245-259.
- [16]Georgiou T, Demiris Y. Personalised track design in car racing games[C]// 2016 IEEE Conference on Computational Intelligence and Games (CIG). Piscataway: IEEE, 2016: 1-8.
- [17]Green M C, Mugrai L, Khalifa A, et al. Mario level generation from mechanics using scene stitching[C]// 2020 IEEE Conference on Games (CoG). Piscataway: IEEE, 2020: 49-56.
- [18]Liu J, Snodgrass S, Khalifa A, et al. Deep learning for procedural content generation[J]. Neural Computing and Applications, 2021, 33(1): 19-37.
- [19]Hauck E, Aranha C. Automatic generation of Super Mario levels via graph grammars[C]// 2020 IEEE Conference on Games (CoG). Piscataway: IEEE, 2020: 297-304.
- [20]Summerville A, Mateas M. Super Mario as a String: Platformer Level Generation Via LSTMs[C]// Proceedings of DiGRA/FDG 2016 Conference. New York: ACM,2016: 1-16.
- [21]Frade M, De Vega F F, Cotta C. Modelling video games' landscapes by means of genetic terrain programming: a new approach for improving users' experience[C]// Proceedings of the 2008 conference on Applications of evolutionary computing, Berlin: Springer, 2008: 485-490.
- [22]van Aanholt L, Bidarra R. Declarative procedural generation of architecture with semantic architectural profiles[C]// 2020 IEEE Conference on Games (CoG). Piscataway: IEEE, 2020: 351-358.
- [23]Freiknecht J, Effelsberg W. Procedural generation of multistory buildings with interior[J]. IEEE Transactions on Games, 2019, 12(3): 323-336.
- [24]Mark B, Berechet T, Mahlmann T, et al. Procedural Generation of 3D Caves for Games on the GPU[C]// Foundations of Digital Games. California: Society for the Advancement of the Science of Digital Games, 2015: 1-9.

- [25]Togelius J, Karakovskiy S, Baumgarten R. The 2009 mario ai competition[C] // IEEE Congress on Evolutionary Computation. Piscataway: IEEE, 2010: 1-8.
- [26]Jain R, Isaksen A, Holmgård C, et al. Autoencoders for level generation, repair, and recognition[C]// Proceedings of the ICCG workshop on computational creativity and games. Palo Alto: Association for Computational Creativity, 2016: 1-9.
- [27]Mott J, Nandi S, Zeller L. Controllable and coherent level generation: A two-pronged approach[C]// Experimental AI in games workshop. New York: E XAG, 2019: 1-7.
- [28]Agahian S, Akan T. Battle royale optimizer for training multi-layer perceptron[J]. *Evolving Systems*, 2022, 13(4): 563-575.
- [29]Nadimi-Shahraki M H, Zamani H, Asghari Varzaneh Z, et al. A systematic review of the whale optimization algorithm: theoretical foundation, improvements, and hybridizations[J]. *Archives of Computational Methods in Engineering*, 2023, 30(7): 4113-4159.
- [30]Summerville A, Snodgrass S, Guzdial M, et al. Procedural content generation via machine learning (PCGML)[J]. *IEEE Transactions on Games*, 2018, 10(3): 257-270.
- [31]Shu T, Wang Z, Liu J, et al. A novel cnet-assisted evolutionary level repairer and its applications to Super Mario Bros[C]// 2020 IEEE Congress on Evolutionary Computation (CEC). Piscataway: IEEE, 2020: 1-10.
- [32]Lucas S M, Volz V. Tile pattern KL-divergence for analysing and evolving game levels[C]// Proceedings of the Genetic and Evolutionary Computation Conference. New York: ACM, 2019: 170-178.
- [33]Green M C, Mugrai L, Khalifa A, et al. Mario level generation from mechanics using scene stitching[C]// 2020 IEEE Conference on Games (CoG). Piscataway: IEEE, 2020: 49-56.
- [34]Chan C K Y, Hu W. Students' voices on generative AI: Perceptions, benefits, and challenges in higher education[J]. *International Journal of Educational Technology in Higher Education*, 2023, 20(1): 43.
- [35]Hasan B M S, Mstafa R J. A study of gender classification techniques based on iris images: A deep survey and analysis[J]. *Science Journal of University of Zakho*, 2022, 10(4): 222-234.
- [36]Murali V, Sudeep P V. Image denoising using DnCNN: An exploration study[C]// Advances in Communication Systems and Networks: Select Proceedings of ComNet 2019. Singapore: Springer, 2020: 847-859.
- [37]Gravina D, Khalifa A, Liapis A, et al. Procedural content generation through quality diversity[C]// 2019 IEEE Conference on Games (CoG). Piscataway:

- IEEE, 2019: 1-8.
- [38]Shu T, Liu J, Yannakakis G N. Experience-driven PCG via reinforcement learning: A Super Mario Bros study[C]// 2021 IEEE Conference on Games (CoG). Piscataway: IEEE, 2021: 1-9.
- [39]Cuerdo M, Baskaran D, Melcer E. Exploring how emotional challenge and affective design in games relates to player reflection[C]// Proceedings of the 19th international conference on the foundations of digital games. New York: ACM, 2024: 1-12.
- [40]Du Y, Watkins O, Wang Z, et al. Guiding pretraining in reinforcement learning with large language models[C]// International Conference on Machine Learning. PMLR, 2023: 8657-8677.
- [41]Khalifa A, Bontrager P, Earle S, et al. Pcgrl: Procedural content generation via reinforcement learning[C]// Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment. Palo Alto: AAAI, 2020, 16(1): 95-101.
- [42]Wu T, He S, Liu J, et al. A brief overview of ChatGPT: The history, status quo and potential future development[J]. IEEE/CAA Journal of Automatica Sinica, 2023, 10(5): 1122-1136.
- [43]Rajwar K, Deep K, Das S. An exhaustive review of the metaheuristic algorithms for search and optimization: taxonomy, applications, and open challenges[J]. Artificial Intelligence Review, 2023, 56(11): 13187-13257.
- [44]Ha H, Florence P, Song S. Scaling up and distilling down: Language-guided robot skill acquisition[C]// Conference on Robot Learning. PMLR, 2023: 3766-3777.
- [45]Pierrot T, Richard G, Beguir K, et al. Multi-objective quality diversity optimization[C]// Proceedings of the Genetic and Evolutionary Computation Conference. New York: ACM, 2022: 139-147.
- [46]Fontaine M C, Togelius J, Nikolaidis S, et al. Covariance matrix adaptation for the rapid illumination of behavior space[C]// Proceedings of the 2020 genetic and evolutionary computation conference, New York: ACM, 2020: 94-102.
- [47]Fontaine M C, Liu R, Khalifa A, et al. Illuminating mario scenes in the latent space of a generative adversarial network[C]// Proceedings of the AAAI Conference on Artificial Intelligence. Palo Alto: AAAI, 2021, 35(7): 5922-5930.
- [48]Pugh J K, Soros L B, Stanley K O. Quality diversity: A new frontier for evolutionary computation[J]. Frontiers in Robotics and AI, 2016, 3(40): 1-17.
- [49]于华宇. 基于双目视频的海参检测及定位系统设计与实现[D]. 哈尔滨工业大学, 2023: 50-54.